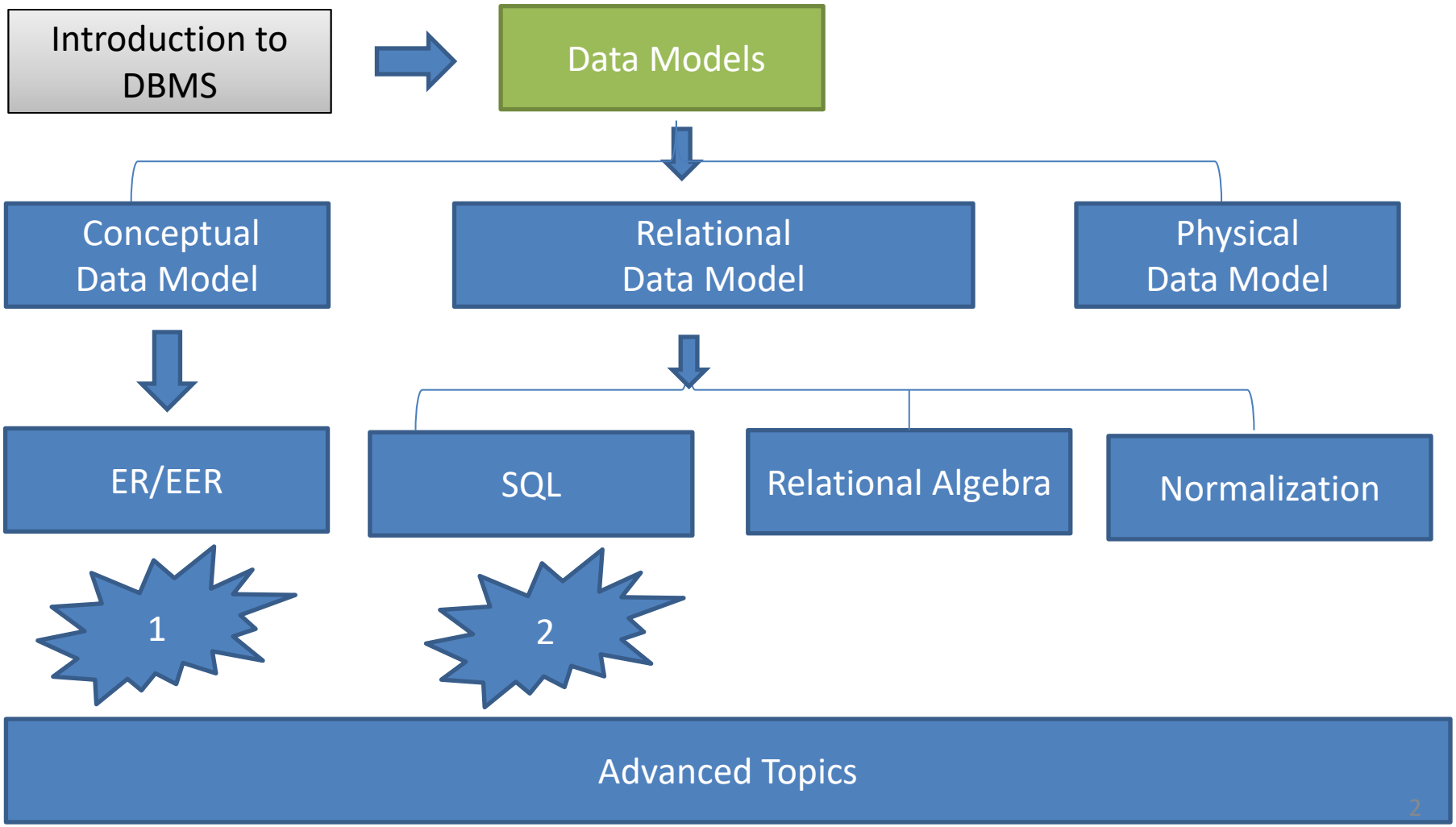
An Introduction to the Database Management Systems

By
Hossein Rahmani

Slides originally by Book(s) Resources

Road Map

(Might change!)



Database System Concepts And Architecture

- Data Models, Schemas, and Instances 
- Three-Schema Architecture and Data Independence
- Database Languages and Interfaces
- The Database System Environment
- Centralized and Client/Server Architectures for DBMSs
- Classification of Database Management Systems

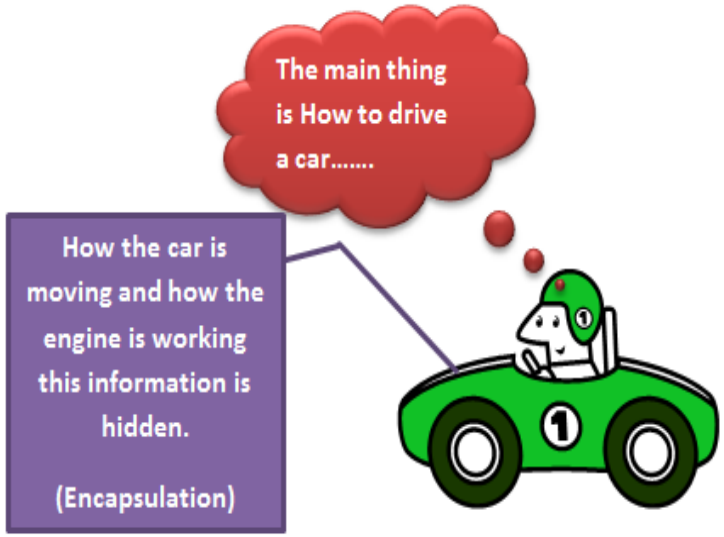
Data Models, Schemas, and Instances

- **Data abstraction**
  - Suppression of details of data organization and storage
  - Highlighting of the essential features for an improved understanding of data

    OR

  - Reduction of a particular body of data to a simplified representation of the whole

Data Abstraction



Data Models, Schemas, and Instances (cont'd.)

- **Data model**
  - Collection of concepts that describe the structure of a database
  - Provides means to achieve data abstraction
  - **Can Contain:**
    - **Basic operations** (Update, Delete)
    - **Dynamic aspect** or **behavior** of a database

      (user defined operations)

Data models

- **Conceptual** (high-level)
  - for end users / analists

    versus
- **Physical** (low-level)
  - for programmers
- In between: **Representational** (implementation) data model
  - Easily understood by end users
  - Also similar to how data organized in computer storage
  - Widely used Relational model

Feature	Conceptual	Logical	Physical
Table Names	No	No	Yes
Column Names	No	No	Yes
Column Data Types	No	No	Yes
Entity Names	Yes	Yes	No
Entity Relationships	Yes	Yes	No
Attributes	No	Yes	No
Primary Keys	No	Yes	Yes
Foreign Keys	No	Yes	Yes

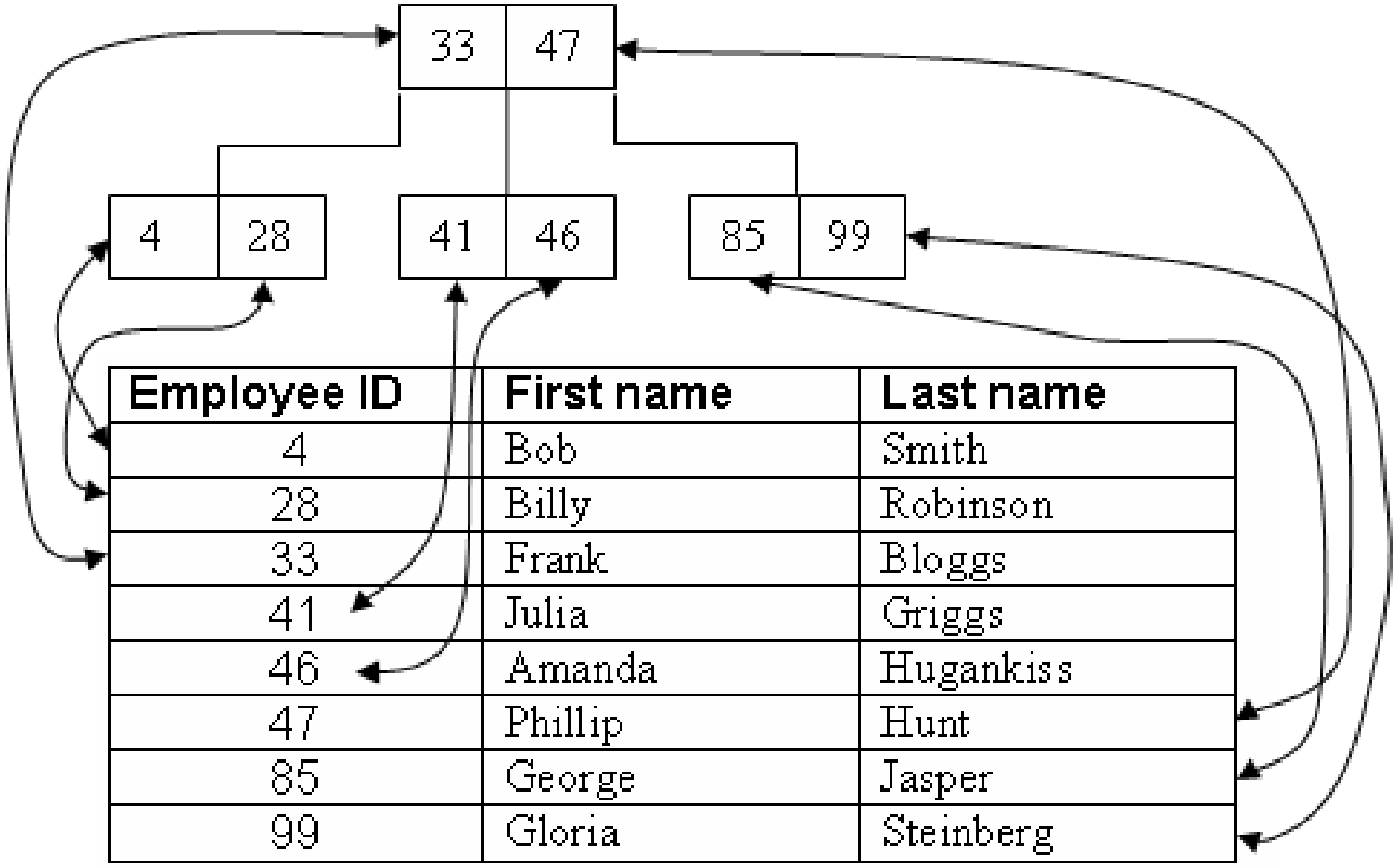
Representational data model

- Relational data model
- Network model
- Hierarchical model
- Object model
- …

Physical data model

- Describe how data is stored as files in the computer
- **Access path**
  - Structure that makes the search for particular database records efficient
- **Index**
  - Example of an access path
  - Allows direct access to data using an index term or a keyword

B-Tree Index

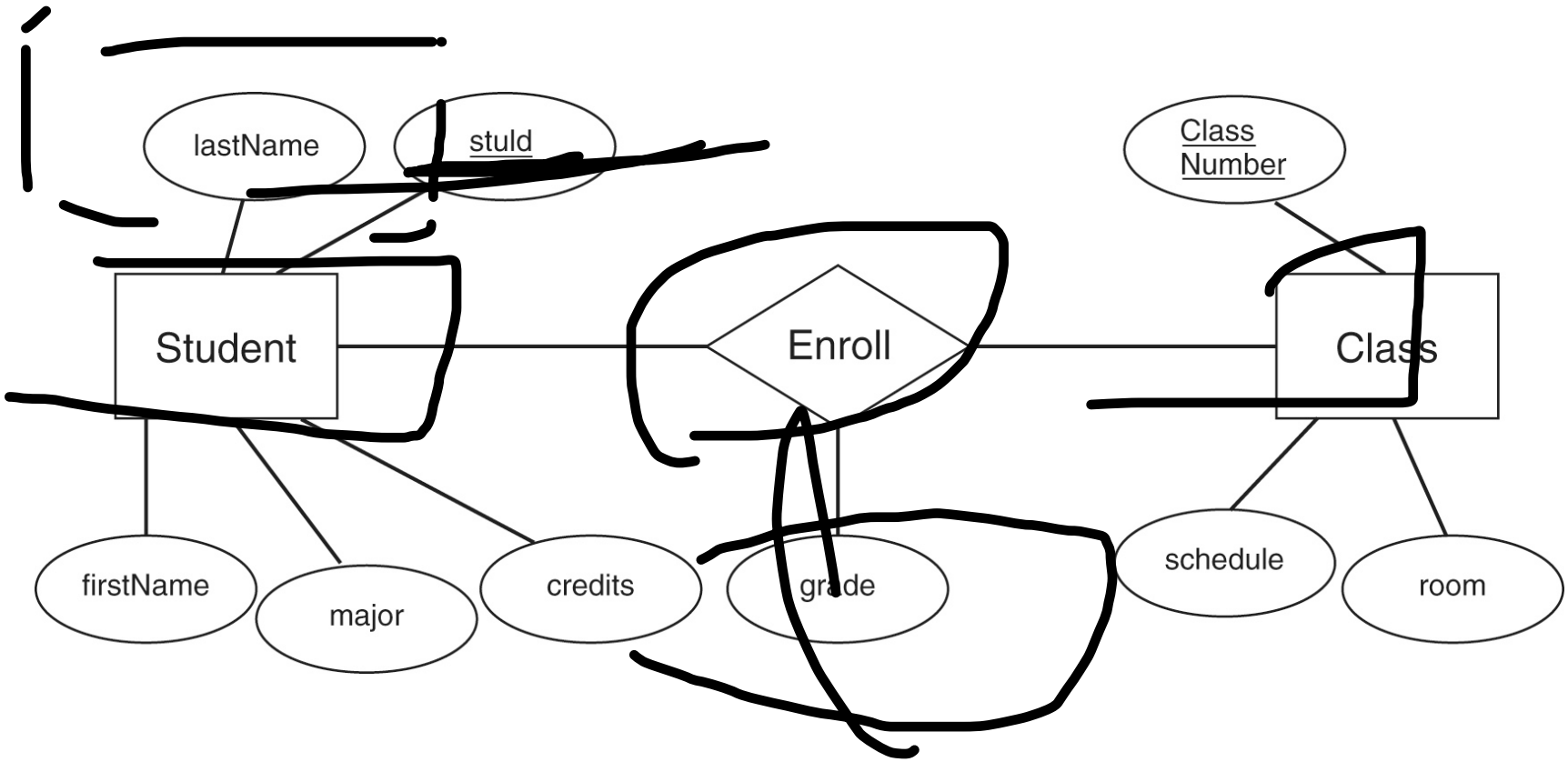


Employee ID	First name	Last name
4	Bob	Smith
28	Billy	Robinson
33	Frank	Bloggs
41	Julia	Griggs
46	Amanda	Hugankiss
47	Phillip	Hunt
85	George	Jasper
99	Gloria	Steinberg

Conceptual data model
Built around…

- **Entity**
  - Represents a real-world object or concept
- **Attribute**
  - Represents some property of interest
  - Further describes an entity
- **Relationship** among two or more entities
  - Represents an association among the entities
  - **Entity-Relationship model**

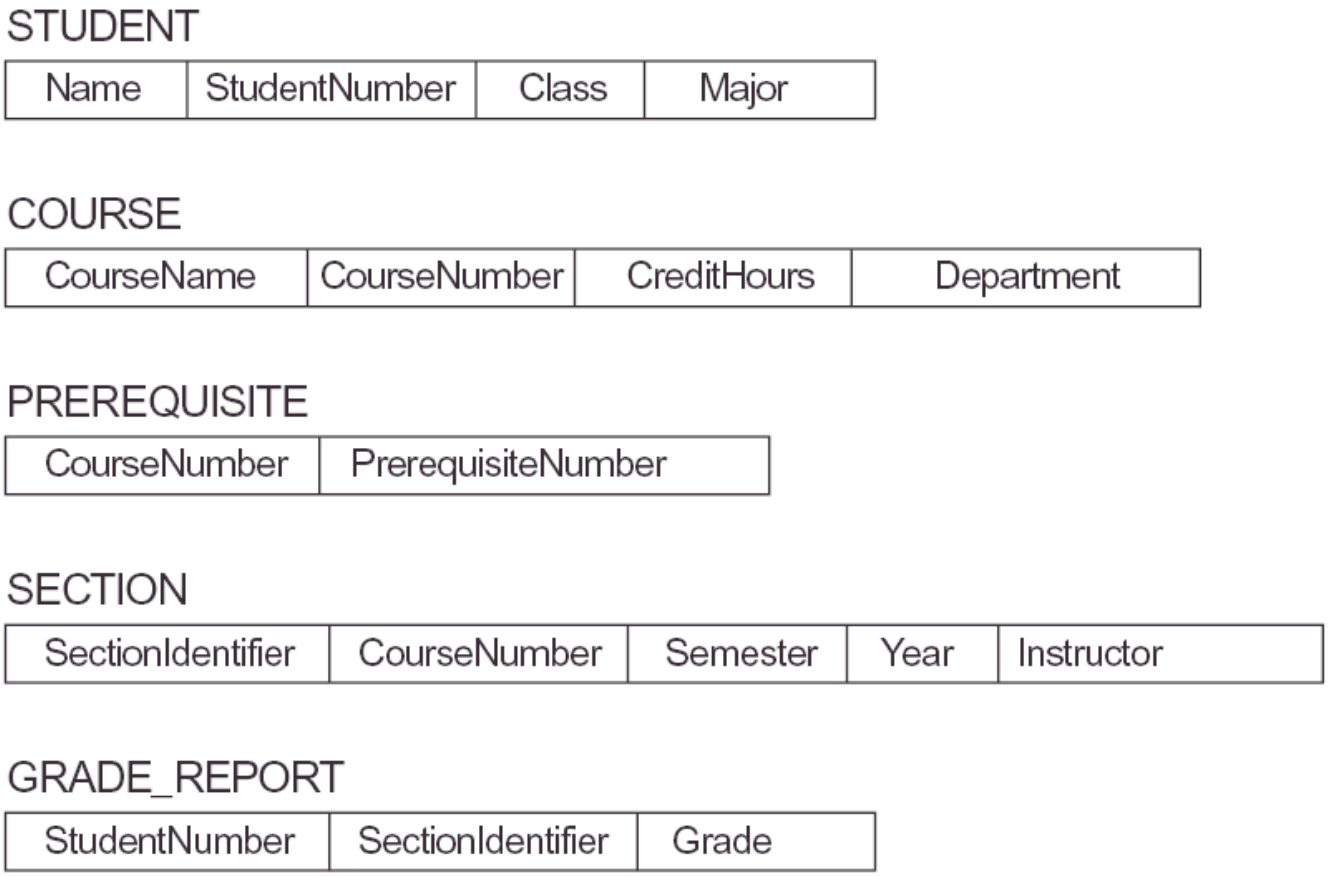
Sample ER Model



Schemas, Instances, and Database State

- **Database schema**
  - Description of a database
- **Schema diagram**
  - Displays selected aspects of schema
- **Schema construct**
  - Each object in the schema
- **Database state** or **snapshot**
  - Data in database at a particular moment in time
- These are stored in a **catalogue**

Database schema diagram



Schemas, Instances, and Database State

- A database schema is (relatively) fixed, but the **content** of a database varies:
  - Database state or *snapshot*
  - Collection of current instances

Database state

STUDENT	Name	StudentNumber	Class	Major
	Smith	17	1	CS
	Brown	8	2	CS

COURSE	CourseName	CourseNumber	CreditHours	Department
	Intro to Computer Science	CS1310	4	CS
	Data Structures	CS3320	4	CS
	Discrete Mathematics	MATH2410	3	MATH
	Database	CS3380	3	CS

SECTION	SectionIdentifier	CourseNumber	Semester	Year	Instructor
	85	MATH2410	Fall	98	King
	92	CS1310	Fall	98	Anderson
	102	CS3320	Spring	99	Knuth
	112	MATH2410	Fall	99	Chang
	119	CS1310	Fall	99	Anderson
	135	CS3380	Fall	99	Stone

GRADE_REPORT	StudentNumber	SectionIdentifier	Grade
	17	112	B
	17	119	C
	8	85	A
	8	92	A
	8	102	B
	8	135	A

PREREQUISITE	CourseNumber	PrerequisiteNumber
	CS3380	CS3320
	CS3380	MATH2410
	CS3320	CS1310

Schemas, Instances, and Database State (cont'd.)

- **Define** a new database
  - Specify database schema to the DBMS
- **Initial state**
  - **Populated** or **loaded** with the initial data
- **Valid state**
  - Satisfies the structure and constraints specified in the schema

Schemas, Instances, and Database State (cont'd.)

- **Schema evolution**
  - Changes applied to schema as application requirements change

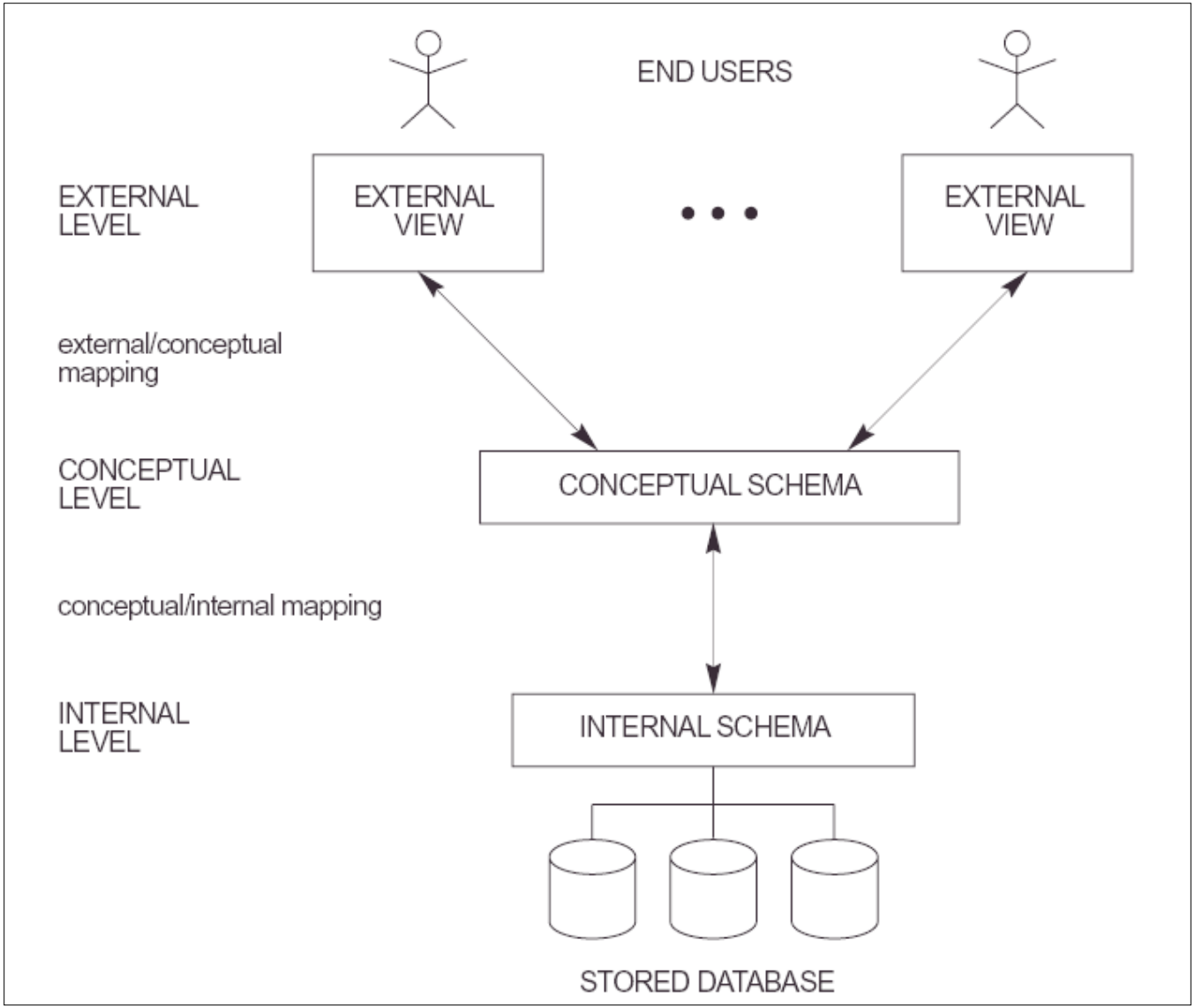
Database System Concepts And Architecture

- Data Models, Schemas, and Instances
- Three-Schema Architecture and Data Independence 
- Database Languages and Interfaces
- The Database System Environment
- Centralized and Client/Server Architectures for DBMSs
- Classification of Database Management Systems

DBMS architecture

- Three-schema-architecture or three-layer-architecture:
  - **Internal** layer: internal schema (physical)
  - **Conceptual** layer: conceptual schema (ER)
  - **External** layer: user views

- Predefined **mappings** between the layers


END USERS
EXTERNAL LEVEL
EXTERNAL VIEW
EXTERNAL VIEW
external/conceptual mapping
CONCEPTUAL LEVEL
CONCEPTUAL SCHEMA
conceptual/internal mapping
INTERNAL LEVEL
INTERNAL SCHEMA
STORED DATABASE

Data Independence

- Capacity to change the schema at one level of a database system
  - Without having to change the schema at the next higher level
- Types:
  - **Logical** data independence
    - Change conceptual schema without changing external views
  - **Physical** data independence
    - Change physical layer without changing conceptual schema

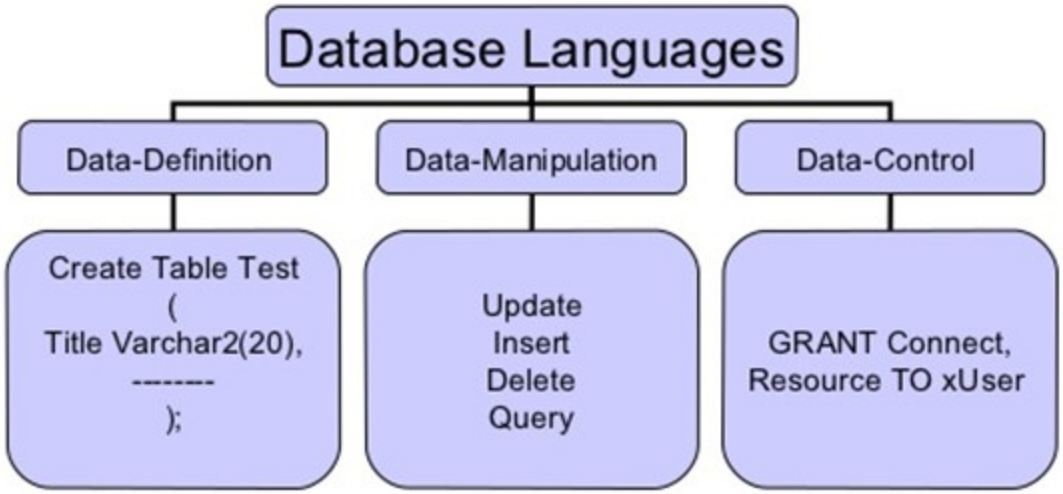
Database System Concepts And Architecture

- Data Models, Schemas, and Instances
- Three-Schema Architecture and Data Independence
- Database Languages and Interfaces 
- The Database System Environment
- Centralized and Client/Server Architectures for DBMSs
- Classification of Database Management Systems

Database languages

- Data-definition language (DDL)
  - Specify conceptual schema
- Storage-definition language (SDL)
  - Specify internal schema
- View-definition language (VDL)
  - Specify user views
- Data-manipulation language (DML)
  - Create, Retrieve, Update and Delete operations

Database Languages



How the Programmer Sees the DBMS

- Start with DDL to *create tables*:

```
CREATE TABLE Students (
        Name CHAR(30)
        SSN CHAR(9) PRIMARY KEY NOT NULL,
        Category CHAR(20)
)  . . .
```

- Continue with DML to *populate tables:*

```
INSERT INTO Students
VALUES('Charles', '123456789', 'undergraduate')
. . . .
```

How the Programmer Sees the DBMS

- Tables:

Students:

SSN	Name	Category
123-45-6789	Charles	undergrad
234-56-7890	Dan	grad
	…	…

Takes:

SSN	CID
123-45-6789	CSE444
123-45-6789	CSE444
234-56-7890	CSE142
	…

Courses:

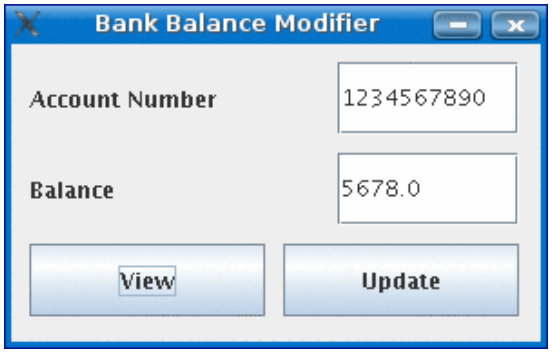
CID	Name	Quarter
CSE444	Databases	fall
CSE541	Operating systems	winter

- Still implemented as files, but behind the scenes can be quite complex

"*data independence*" = separate *logical* view from *physical implementation*

Database interfaces

- Menu-based (browsing)
- Form-based (form spec language)
- GUI
- Natural language interface
  - "Free Form" textual query
- Specialized interfaces for parametric users
  - Small frequent operations
- Interfaces for the administrator
  - Privileged commands by DBA, Security commands etc



Database System Concepts And Architecture

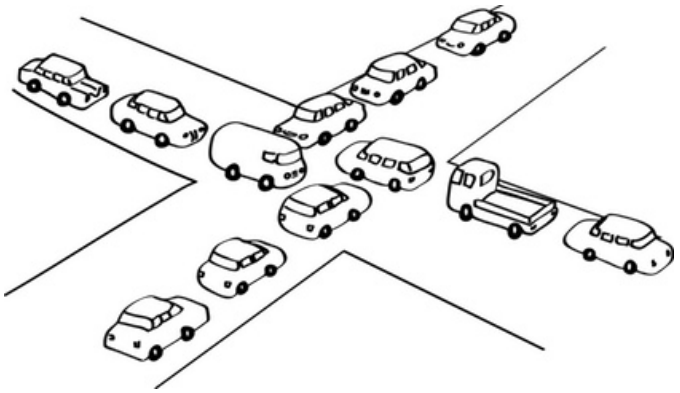
- Data Models, Schemas, and Instances
- Three-Schema Architecture and Data Independence
- Database Languages and Interfaces
- The Database System Environment 
- Centralized and Client/Server Architectures for DBMSs
- Classification of Database Management Systems

The Database System Environment

- DBMS component modules
  - Buffer management
  - Stored data manager
  - DDL compiler
  - Interactive query interface
    - Query compiler
    - Query optimizer

The Database System Environment (cont'd.)

- DBMS component modules
  - Runtime database processor
  - System catalog
  - Concurrency control system
  - Backup and recovery system



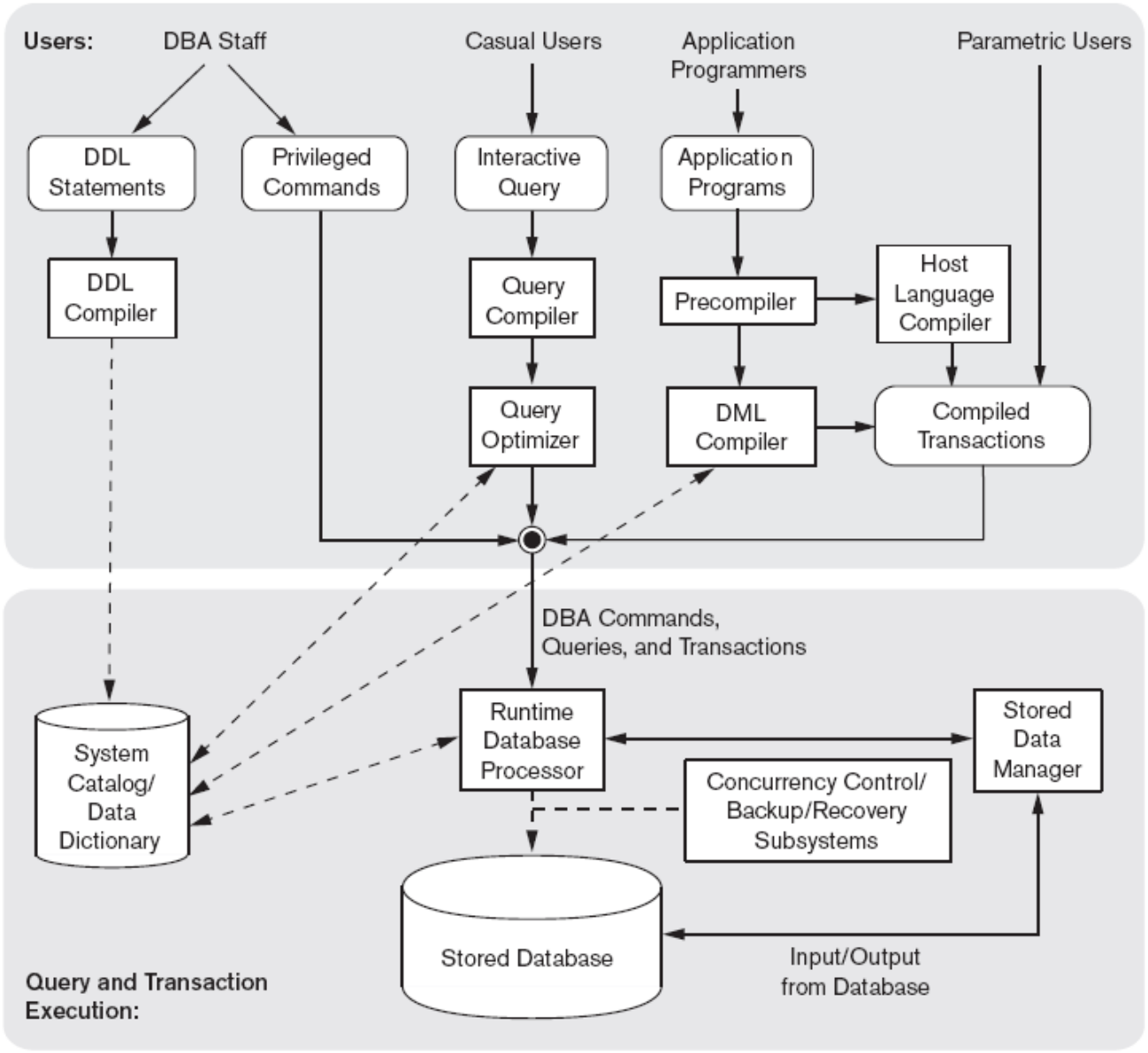


Figure 2.3
Component modules of a DBMS and their interactions.

Database system utilities

- Loading data
  - Loading existing data files
  - Transferring data from another DBMS
  - Data conversion is needed
- Backup
- Reorganisation
  - Improve performance
- Performance monitoring
  - Provide statics to DBA
- Other (sorting files, data compression, user-access monitoring, network interfacing, …)

Other tools

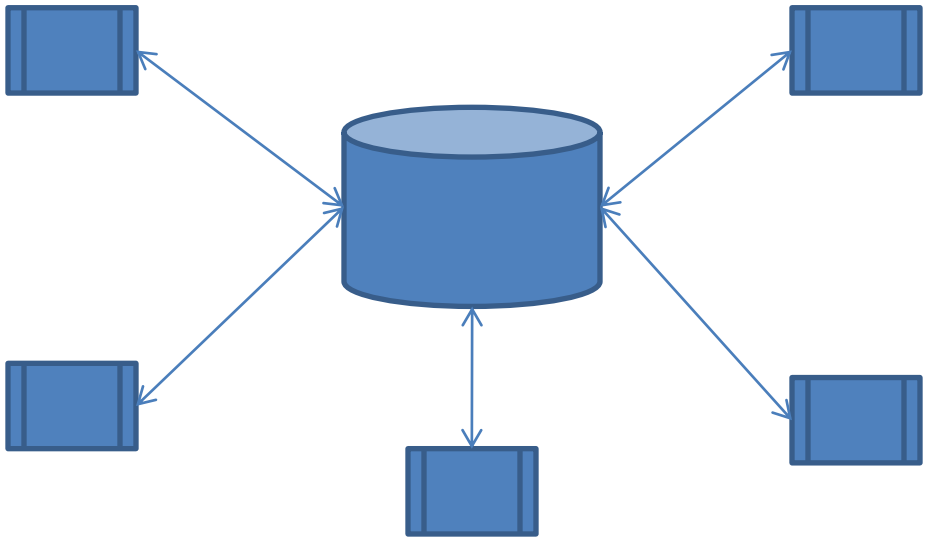
- CASE tools (for designing databases)
- Data dictionary system
  - Store design decisions, usage standards, etc
  - Used mainly by users and not by DBMS software
- (Rapid) Application development environment
  - Powerbuilder (Sybase) / JBuilder (Borland) / Visual Basic
- Communication software (DB/DC)
  - Allow users to remotely access data

Database System Concepts And Architecture

- Data Models, Schemas, and Instances
- Three-Schema Architecture and Data Independence
- Database Languages and Interfaces
- The Database System Environment
- Centralized and Client/Server Architectures for DBMSs
- Classification of Database Management Systems

Centralized architecture

- Centralized architecture
  - DBMS runs on a central computer (mainframe) to which (dumb) terminals are connected



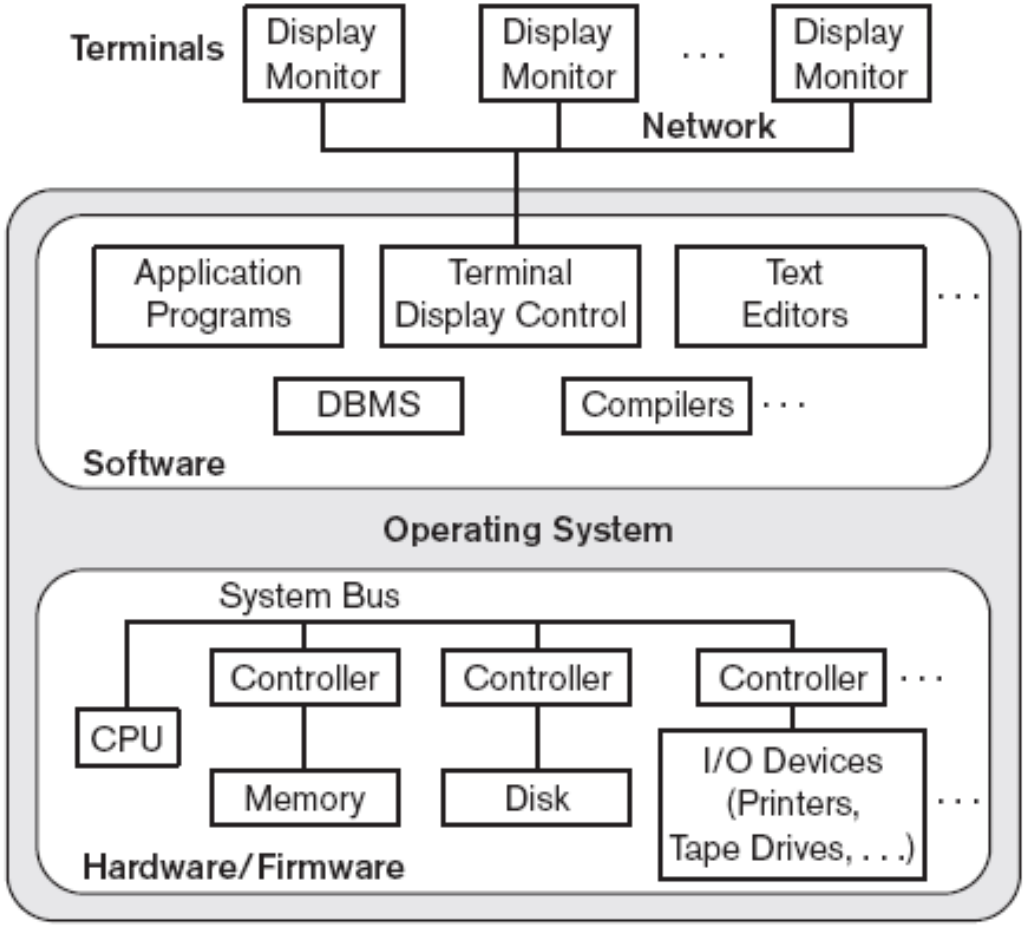


Figure 2.4
A physical centralized architecture.

Introduction of LAN and PC's
Terminals became smart PC's and were able to take over certain tasks of the DBMS

Basic Client/Server Architectures

- **Servers** with specific functionalities
  - **File server**
    - Maintains the files of the client machines.
  - **Printer server**
    - Connected to various printers; all print requests by the clients are forwarded to this machine
  - **Web servers** or **e-mail servers**

Basic Client/Server Architectures (cont'd.)

- **Client machines**
  - Provide user with:
    - Appropriate interfaces to utilize these servers
    - Local processing power to run local applications

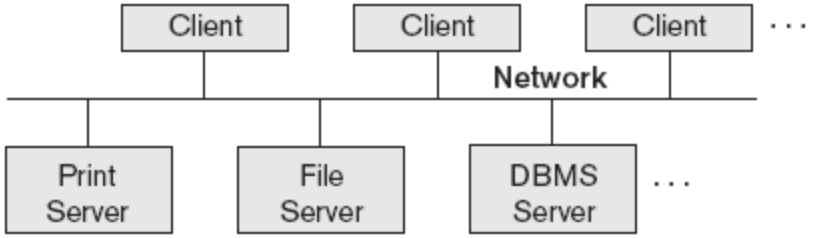


Figure 2.5
Logical two-tier client/server architecture.

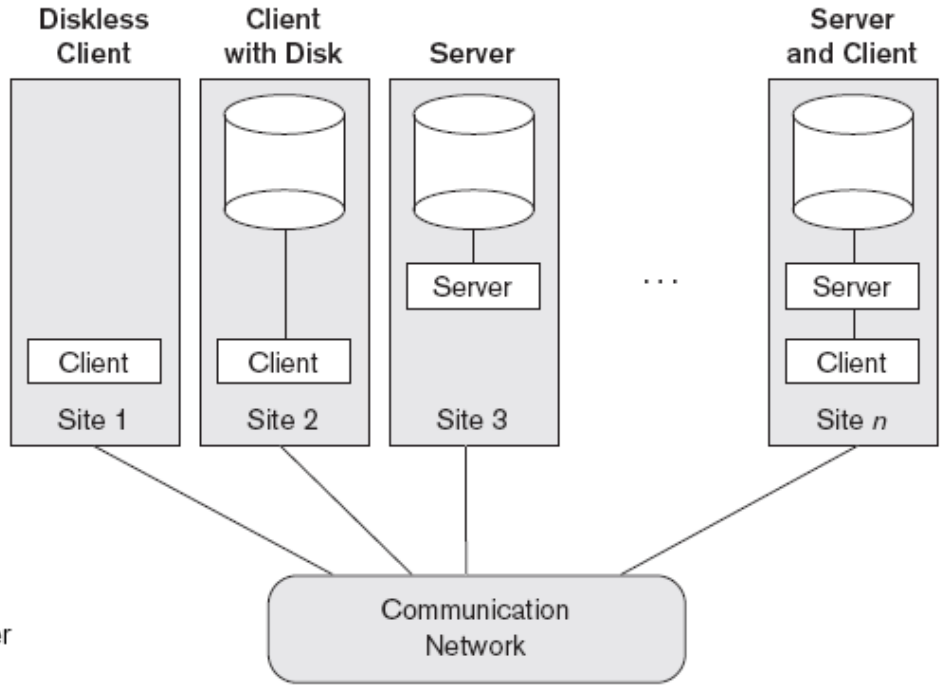


Figure 2.6
Physical two-tier client/server architecture.

Basic Client/Server Architectures (cont'd.)

- **Client**
  - User machine that provides user interface capabilities and local processing
- **Server**
  - System containing both hardware and software
  - Provides services to the client machines
    - Such as file access, printing, archiving, or database access

Two-tier architecture for DBMSs

- **Server** machine runs DBMS (Oracle, SQLserver, MySQL)
  - Contains all data and meta-data
  - Runs queries and updates
- **Client** machine runs application software and communicates with Server
  - Via ODBC, JDBC, .NET, SQLnet, etc.

Three-tier architecture

- Database **Servers** running DBMS
- **Clients** run Web-browser (*thin* client)
- In between are ***Application Servers*** that run the database applications (process management, business logic)

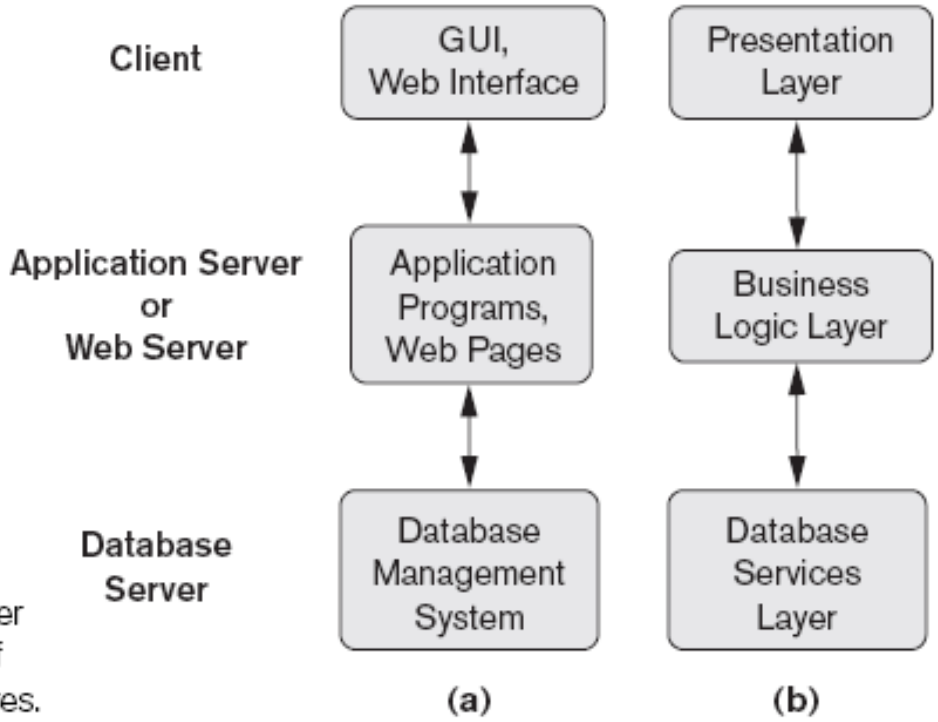
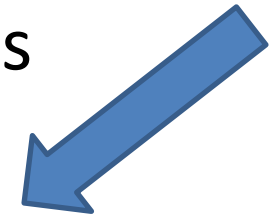


Figure 2.7
Logical three-tier client/server architecture, with a couple of commonly used nomenclatures.

Database System Concepts And Architecture

- Data Models, Schemas, and Instances
- Three-Schema Architecture and Data Independence
- Database Languages and Interfaces
- The Database System Environment
- Centralized and Client/Server Architectures for DBMSs
- Classification of Database Management Systems



DBMS Classification

- Criteria
  - Data model (relational, network, object)
  - Number of users (1, more, many)
  - Number of places (centralized, distributed)
  - Costs (open source, commercial)
  - General / specialized
  - etc

Summary

- Concepts used in database systems
- Main categories of data models
- Types of languages supported by DMBSs
- Interfaces provided by the DBMS
- DBMS classification criteria:
  - Data model, number of users, number of sties, cost

Quiz

- If you were designing a Web-based system to make airline reservations and sell airline tickets, which DBMS architecture would you choose? Why?

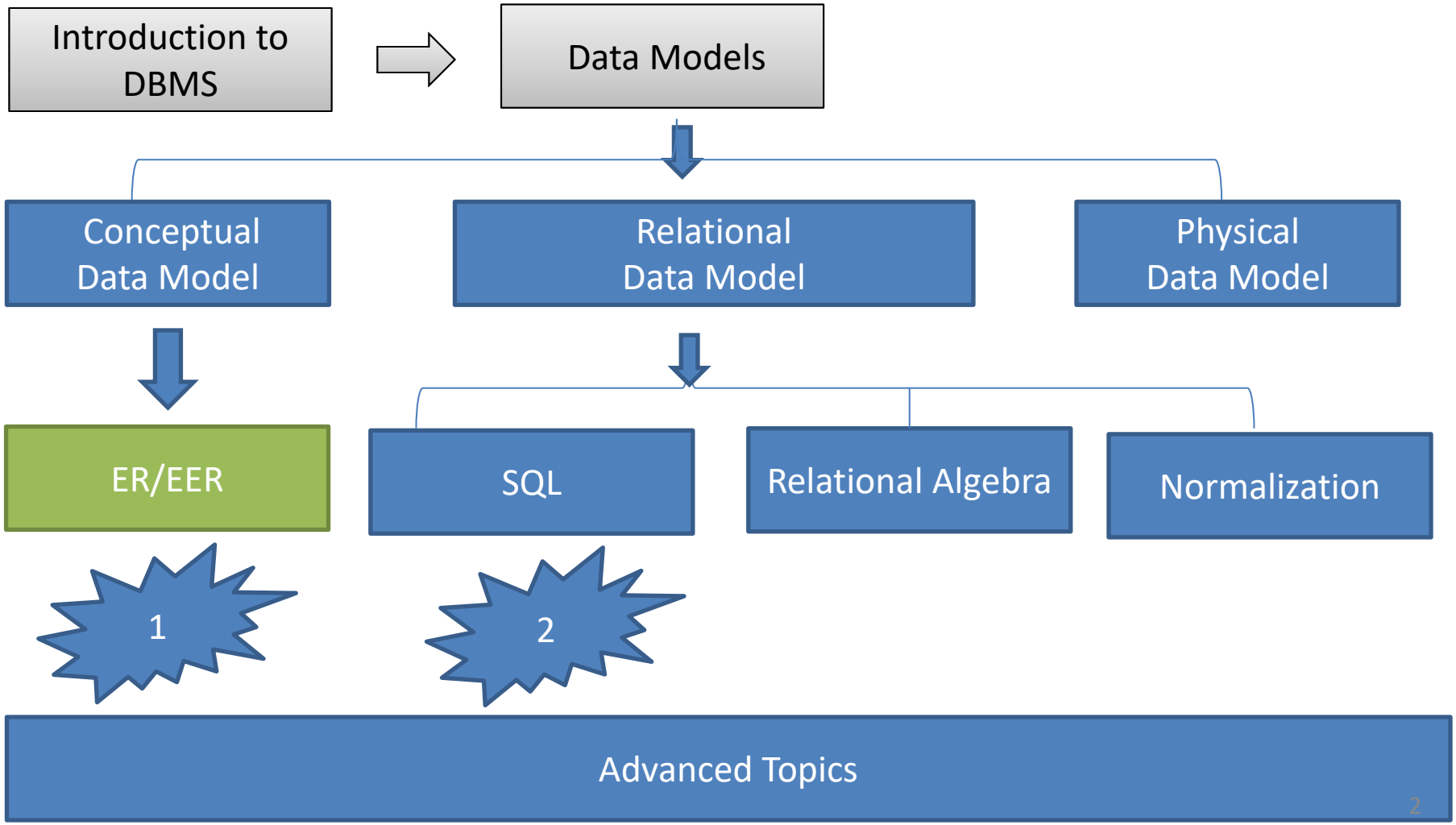
An Introduction to the Database Management Systems

By
Hossein Rahmani

Slides originally by Book(s) Resources

Road Map

(Might change!)



Data Modeling using the Entity Relationship (ER) Model

- High-Level Conceptual Data Models
- Entity Relationship Model and its elements
- From text to ER-schema

Last Session: Three levels of data models

- Conceptual (high-level)
  - for end users / analysts

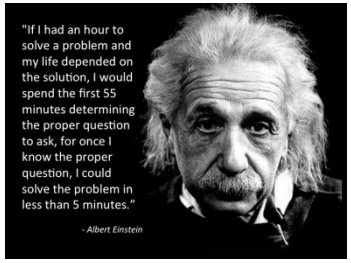
    versus
- Physical (low-level)
  - for programmers
- In between: Representational (implementation) data model
- Today: conceptual model:

  Entity-relationship (ER) model

Entity-Relationship Model

- Conceptual data model
- ER-model is used to create a *conceptual (database) schema*
- Independent of the *representational* model (and of the kind of DBMS)
- The schema will be translated into a *logical schema* (e.g., relational schema, MS-access tables)

Using High-Level Conceptual Data Models for Database Design



- **Requirements collection and analysis**
  - Database designers interview prospective database users to understand and document data requirements
  - Result:
    - **Data requirements**
    - **Functional requirements** of the application

Using High-Level Conceptual Data Models (cont'd.)

- **Conceptual schema**
  - Conceptual design
  - Description of data requirements
  - Includes detailed descriptions of the entity types, relationships, and constraints
  - Transformed from high-level data model into implementation data model

A Sample Database Application

- COMPANY
  - Employees, departments, and projects
  - Company is organized into departments
  - Department controls a number of projects
  - Employee: store each employee's name, Social Security number, address, salary, sex (gender), and birth date
  - Keep track of the dependents of each employee

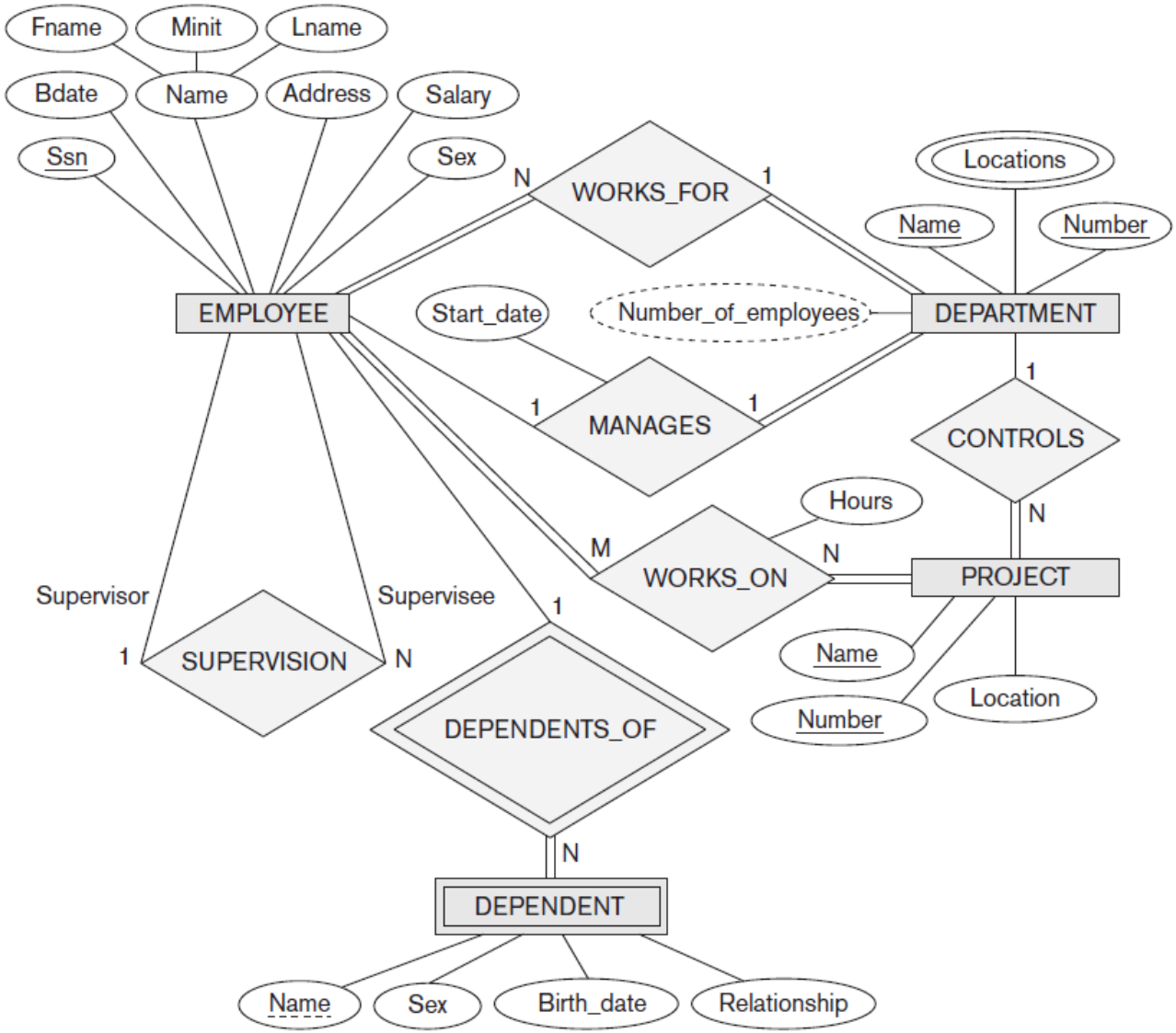
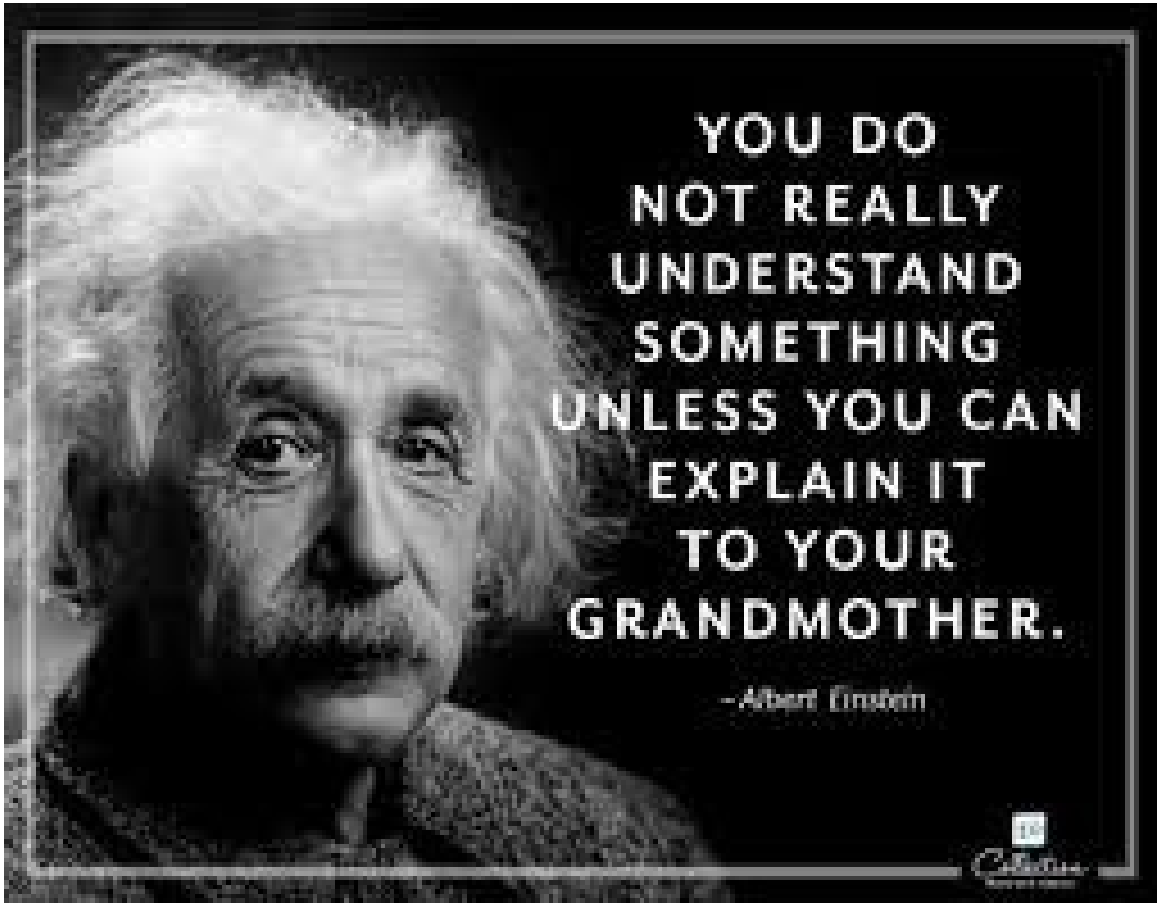


Figure 7.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter and is summarized in Figure 7.14.

What you understand from the previous ER diagram?



Data Modeling using the Entity Relationship (ER) Model

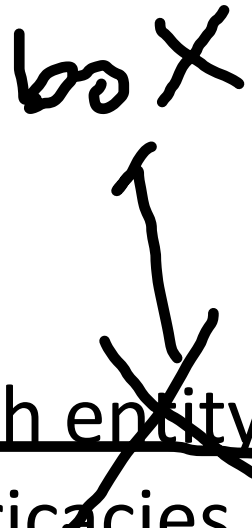
- High-Level Conceptual Data Models
- Entity Relationship Model and its elements
- From text to ER-schema

Entity Relationship Model

- Relevant information from the mini world (universe of discourse) is described in terms of:
  - Entities
  - Attributes
  - Relations

Entities

- An entity in the ER model is the representation of a "thing" in the real world that exists on its own.
  - Tangibles: a *person, a house, a car*
  - Intangibles : a *company, a job, a course*

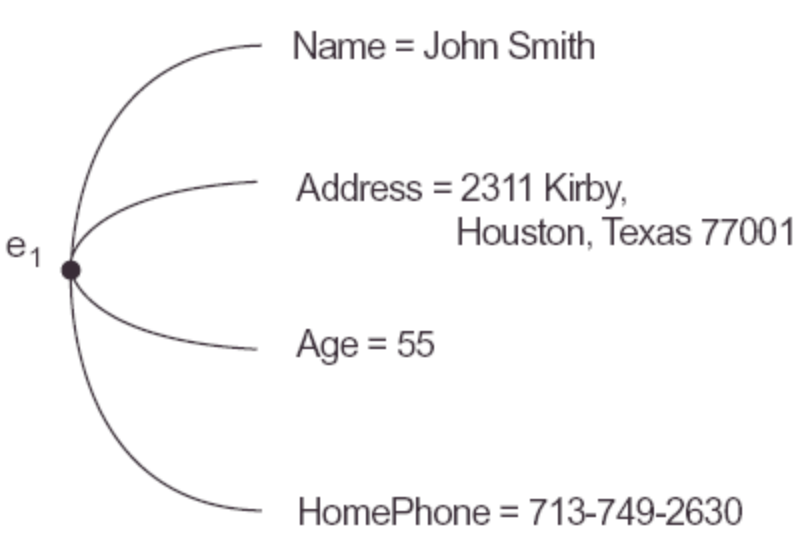


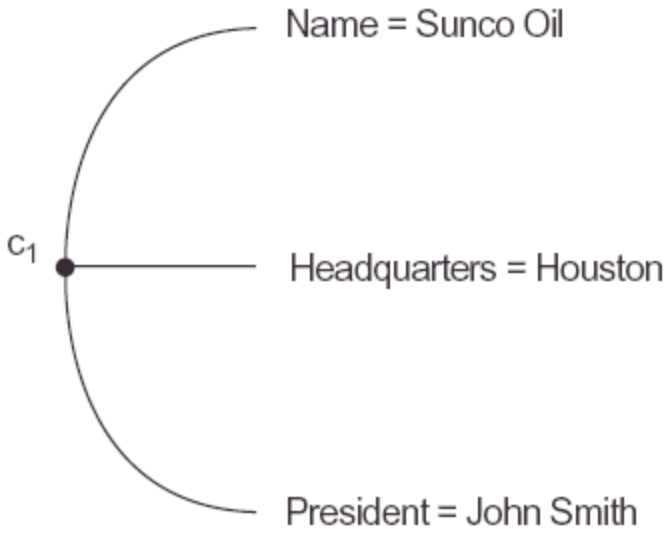
Attributes

- Each entity has *attributes* that describe the intricacies of that specific entity

- A specific entity has *values* for all its attributes

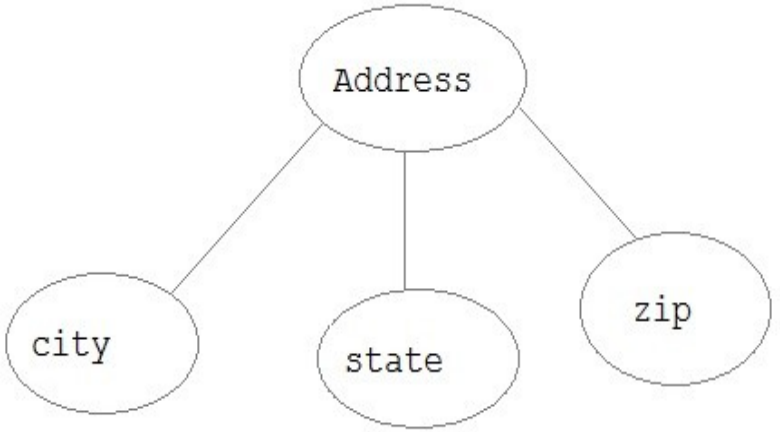
Entities and attributes





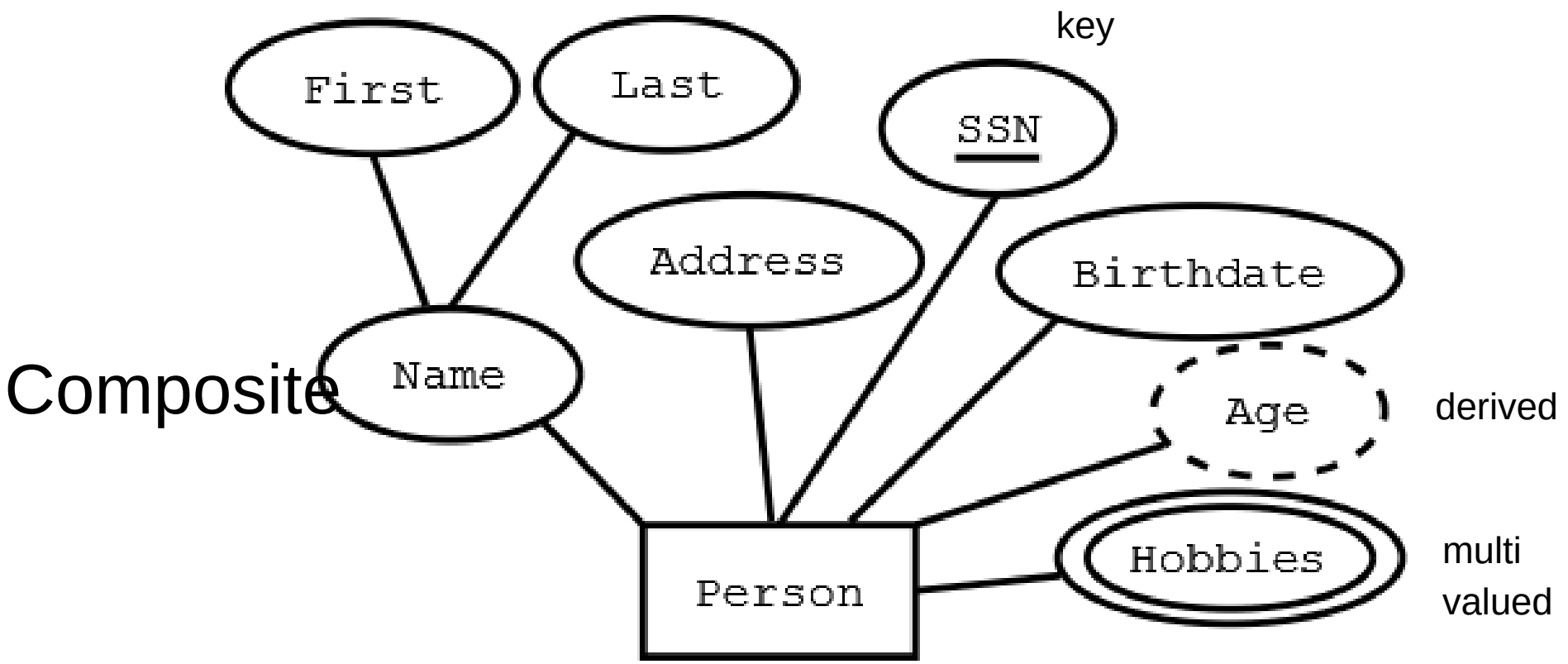
Kinds of attributes

- Composite
  - an attribute that can be split:
    - address = “parkstraat 12a, 5243 GE, Hoogestande”
    - “parkstraat” - “12” - “a” - “5243 GE” - “Hoogestande”
    - attribute-hierarchy
- Atomic
  - cannot be split
    - age = 23



Kinds of attributes

- Single-valued versus multi-valued
  - age = 23 (single-valued)
  - Phone.nr = 043-2235541 and 06-2355534
  - hobby = swimming and piano and party
- Stored versus derived
  - age derived from date of birth
- Complex attributes:
  multi-valued “{}” and composite “()”


key
First
Last
SSN
Address
Birthdate
Name
Composite
Age
derived
Hobbies
multi
valued
Person

Null-values

- An attribute can have a special value *null* (or *nil*). The meaning of the value depends on the context.
- Possible meanings of null:
  - *Not applicable / cannot be determined*
    - *College_degree*
  - *Not relevant*
  - *Applicable but (still) unknown*
  - *Should be known but is missing*

Entity-Types

- In a database there exist groups of *similar* entities:
  - having the same attributes
  - having different values for the attributes

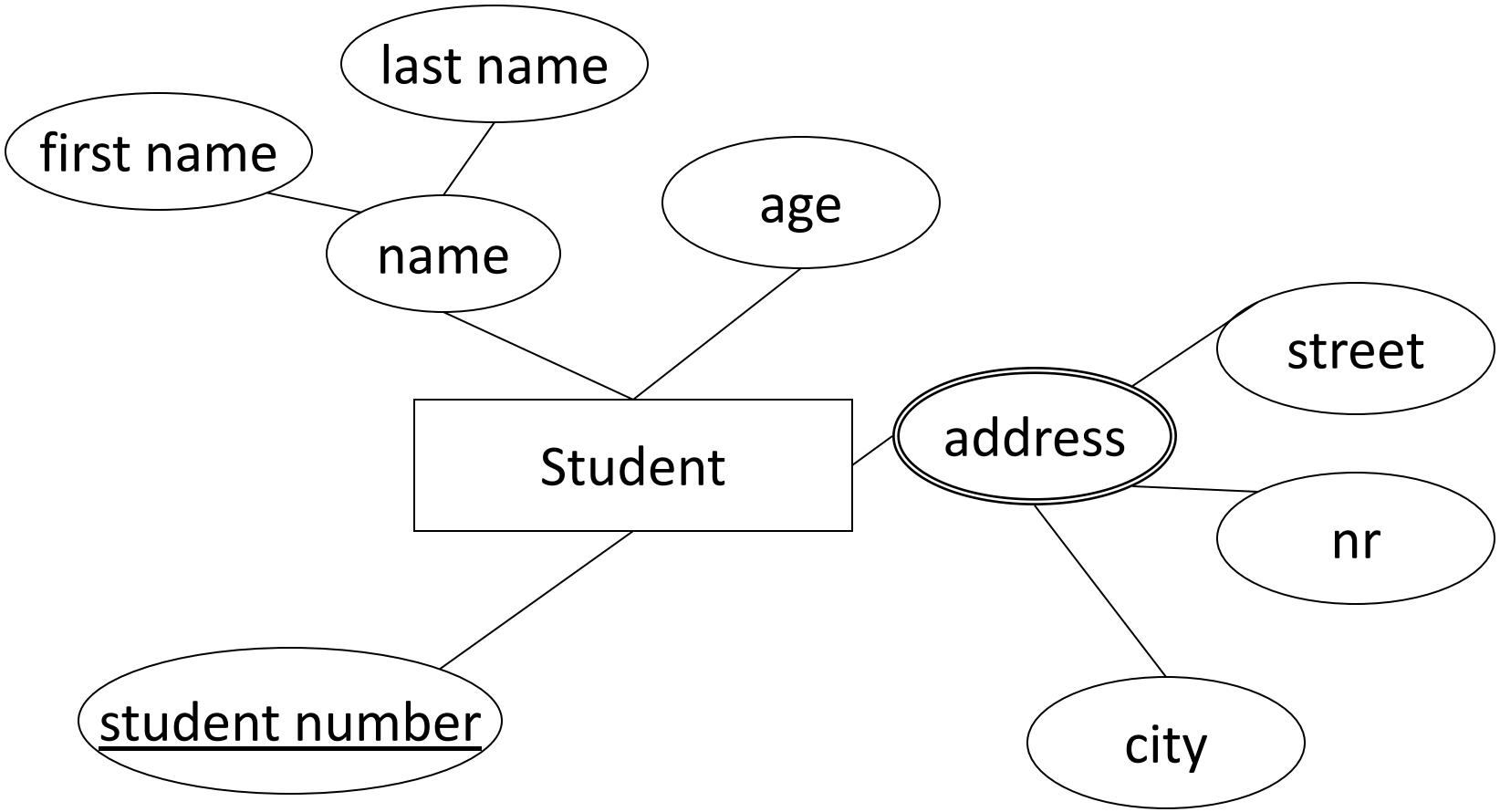
- An **entity-type** represents a *collection of similar entities*.
  (students, houses, companies)

Entity-Type

- In an **ER-schema** entity-types are determined by a unique *name* and a set of *attribute-names*.

- In an **ER-diagram** an entity-type is represented by a rectangle, containing the name, to which ellipses are connected, containing the attribute names. (multivalued: double lines)

Entity-Type


last name
first name
name
age
Student
address
street
nr
city
student number

Key attribute

- A **key** is a collection of attributes of an entity-type for which the *combination of values is unique* for each entity in all possibly occurring entity-sets
  (uniqueness constraint)

- Attributes within a key are called key attributes. (underlined in an ER-diagram)

Key attributes

- An entity-type can have more than one key
- An entity-type can also have no keys (i.e. **weak** entity-type).
- The attributes of any key may never be all null at the same time

Attribute domain

- Every atomic attribute is associated with a value domain
  - natural numbers
  - [30,50]
  - texts of up to 50 characters
  - yes/no
- This is not represented in the ER-diagram (but does belong to the ER-schema!)

Initial Conceptual Design of the COMPANY Database

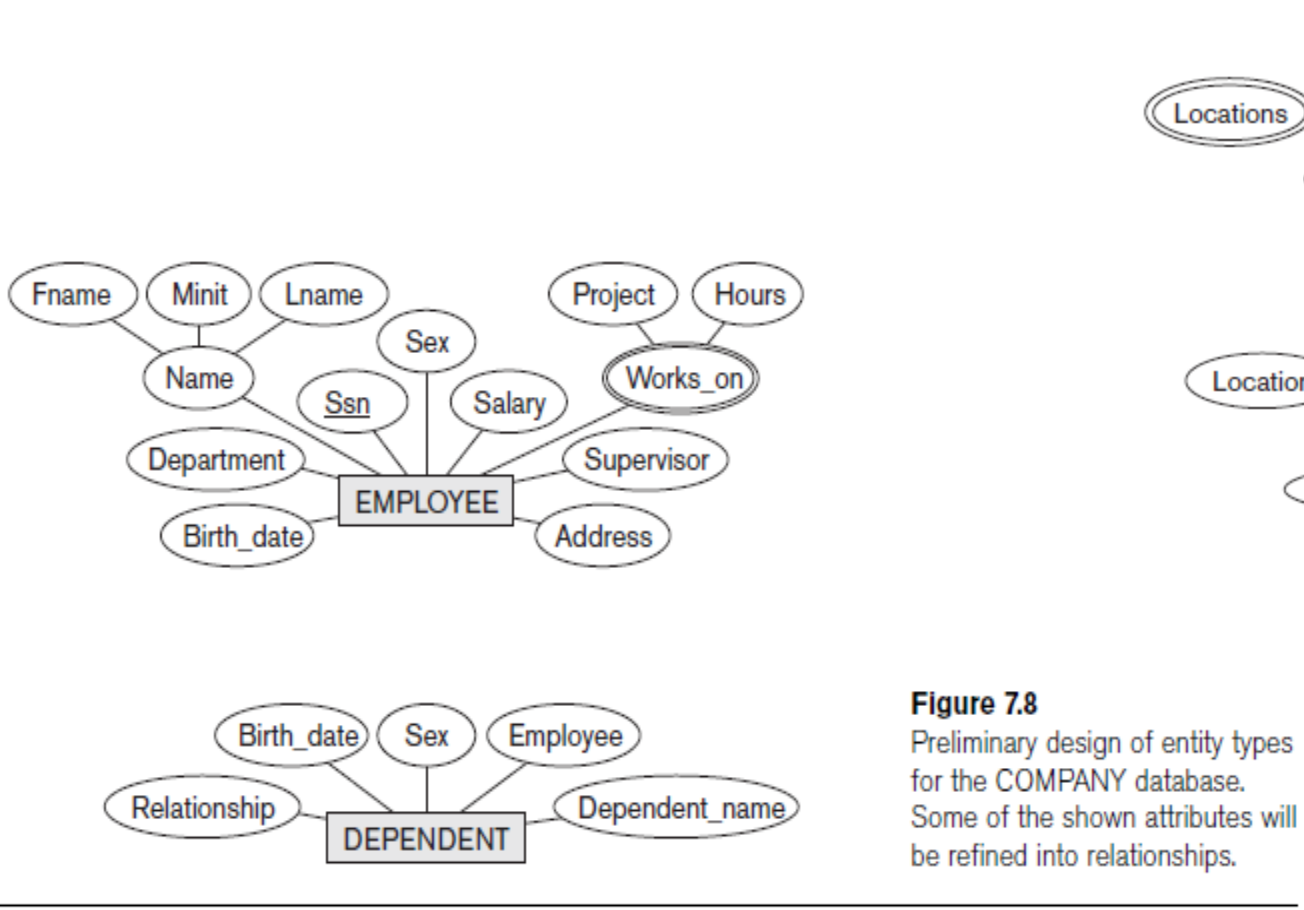


Figure 7.8
Preliminary design of entity types for the COMPANY database. Some of the shown attributes will be refined into relationships.

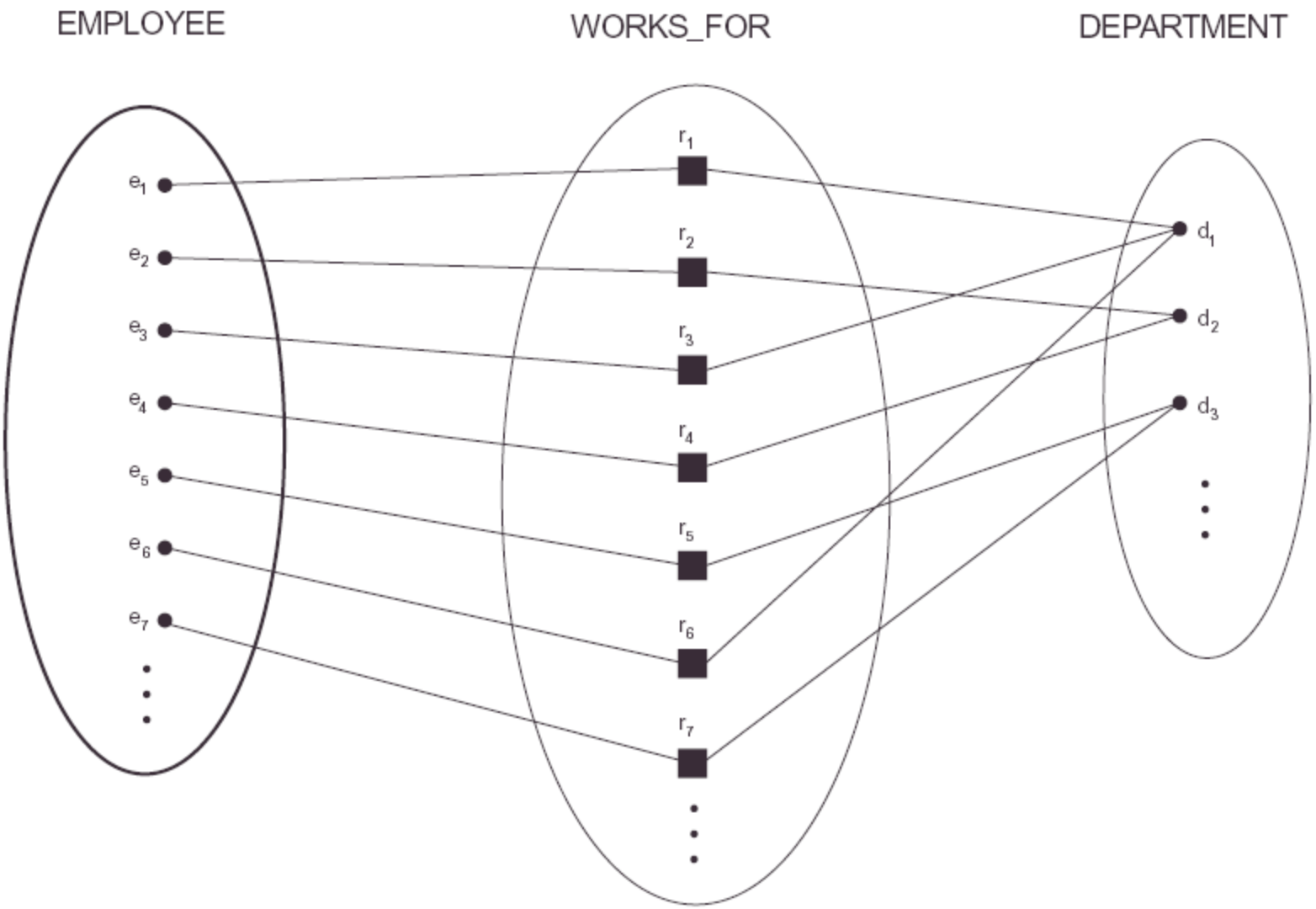
Relations

- A *relation* is the representation of a relevant, existing association between two (or more) entities in the mini world.

  - Student i99883 follows ‘databases’
  - FHS is a faculty of the UM

Relation-Types

- A relation-type between two or more entity-types represents the *set of possible relations* between entities of these entity-types
  - A relation-type *R* on
    participating entity-types $E_1, E_2, .. E_n$
    is a set of
    relation-instances $(e_1, e_2, .. e_n)$
    and is a subset of $E_1 \times E_2 \times .. \times E_n$

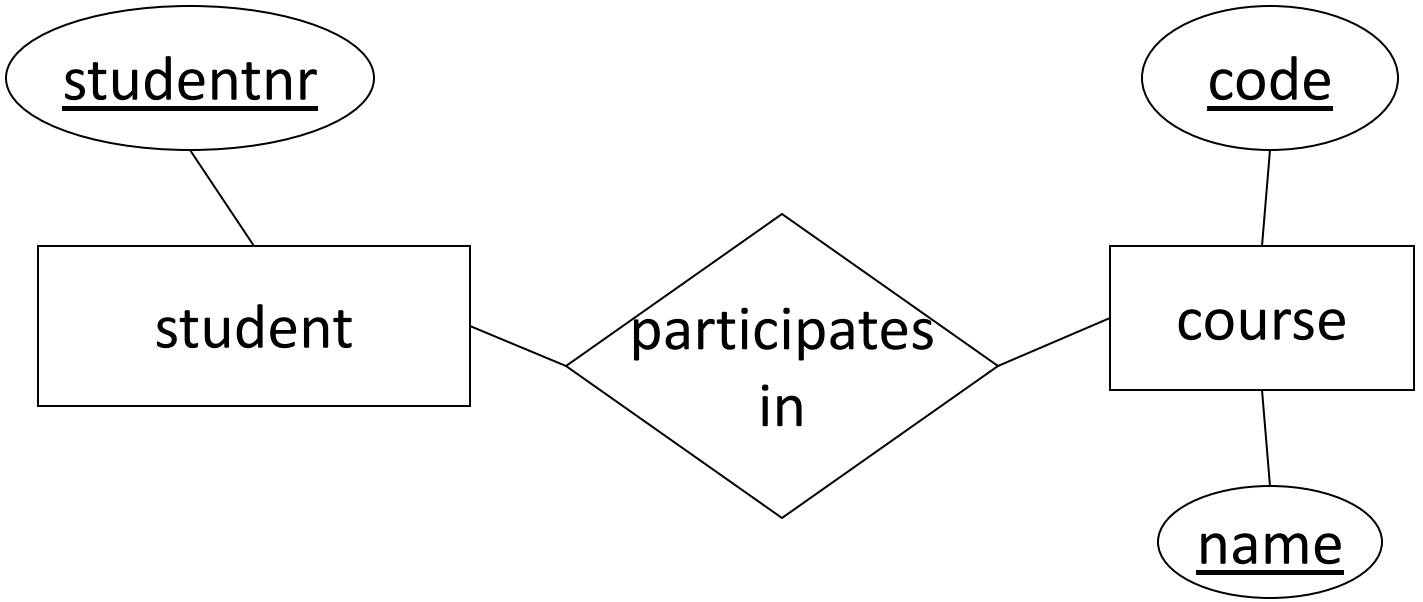

EMPLOYEE
WORKS_FOR
DEPARTMENT
e1
e2
e3
e4
e5
e6
e7
r1
r2
r3
r4
r5
r6
r7
d1
d2
d3

Relation-Types

- Each relation-type has a *name*
- Examples:
  - Faculty is-faculty-of University
  - Student participates-in Course

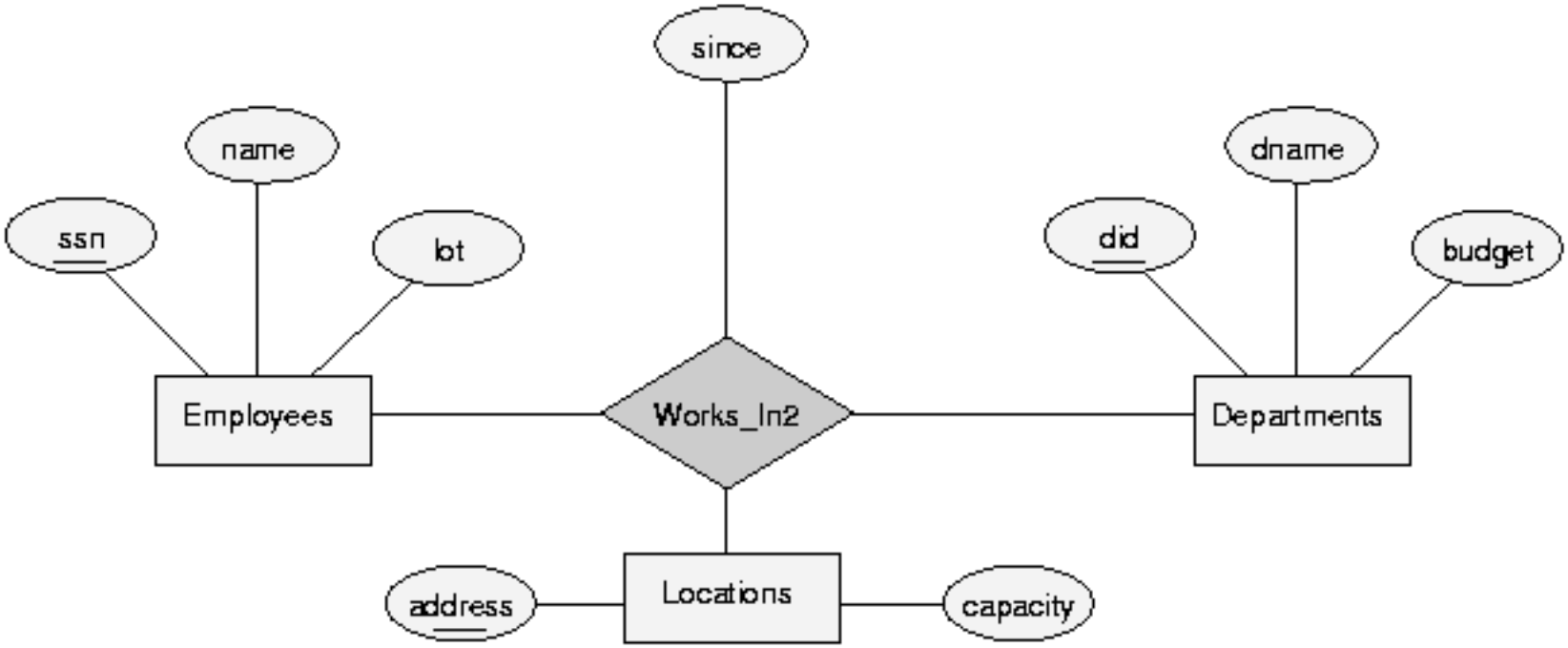
- In ER-diagrams relation-types are represented by a diamond-shape containing the name, connected to the participating entity-types

Relation-Types



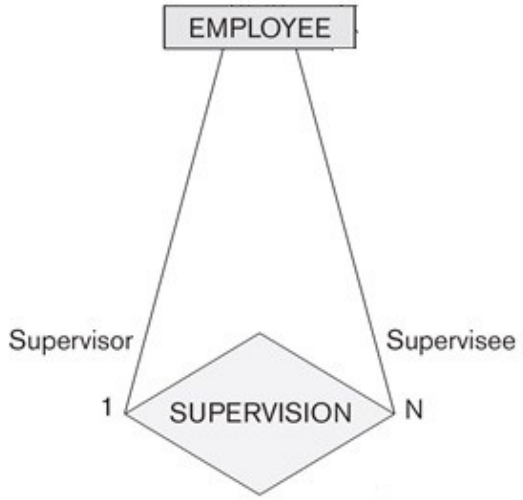
Degree of relation-types

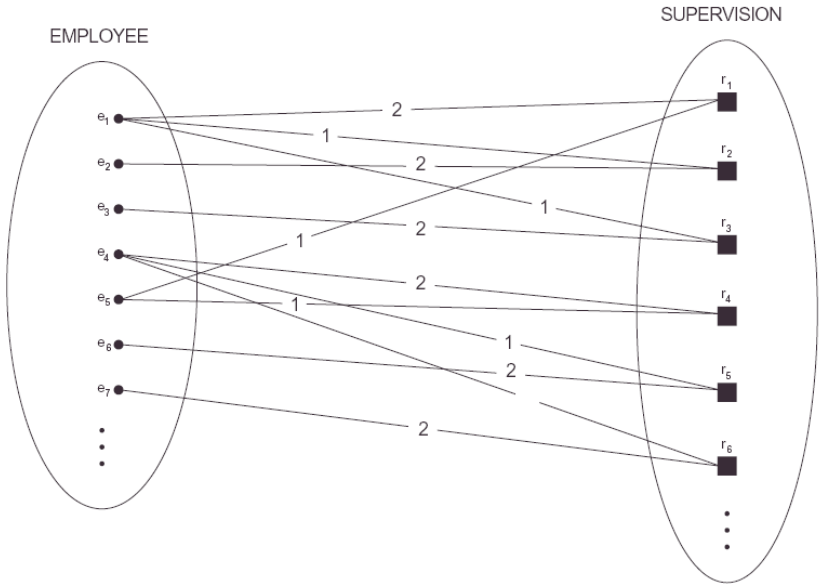
- **Degree** = number of participations by entity-types
  - binary = degree 2
  - ternary = degree 3
- Every degree (> 1) can occur
- In most cases the degree is 2


since
name
dname
ssn
lot
did
budget
Employees
Works_In2
Departments
address
Locations
capacity

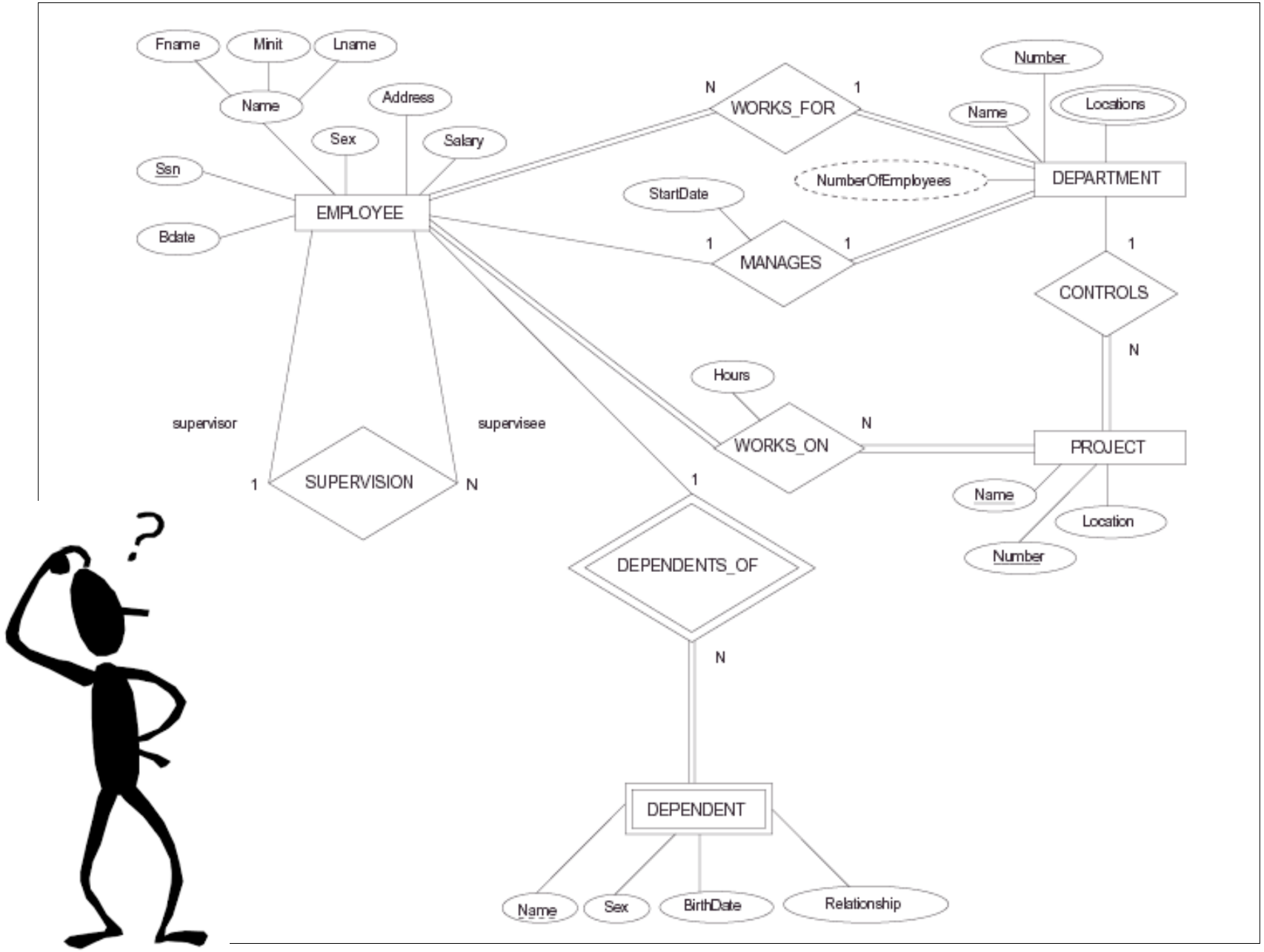
Recursive relation-types

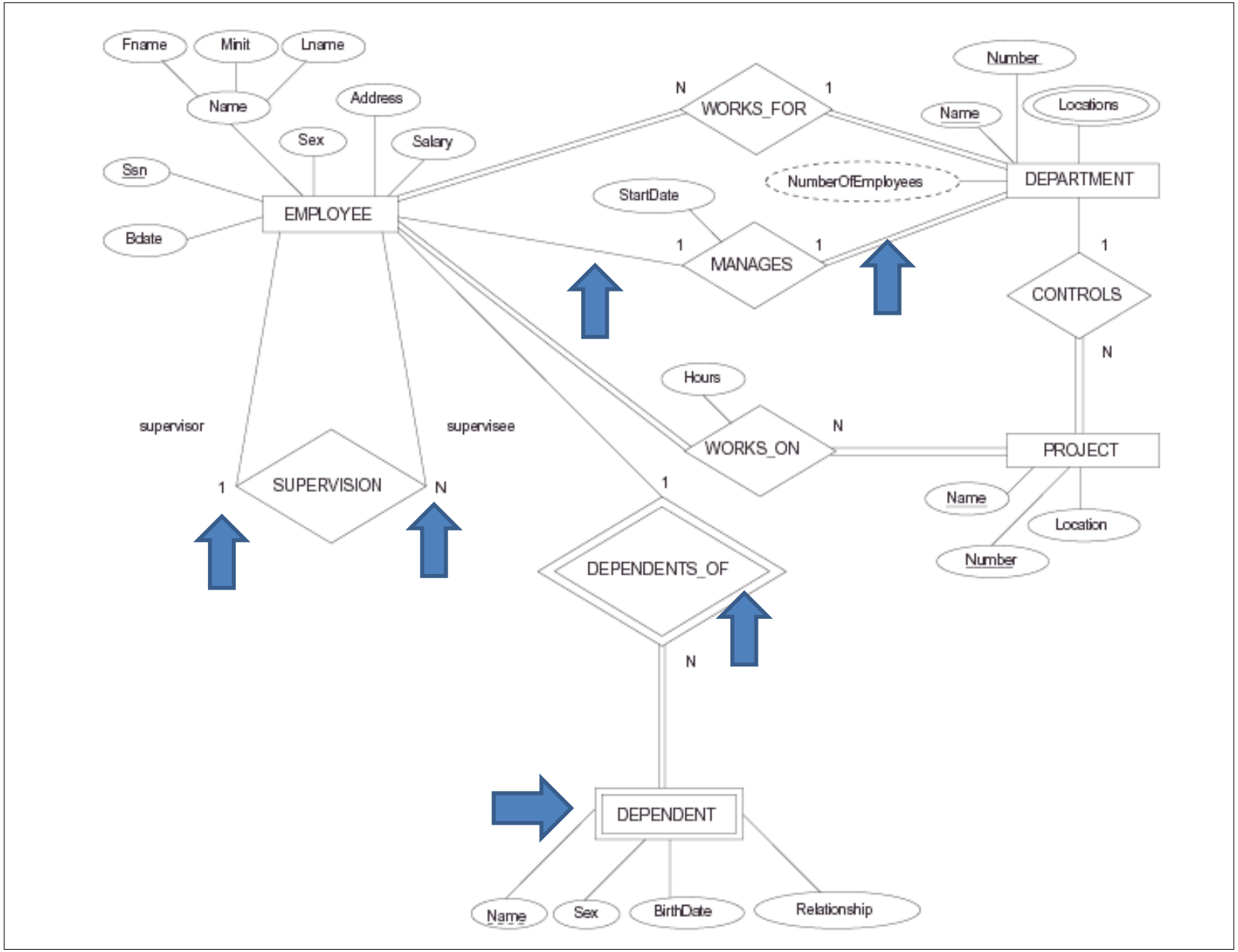
- **Roles**: each entity-type plays a *role* in a relation-type. This role can be mentioned explicitly in a diagram.
- An entity-type can play multiple roles simultaneously in a relation-type:
  - recursive relation-type
- In that case it **must** be mentioned what the roles are of such an entity





Employee plays two roles: supervisor (2) and supervisee (1)

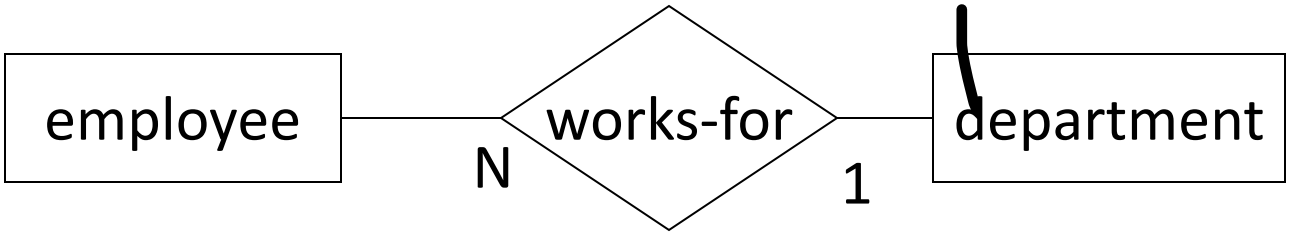

Fname
Minit
Lname
Name
Address
Sex
Salary
Ssn
Bdate
EMPLOYEE
N
WORKS_FOR
1
Number
Name
Locations
DEPARTMENT
NumberOfEmployees
StartDate
1
MANAGES
1
1
CONTROLS
N
Hours
supervisor
supervisee
1
SUPERVISION
N
WORKS_ON
N
PROJECT
Name
Location
Number
1
DEPENDENTS_OF
N
DEPENDENT
Name
Sex
BirthDate
Relationship


Fname
Minit
Lname
Name
Address
Sex
Salary
Ssn
Bdate
EMPLOYEE
N
WORKS_FOR
1
Number
Name
Locations
DEPARTMENT
NumberOfEmployees
StartDate
1
MANAGES
1
1
CONTROLS
N
Hours
supervisor
supervisee
1
SUPERVISION
N
N
WORKS_ON
PROJECT
Name
Location
Number
1
DEPENDENTS_OF
N
DEPENDENT
Name
Sex
BirthDate
Relationship

Cardinality ratio

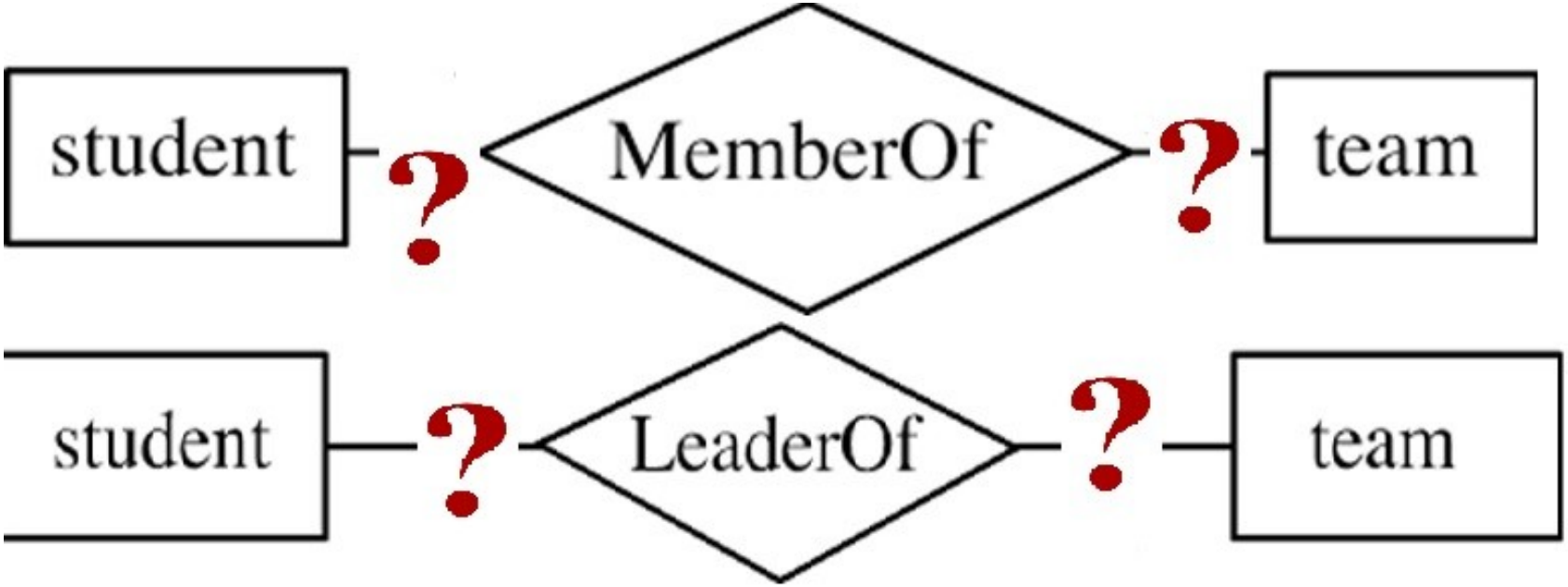
- To how many relation-instances can each entity participate?
  - The ratio for university:faculty is 1:N
    (a faculty can only belong to one university, but a university can have more than one faculty)
- Possible ratios are: 1:1, 1:N, N:1, N:M
- These values are written next to the diamonds in the diagram

Cardinality ratio

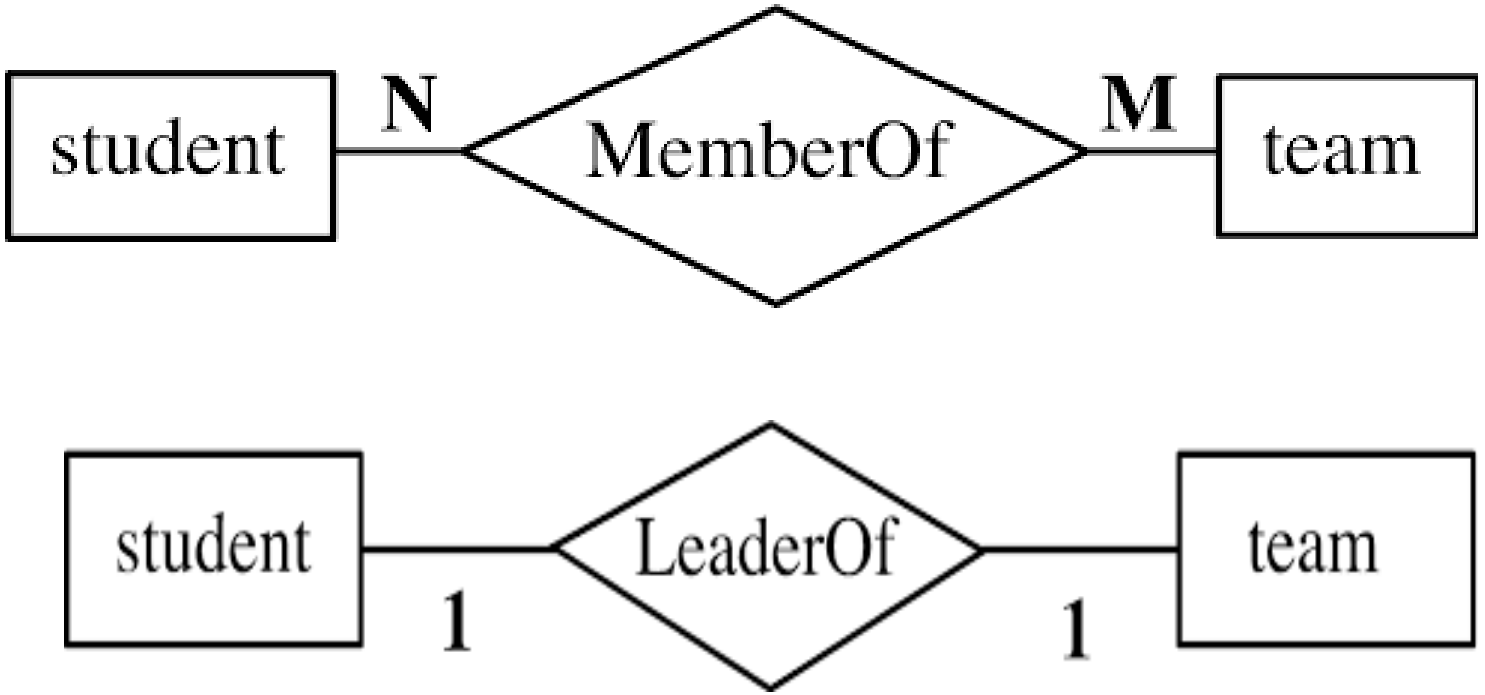


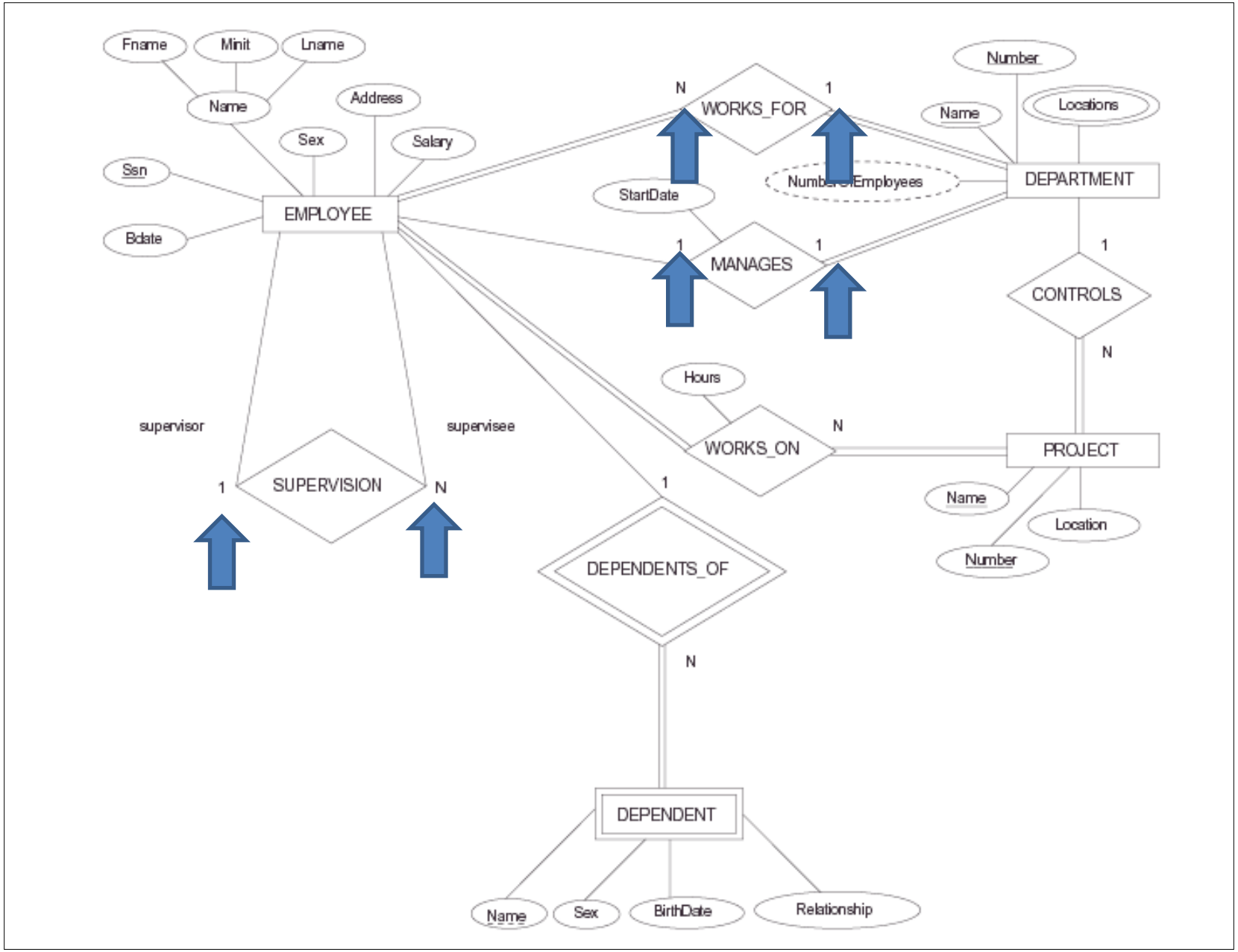
Be aware: in some books the 1 and N are swapped!
Always carefully check what these numbers mean.
In our convention, we read preferentially from left-to-right or top-to-bottom.

Cardinality ratio (More Samples)



Cardinality ratio (More Samples)

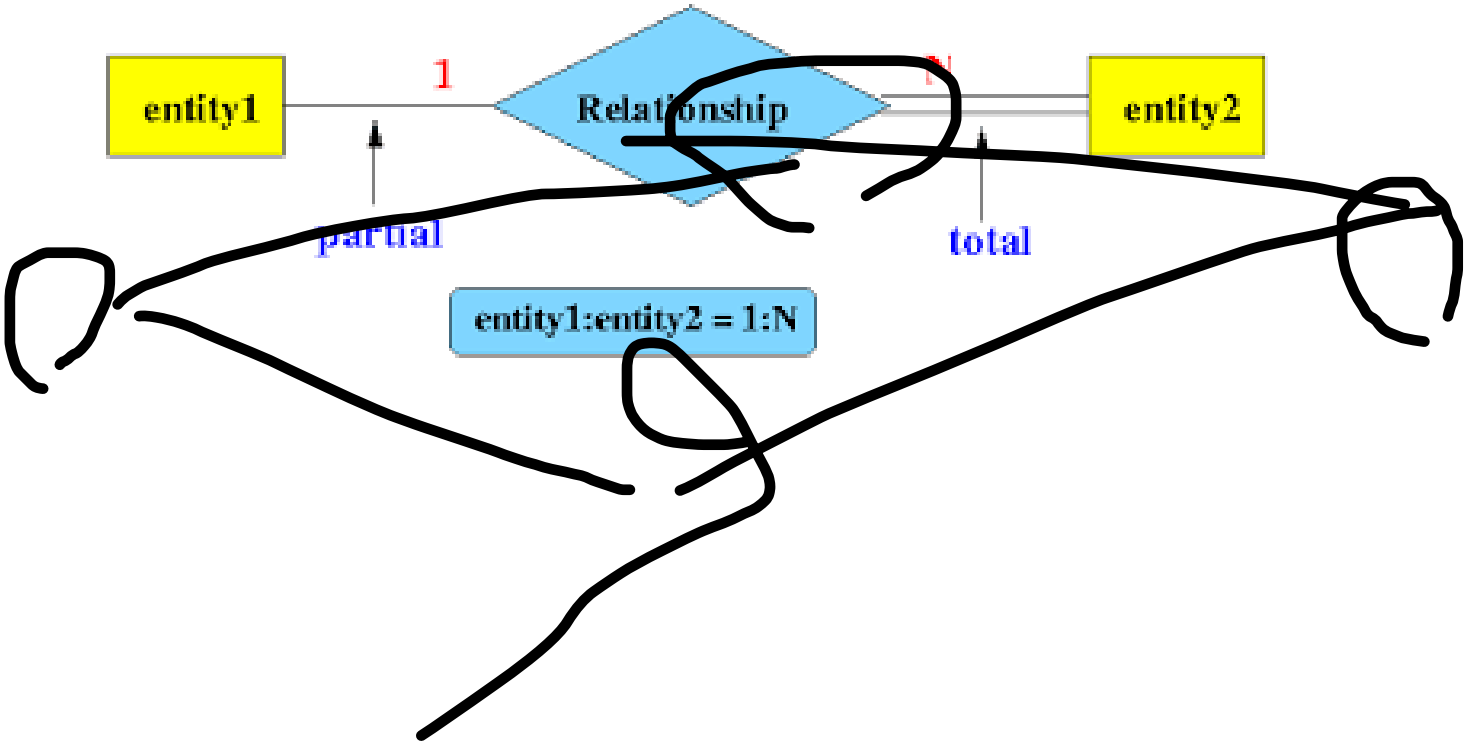



Fname
Minit
Lname
Name
Address
Sex
Salary
Ssn
Bdate
EMPLOYEE
N
WORKS_FOR
1
Number
Name
Locations
DEPARTMENT
StartDate
NumberOfEmployees
1
MANAGES
1
1
CONTROLS
N
Hours
supervisor
supervisee
1
SUPERVISION
N
WORKS_ON
N
PROJECT
Name
Location
Number
1
DEPENDENTS_OF
N
DEPENDENT
Name
Sex
BirthDate
Relationship

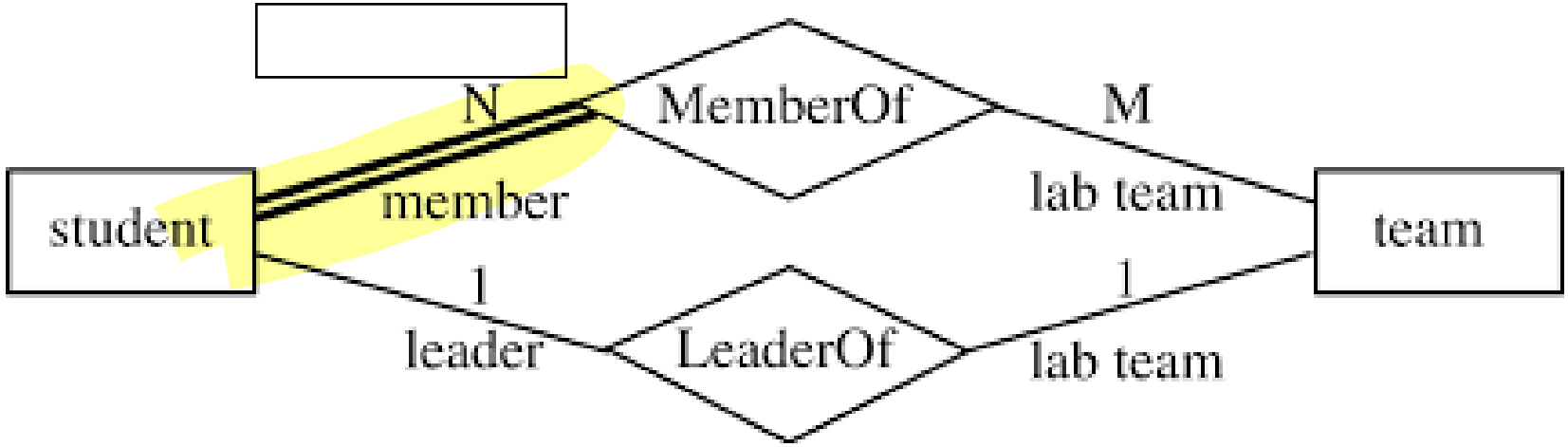
Participation restriction

- An entity-type participates **totally** if every entity of that type participates in at least one relation-instance
- If not, the participation is **partial**
- Totality is indicated by a **double line** in the diagram
- Cardinality and participation form the *structural restrictions on relation-types*

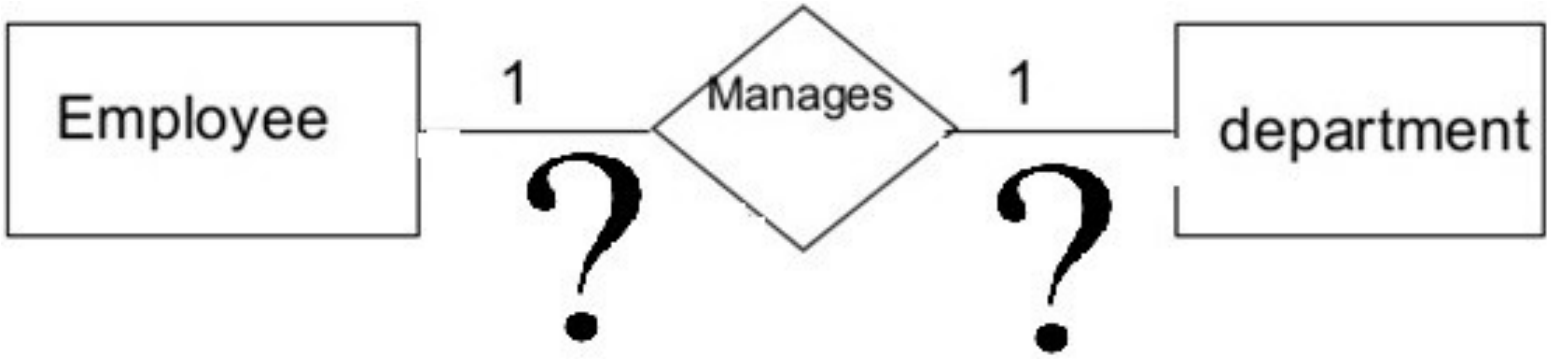
Partial/Total Participation



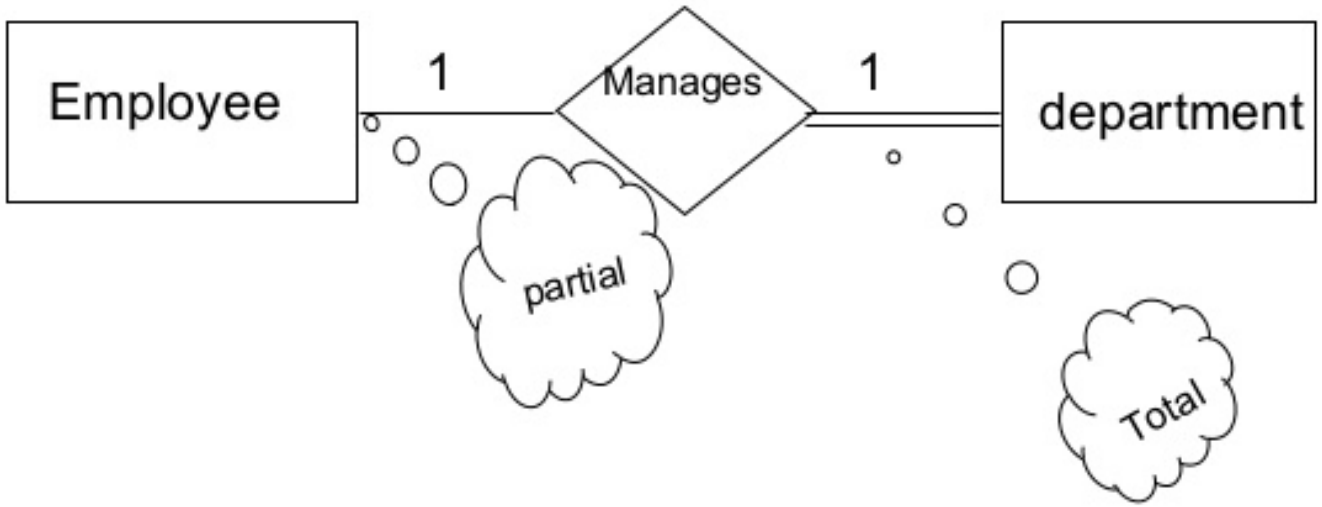
Total/Partial Participation Example 1



Total/Partial Participation Example 2

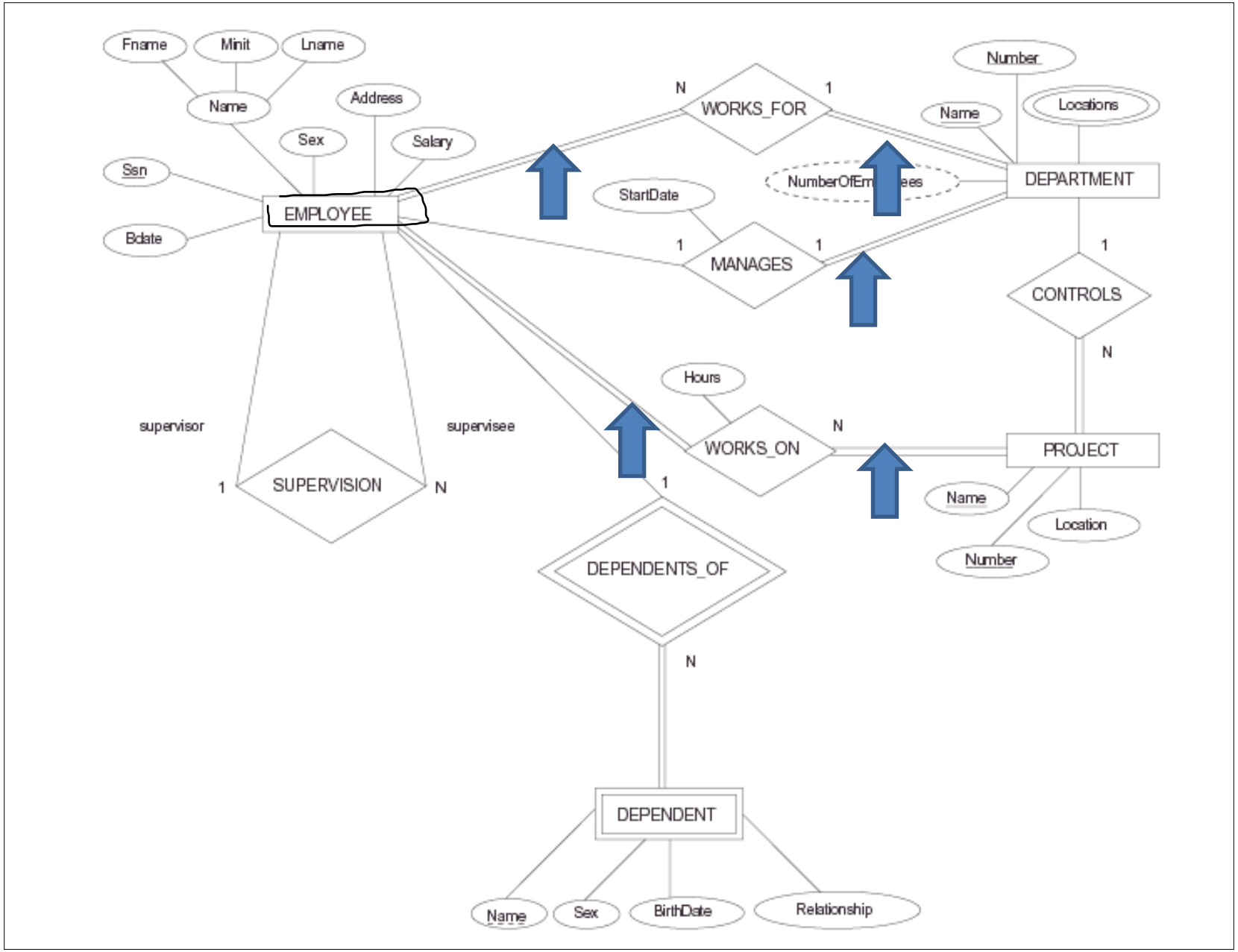


Total/Partial Participation Example 2

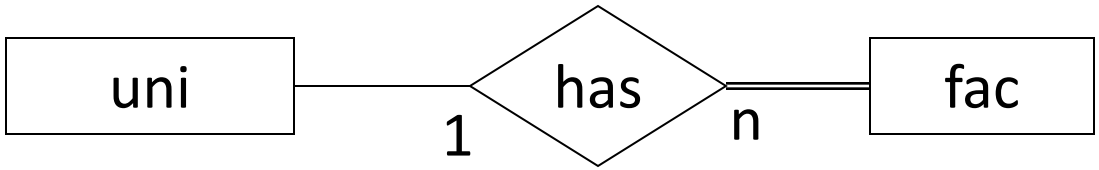


Total/Partial Participation Example 3

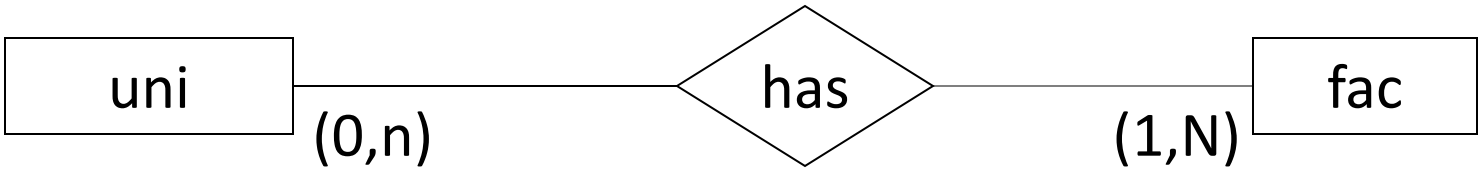
Total/Partial Participation Example 3


Fname
Minit
Lname
Name
Address
Sex
Salary
Ssn
Bdate
EMPLOYEE
N
WORKS_FOR
1
Number
Name
Locations
DEPARTMENT
StartDate
NumberOfEmployees
1
MANAGES
1
1
CONTROLS
N
Hours
supervisor
supervisee
1
SUPERVISION
N
WORKS_ON
N
PROJECT
Name
Location
Number
1
DEPENDENTS_OF
N
DEPENDENT
Name
Sex
BirthDate
Relationship

Alternative Notation

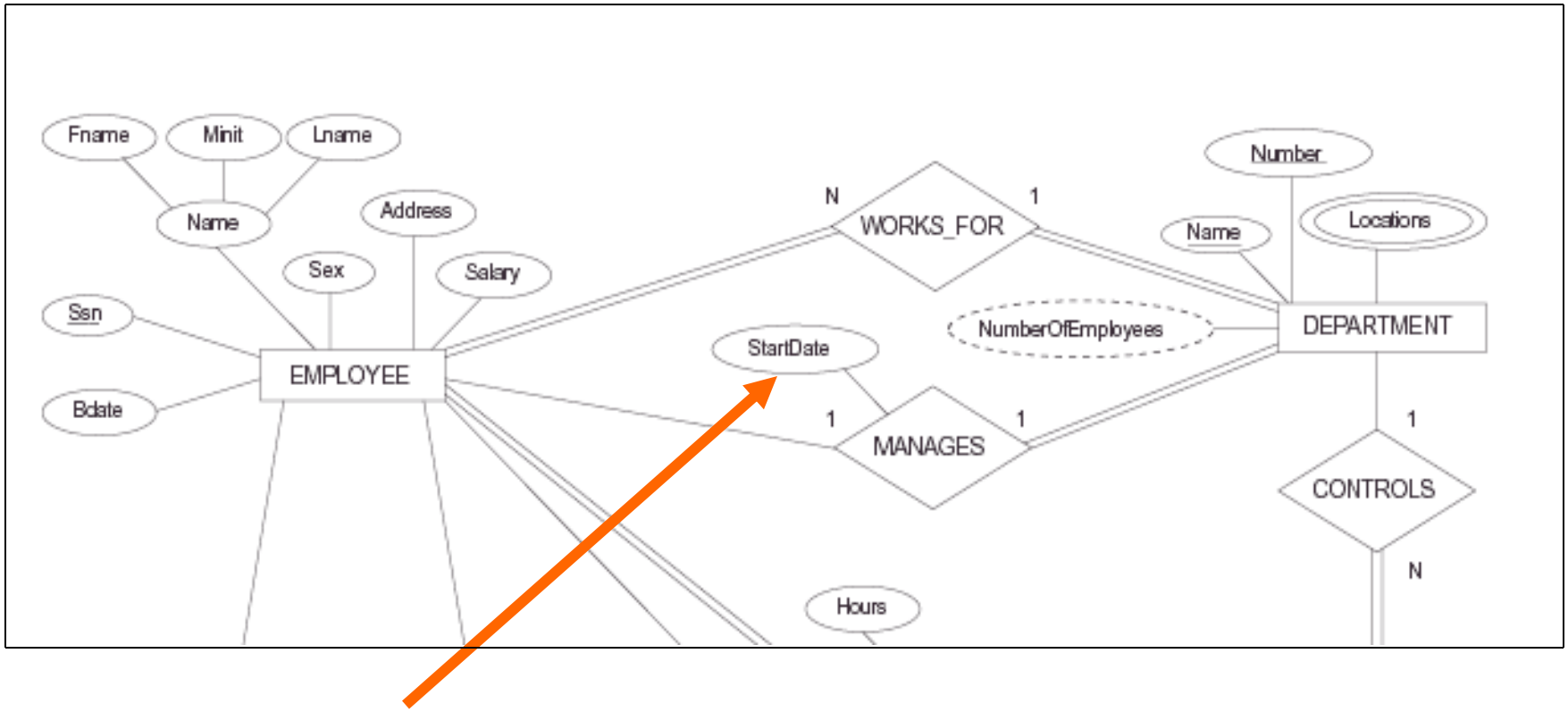


- Write at the entity-type the *minimum* and *maximum* number of participations per entity allowed:



Attributes of relation-types

- Also relation-types can have attributes
  - Student id122773 participates in databases *in the academic year 2012/2013*
- Indicated by ellipses connected to the diamond
- There are no keys for relation-types
- In 1:1 or 1:N relations: the attributes can move to an entity-type



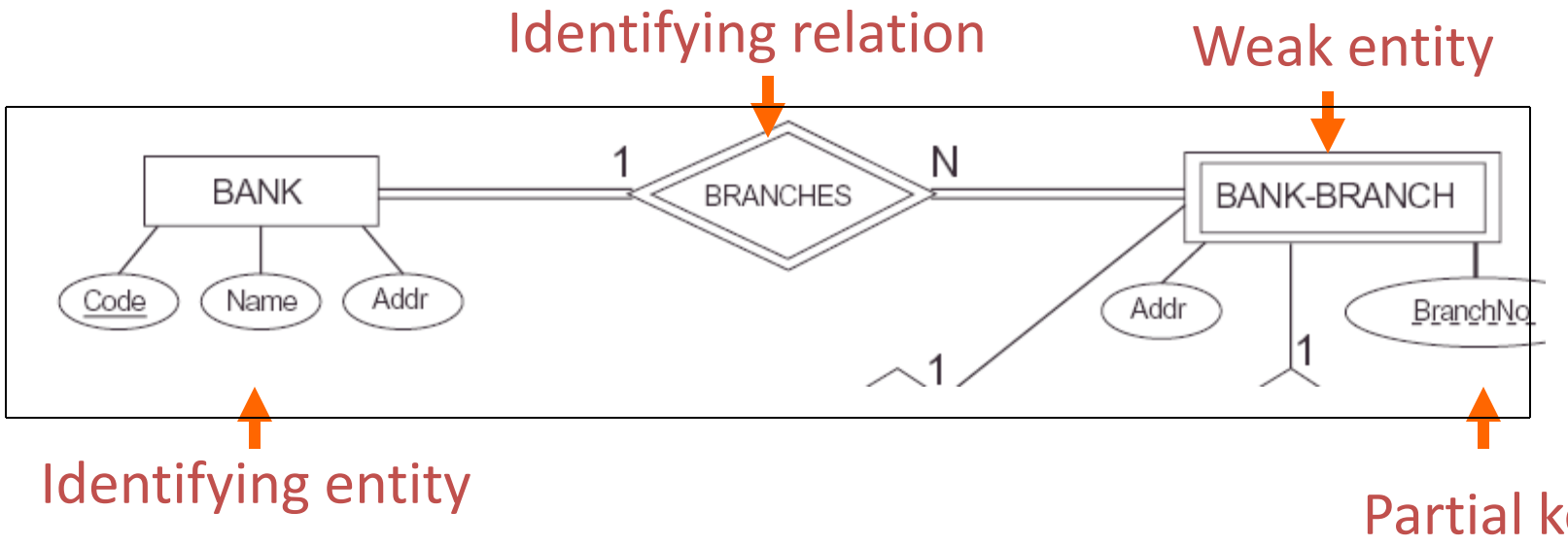
May move to EMPLOYEE or to DEPARTMENT

Weak Entity Types

- Do not have key attributes of their own
  - Identified by being related to specific entities from another entity type
- **Identifying relationship**
  - Relates a **weak** entity type to its **owner**
- Always has a **total** participation constraint

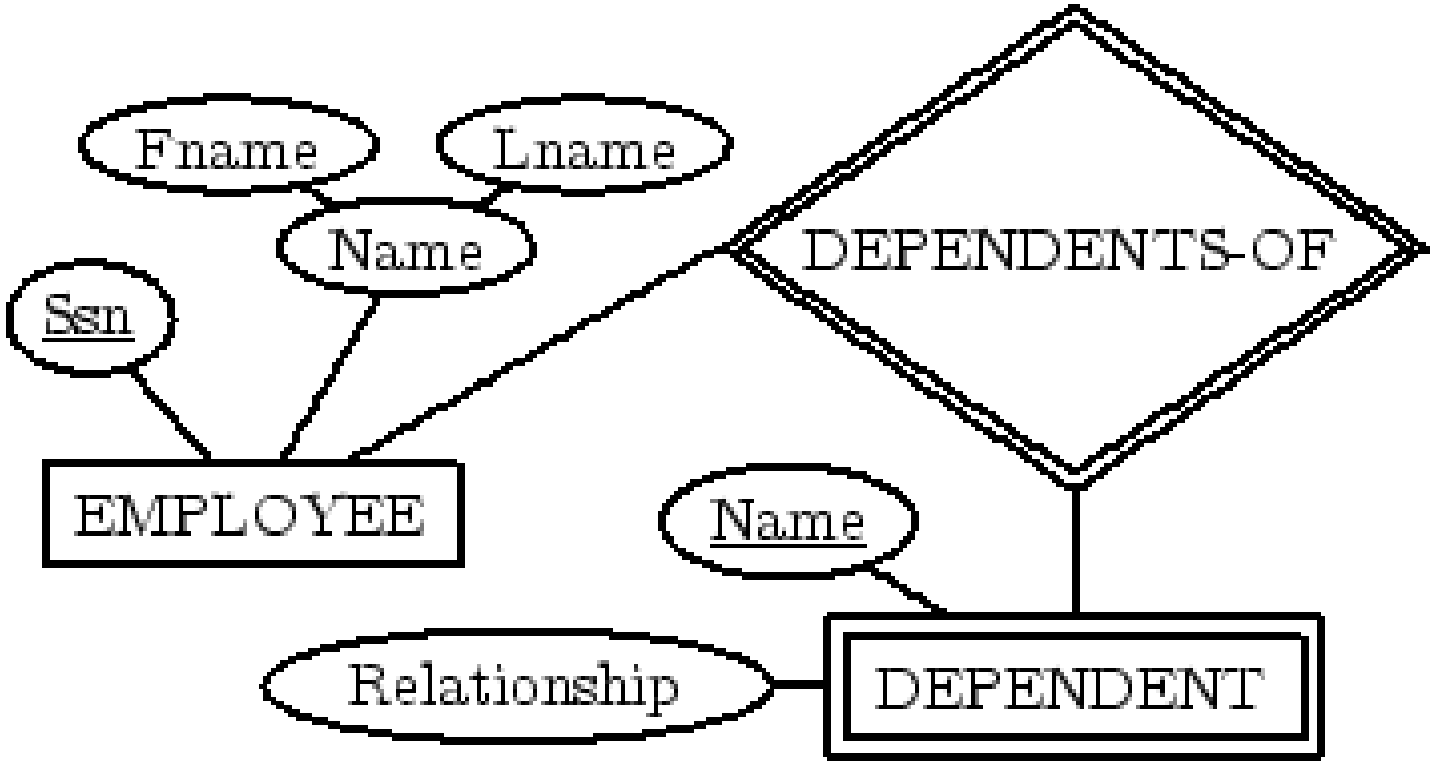
Weak entity-types

- Indicated by double lines for the weak entity type (rectangle) and the identifying relation type (diamond)
  - partial keys are underlined with a broken line

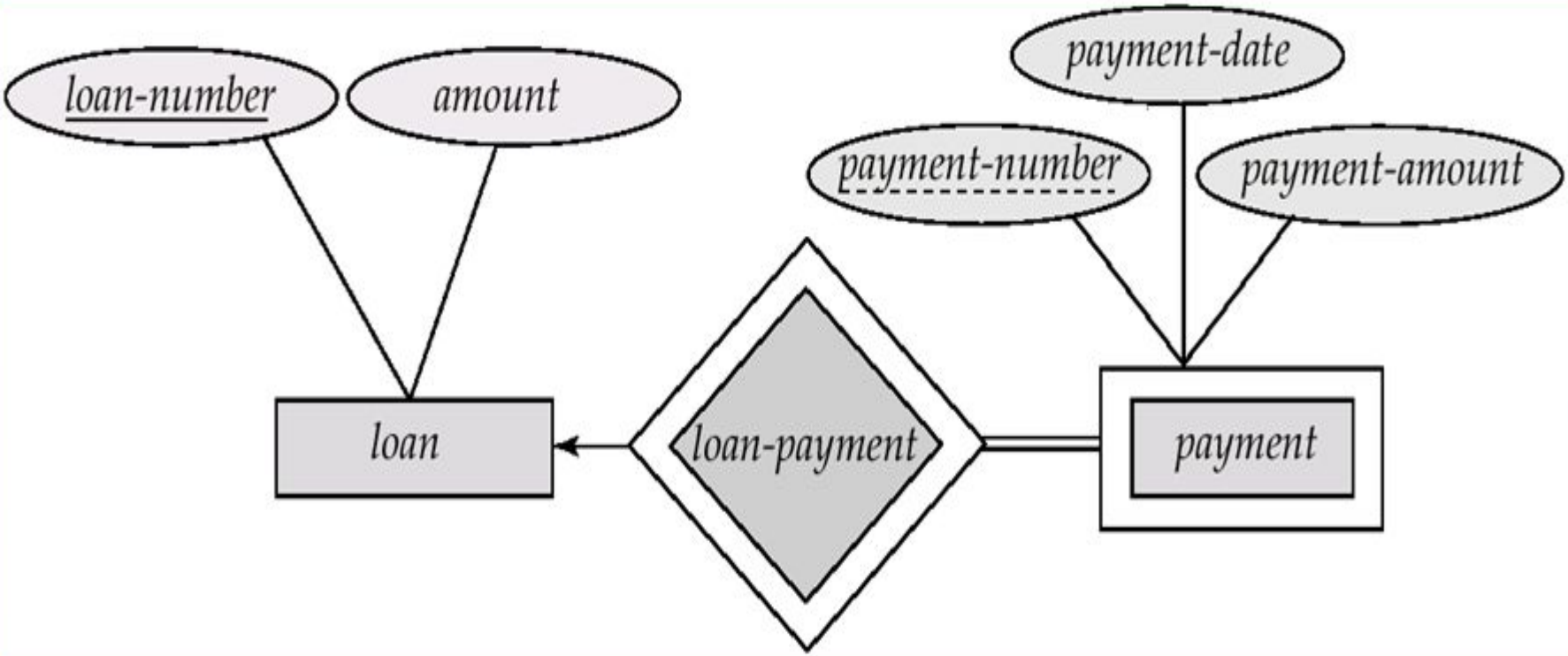


New key: bank.code + bank-branch.branchno

Weak Entity Samples



Weak Entity Samples



Summary of notation in ER diagrams

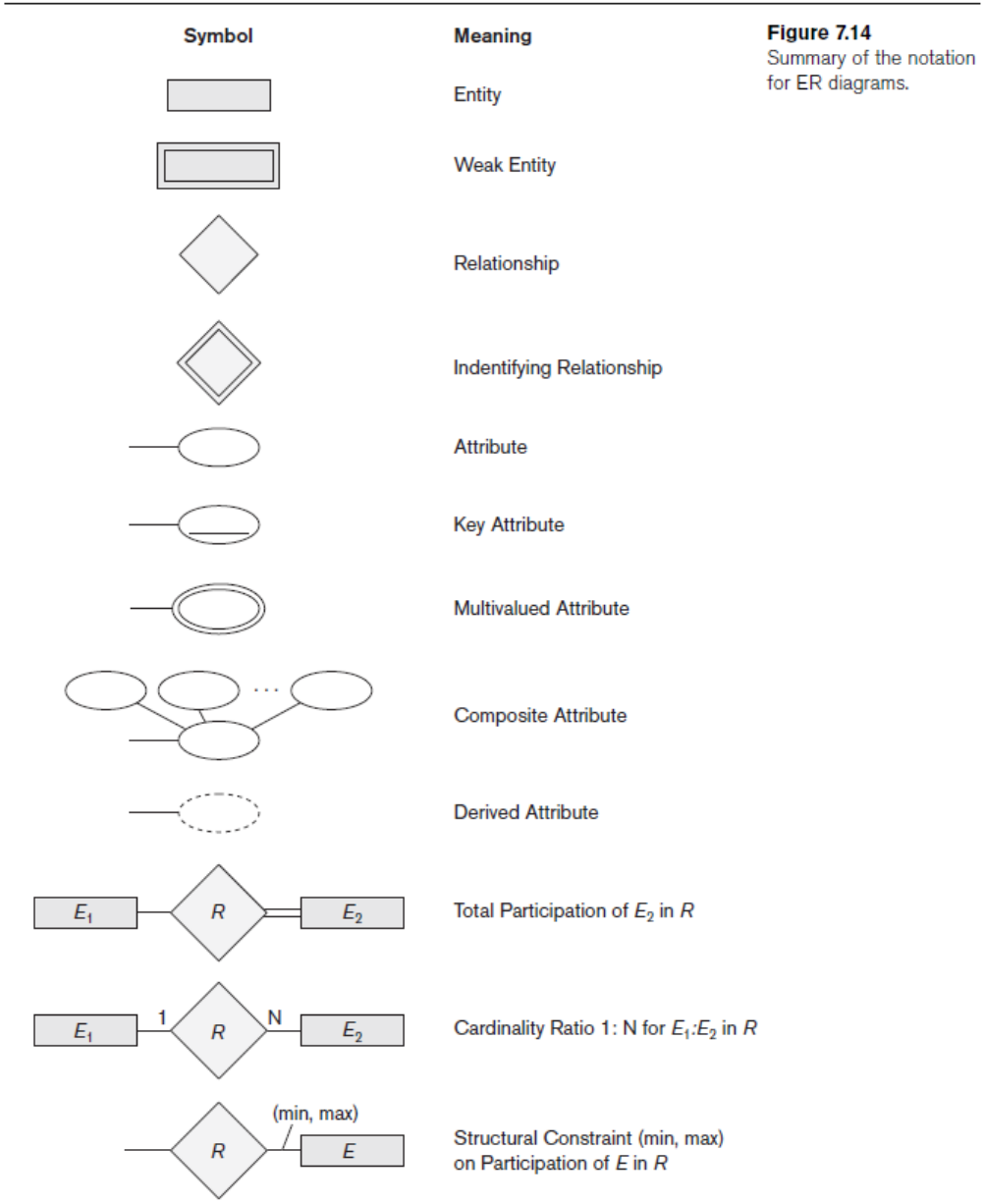
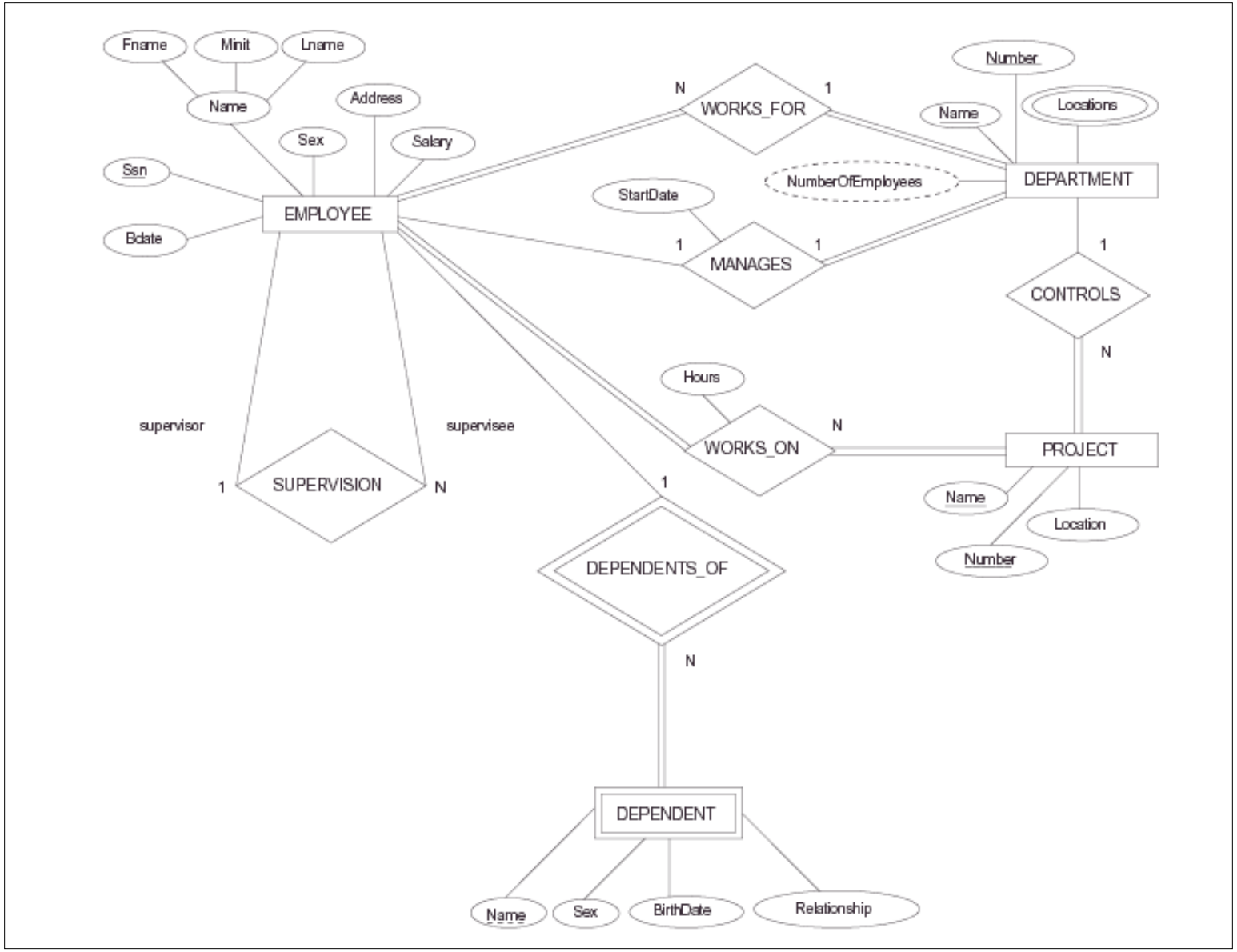


Figure 7.14
Summary of the notation for ER diagrams.

Designing a complete ER-diagram

- Iterative task: designing, going back to the users, re-designing, etc.
- Questions might be:
  - Should initial attributes better by modeled as relation types?
  - What are the cardinalities of the relations?
  - Should relations be total?
  - Should entities be split into identifying and weak entities?
- This leads to a final ER-diagram

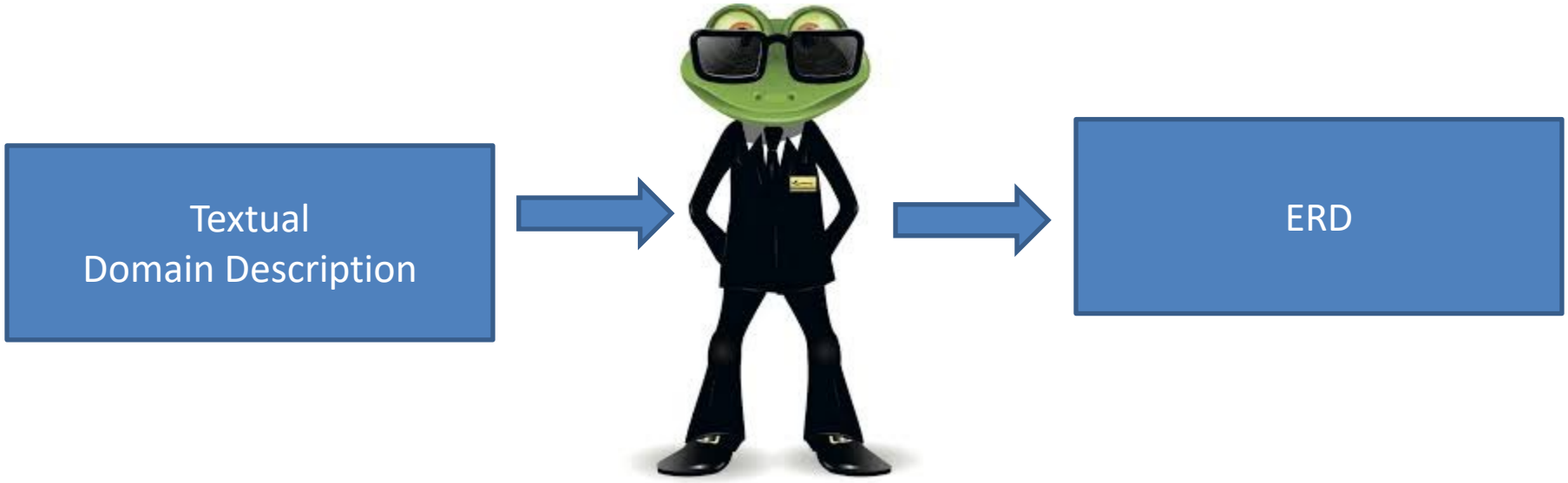

Fname
Minit
Lname
Name
Address
Sex
Salary
Ssn
Bdate
EMPLOYEE
N
WORKS_FOR
1
Number
Name
Locations
DEPARTMENT
NumberOfEmployees
StartDate
1
MANAGES
1
1
CONTROLS
N
Hours
WORKS_ON
N
PROJECT
Name
Location
Number
supervisor
supervisee
1
SUPERVISION
N
1
DEPENDENTS_OF
N
DEPENDENT
Name
Sex
BirthDate
Relationship

Data Modeling using the Entity Relationship (ER) Model

- High-Level Conceptual Data Models
- Entity Relationship Model and its elements
- From text to ER-schema

From text to ER-schema

- Often, you will create an ER diagram based on a textual description. There are some rules-of-thumb that can help you in this task



From text to ER-schema

- Noun: entity-type
  - In our **university**, **student**s follow **course**s that are given by a **teacher**.
- Verb that connects nouns: relation type
  - In our university, students **follow** courses that are **given** by a teacher.
- Adjective: attribute at entity-type
  - The **large** shop sells **expensive** cars.

From text to ER-schema

- Adverb: attribute at relation-type
  - The student **successfully** finishes the course.
  - The car is rent by the client **for a certain period of time**.

From text to ER-schema

- When you are done, inspect the diagram and improve it where possible:
  - Identify possible keys
  - Identify weak entity-types
  - Entity-types without attributes can better become attributes of a relation or another entity
  - Identify and remove redundant relations
  - Remove entity-types without relations
  - Remove entity-types with only a single entity

From text to ER-schema

- Describe the domain of all attributes
- Write down all restrictions that cannot be represented in the diagram
- Describe all operations on the data that are needed for the database application (data entry, removal, etc.)

Other restrictions

- The ER-diagram and schema cannot represent all restrictions from the mini world
- Those restrictions, however, can be extremely important when building the database application
- Write down these restrictions as a note to the diagram and schema

Summary

- Basic ER model concepts of entities and their attributes
  - Different types of attributes
  - Structural constraints on relationships

Quiz/part1

- A university database contains information about professors (identied by social security number, or SSN) and courses (identied by courseid). Professors teach courses; each of the following situations concerns the Teaches relationship set. For each situation, draw an ER diagram that describes it (assuming that no further constraints hold).
- 1. Professors can teach the same course in several semesters, and each offering must be recorded.
- 2. Professors can teach the same course in several semesters, and only the most recent such offering needs to be recorded. (Assume this condition applies in all subsequent questions.)

Quiz/Part2

- 3. Every professor must teach some course.
- 4. Every professor teaches exactly one course (no more, no less).
- 5. Every professor teaches exactly one course (no more, no less), and every course must be taught by some professor.
- 6. Now suppose that certain courses can be taught by a team of professors jointly, but it is possible that no one professor in a team can teach the course.

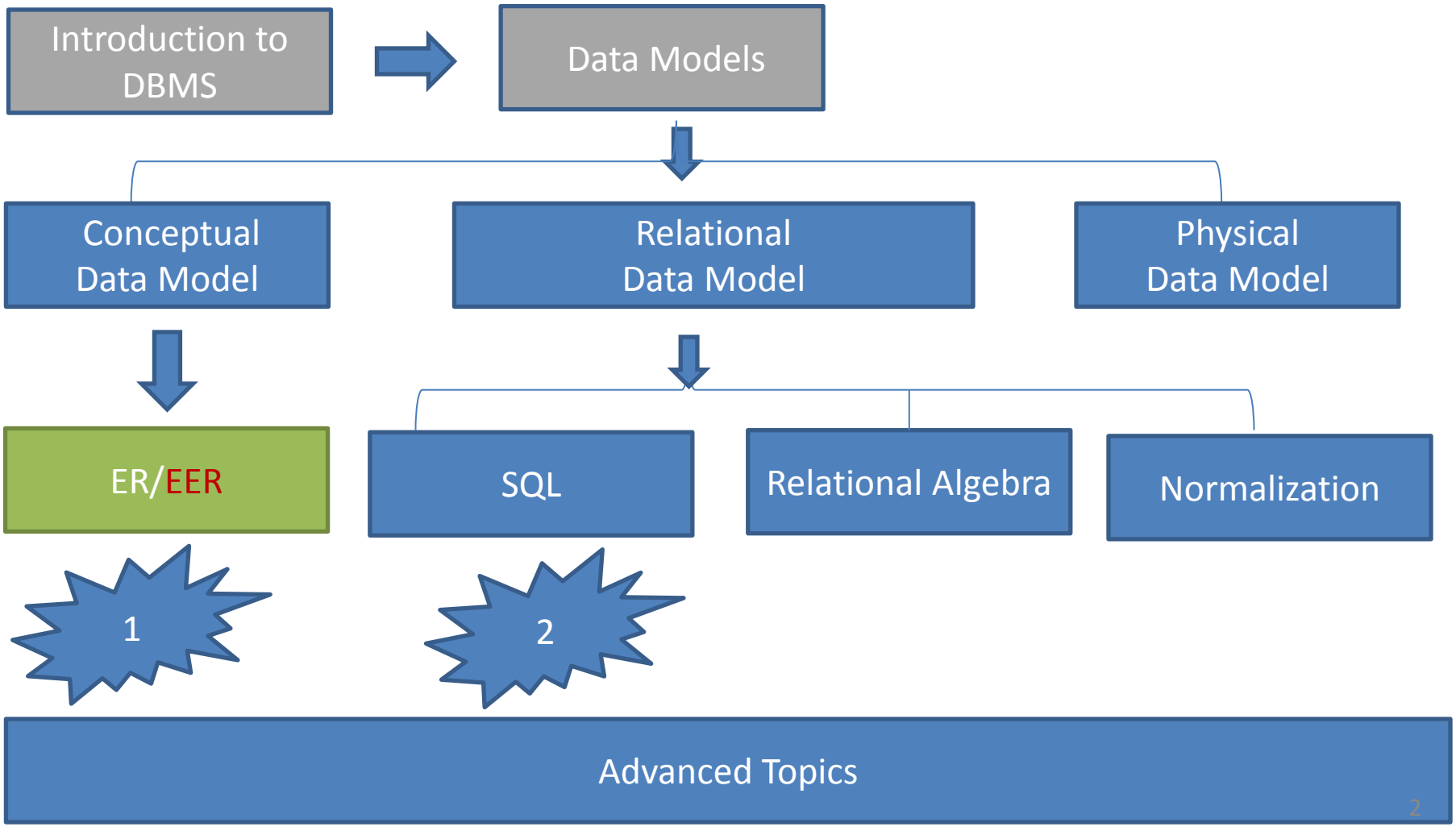
An Introduction to the Database Management Systems

By
Hossein Rahmani

Slides originally by Book(s) Resources

Road Map

(Might change!)

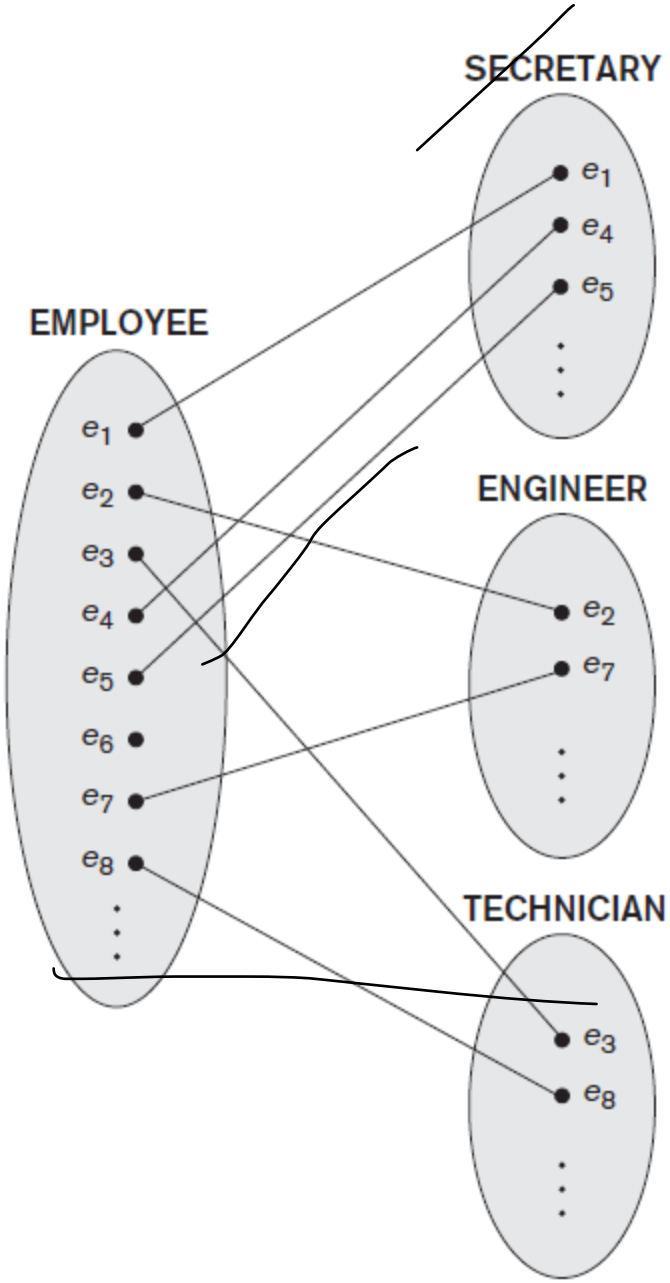


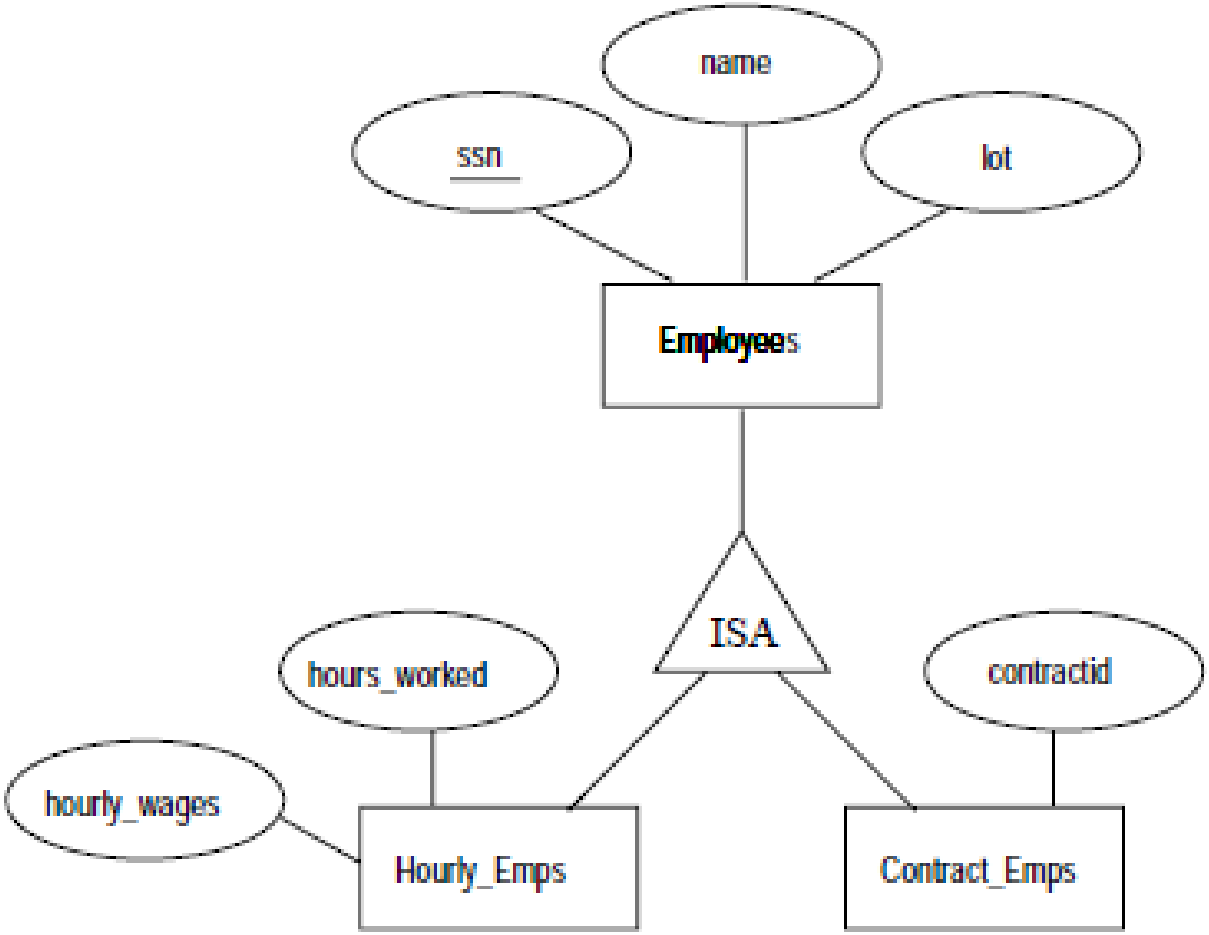
The Enhanced Entity-Relationship (EER) Model

- Subclasses, Superclasses
- Specialization and Generalization
- Constraints In Specialization/Generalization
- Modeling of UNION Types Using Categories
- Aggregation

Entity types in ER diagrams

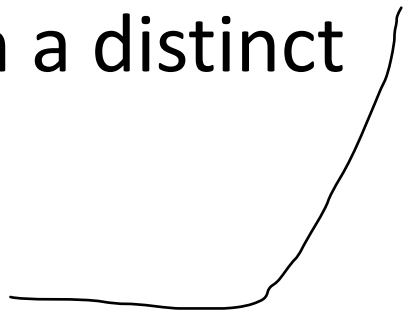
- Until now, each entity was assigned to one entity type
- In many cases an entity type has numerous subgroupings or subtypes of its entities that are meaningful and need to be represented explicitly
- Members of the EMPLOYEE entity type may be distinguished further into SECRETARY, ENGINEER, MANAGER, etc


SECRETARY
e_1
e_4
e_5
EMPLOYEE
e_1
e_2
e_3
e_4
e_5
e_6
e_7
e_8
ENGINEER
e_2
e_7
TECHNICIAN
e_3
e_8


name
ssn
lot
Employees
ISA
hours_worked
contractid
hourly_wages
Hourly_Emps
Contract_Emps

Subclasses and Superclasses

- Each of these subgroupings is called a **subclass** or **subtype** of the EMPLOYEE entity type, and the EMPLOYEE entity type is called the **superclass** or **supertype**
- Each subclass member is the same as the entity in the superclass, but in a distinct *specific role*



Subclasses and Superclasses

- An entity type *A* is a **subclass** of *B* if all entities of type *A* are always also of type *B* (but not vice versa)
- Entity type *B* is a **superclass** of *A*
- A subclass *inherits* all attributes and relation types of its superclass
- Entities belong to *more than one* entity type!

The Enhanced Entity-Relationship (EER) Model

- Subclasses, Superclasses
- Specialization and Generalization 
- Constraints In Specialization/Generalization
- Modeling of UNION Types Using Categories
- Aggregation

Specialization

- *Specialization* is identifying a set of subclasses of an entity type
  - Additional attributes in the subclasses are called *specific* (or *local*) attributes
  - Additional relations on the subclasses are called specific (or local) relations
  - There can be more than one specialization of an entity type simultaneously!
- Example:
  - specialize student to graduate and freshman;
  - specialize student to field of study

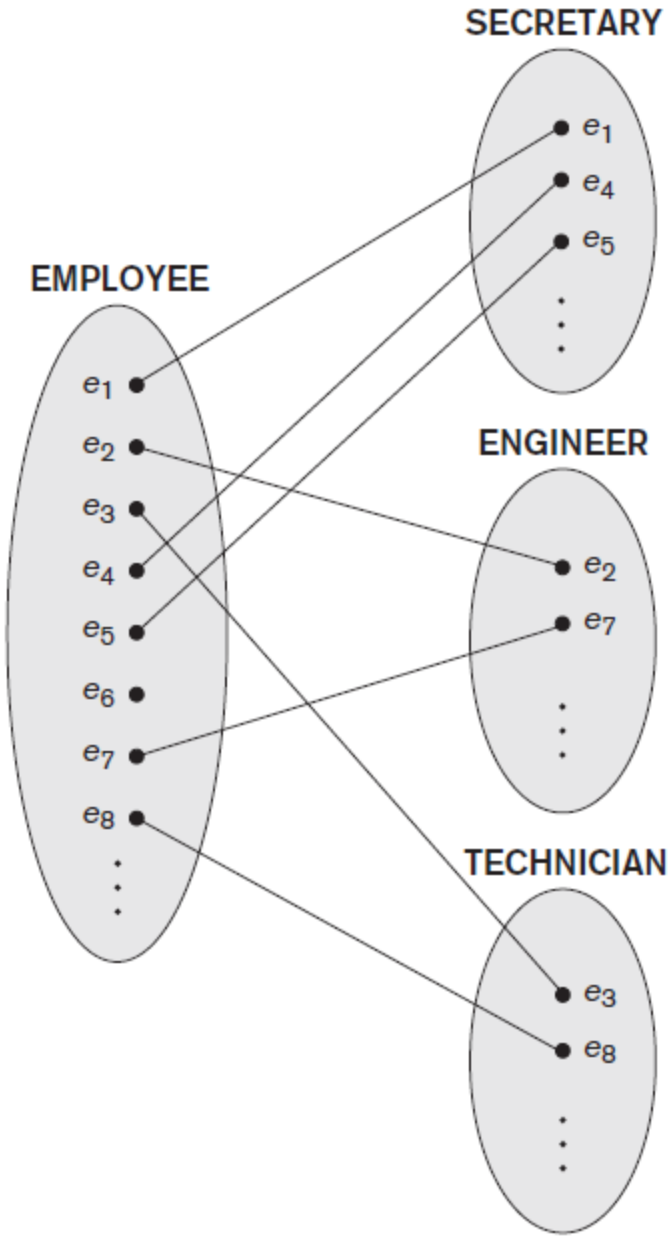
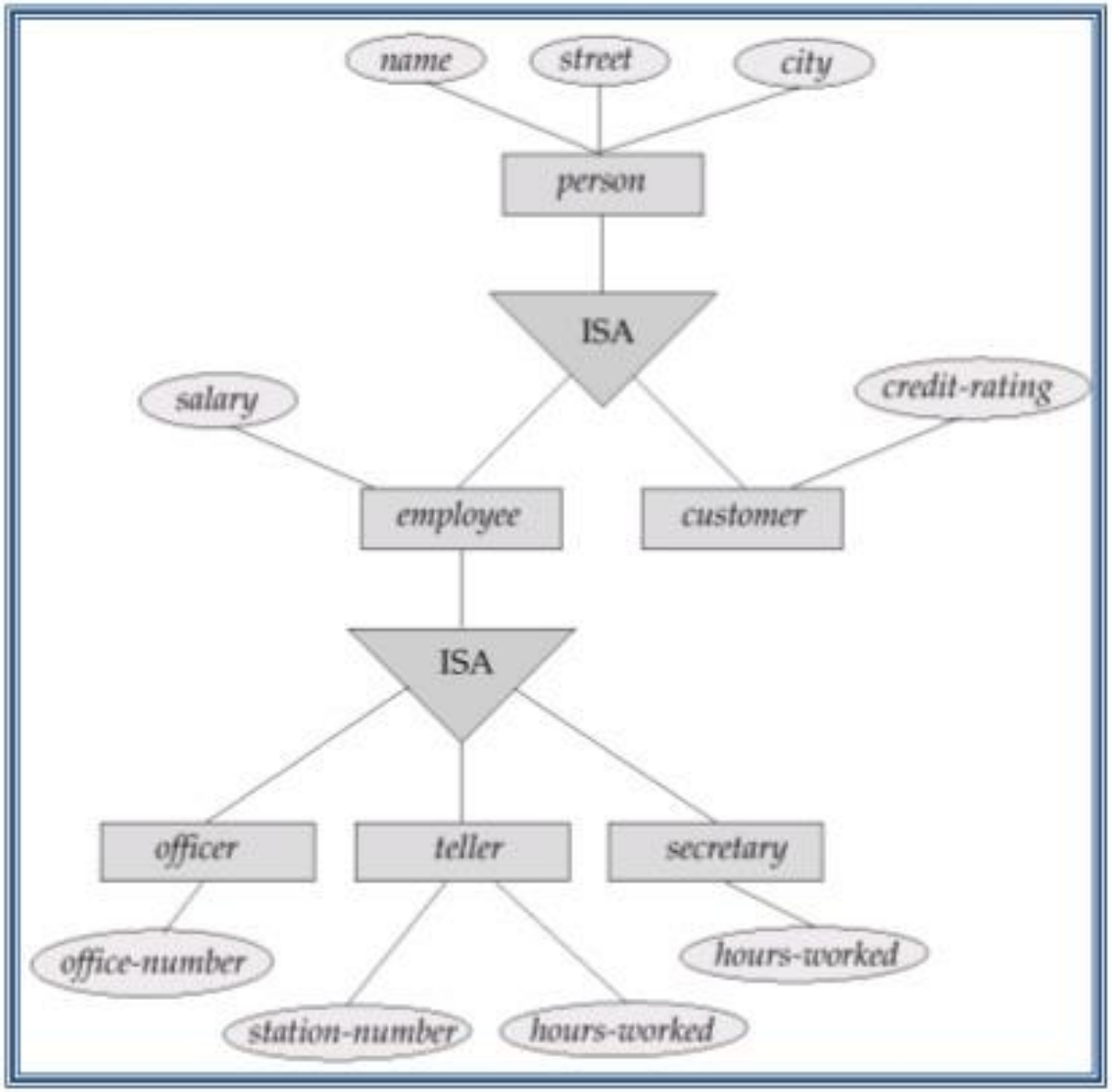
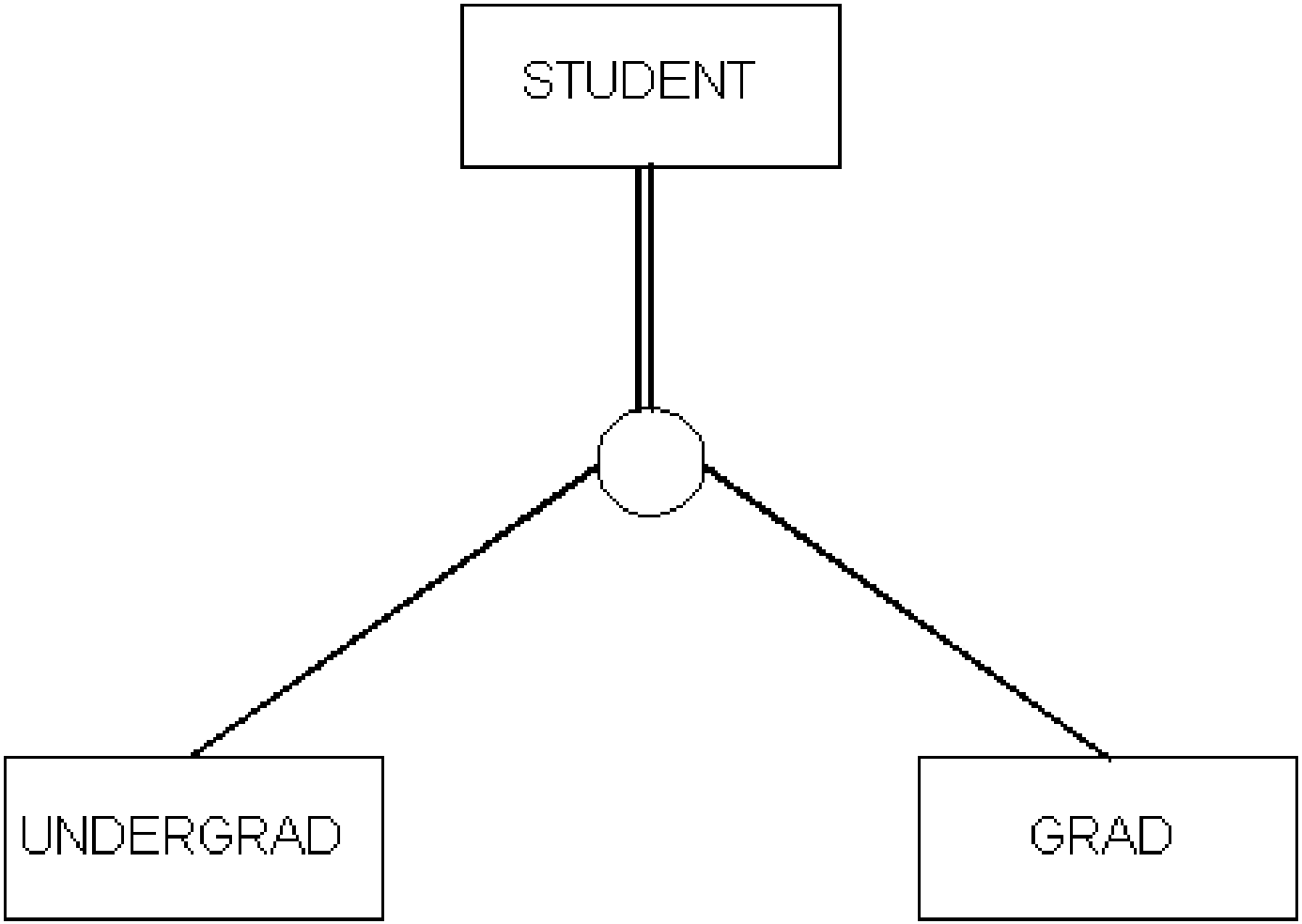


Figure 8.2
Instances of a specialization.

Specialization Example

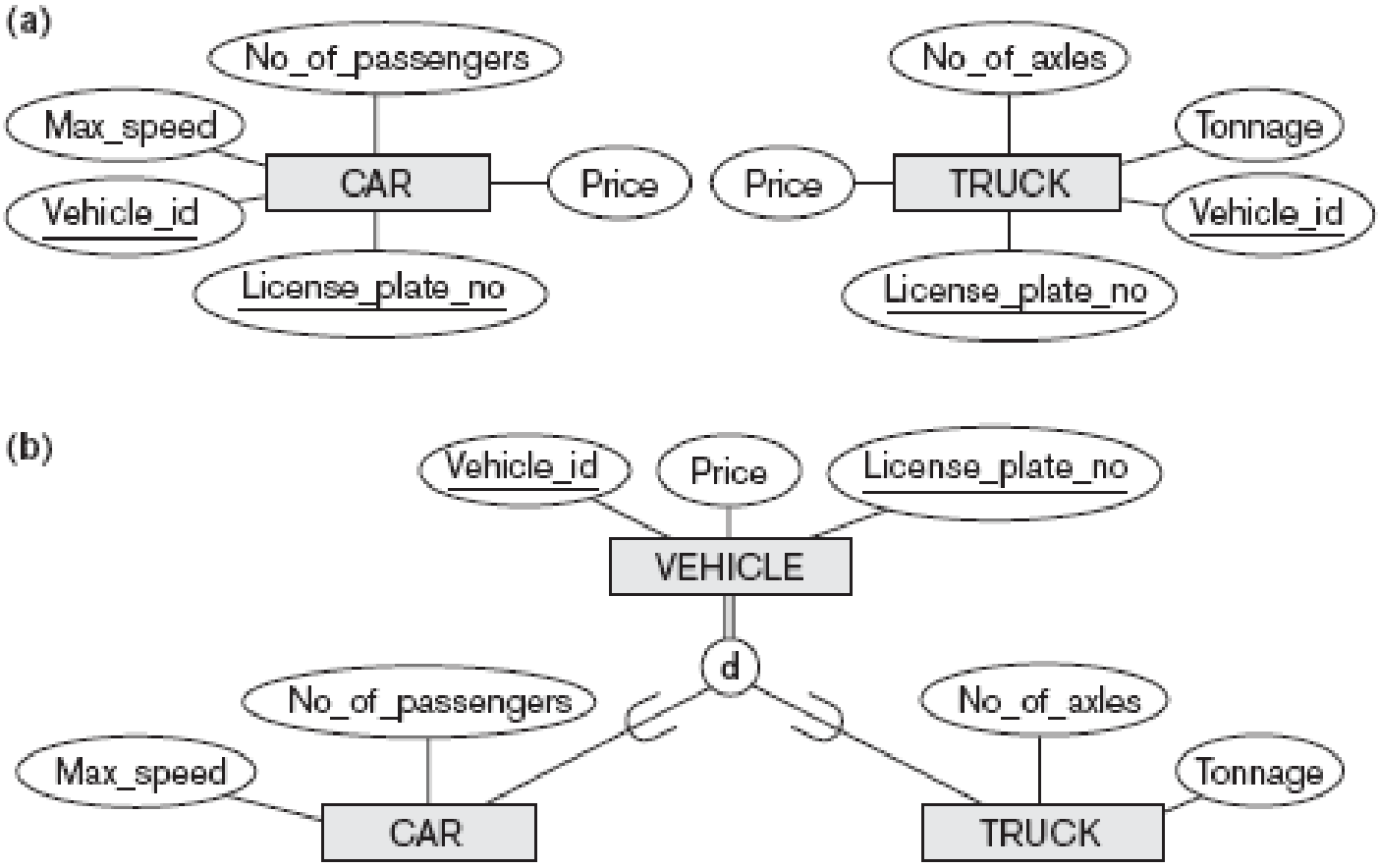



STUDENT
UNDERGRAD
GRAD

Generalization

- Generalization is the construction of an entity type that embodies the common properties (attributes and relations) of a number of given entity types
  - The new entity type is the generalized superclass of the entity types
  - An entity type can participate in multiple generalizations
- Example:
  - generalize bicycle and car to vehicle

Example


(a)
No_of_passengers
Max_speed
CAR
Price
Vehicle_id
License_plate_no
No_of_axles
Tonnage
Price
TRUCK
Vehicle_id
License_plate_no
(b)
Vehicle_id
Price
License_plate_no
VEHICLE
d
No_of_passengers
Max_speed
CAR
No_of_axles
Tonnage
TRUCK

15

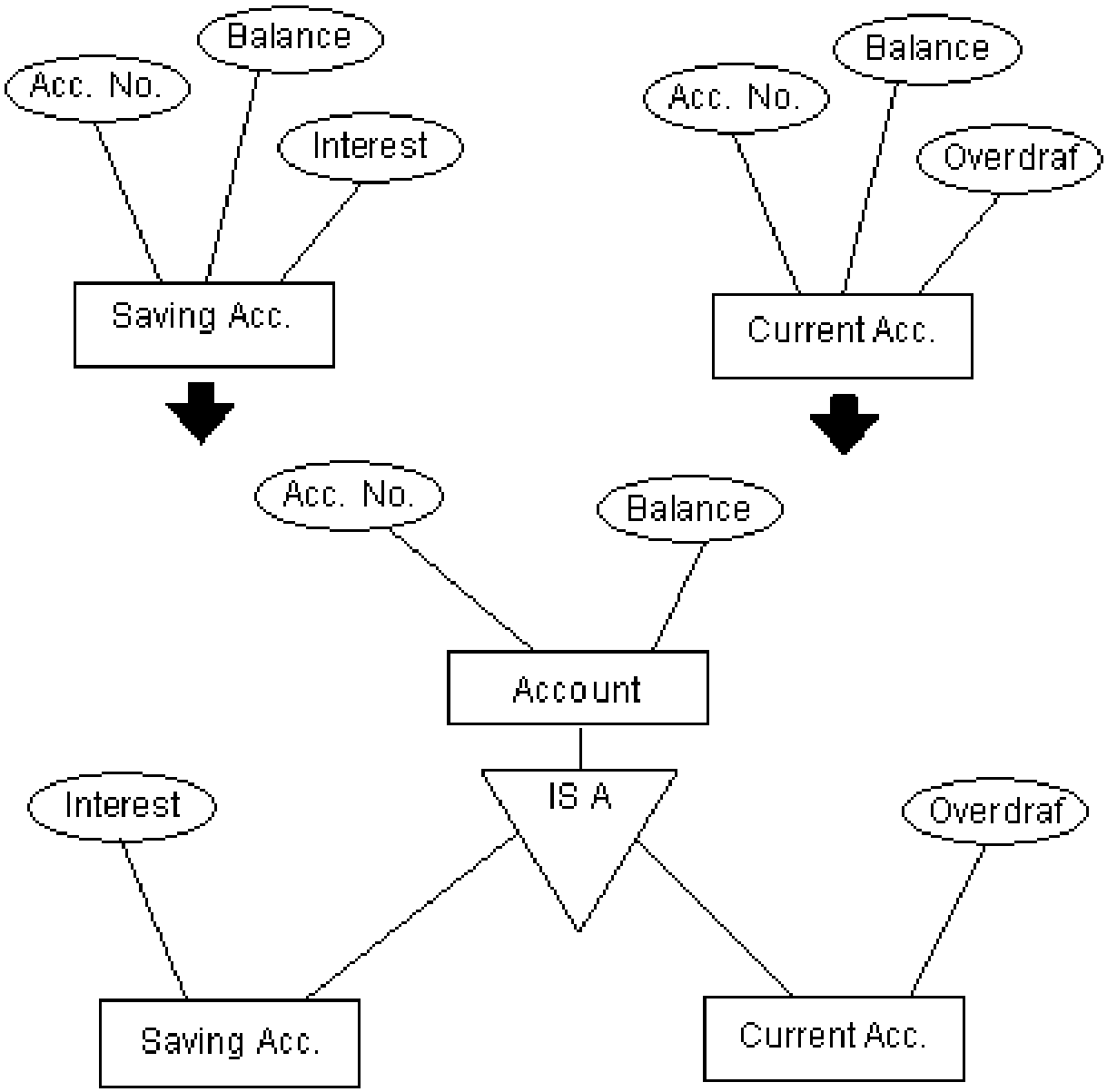
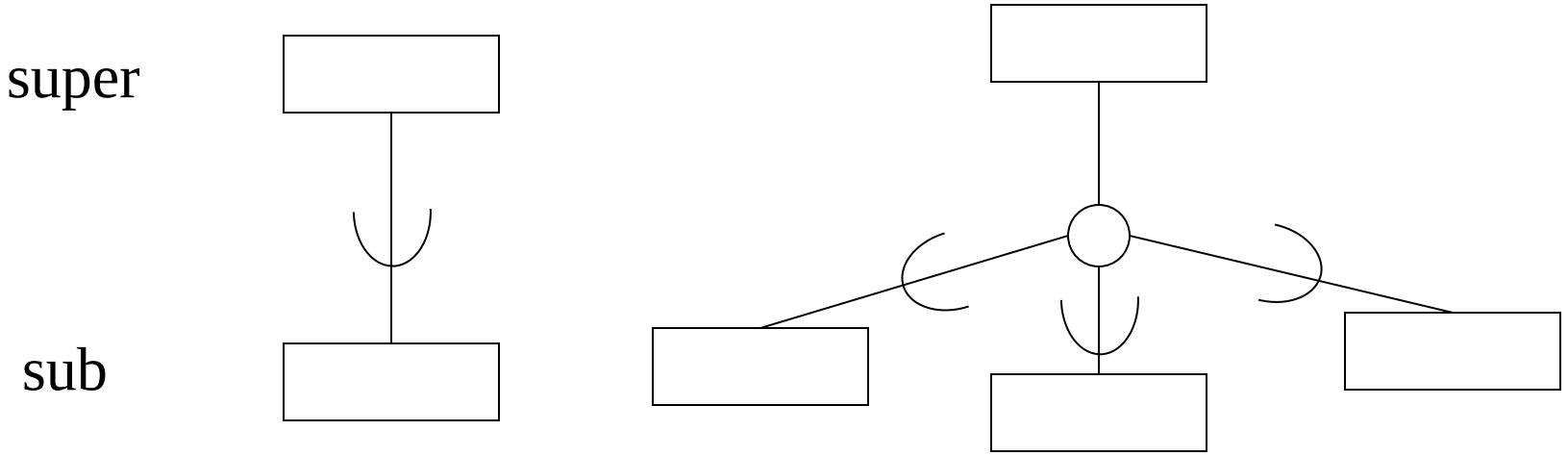


Figure 3.7 : Generalization procedure

EER construct

- EER (extended ER model / schema) contains one construct for specialization and generalization together:



The Enhanced Entity-Relationship (EER) Model

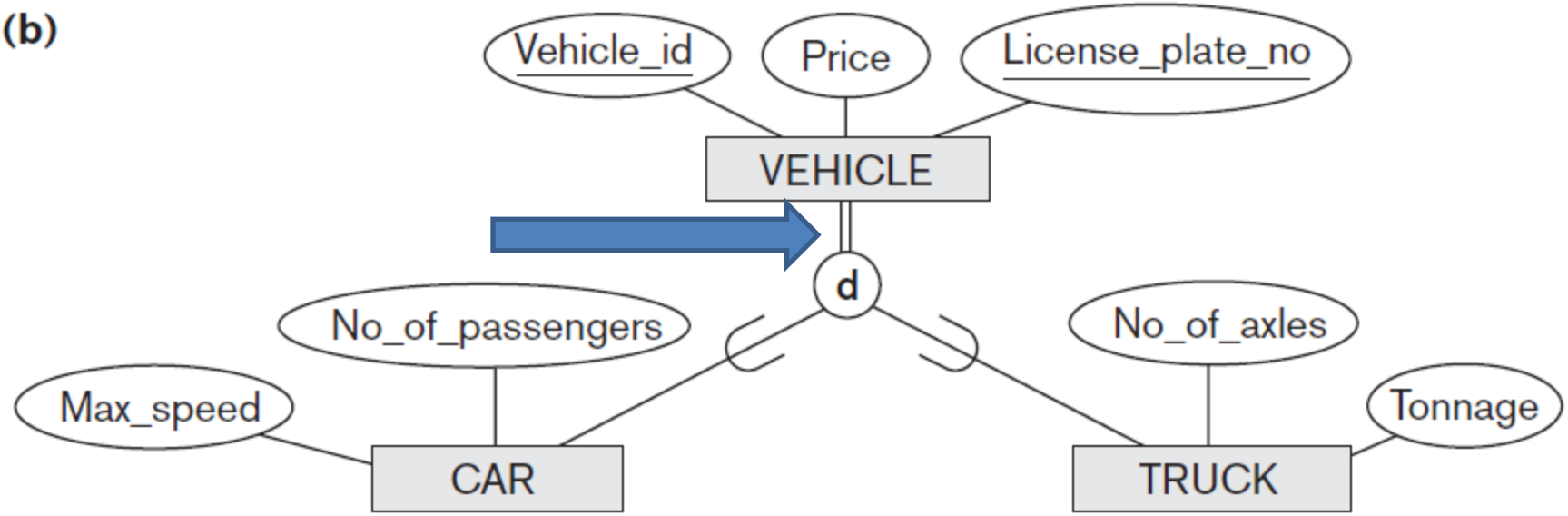
- Subclasses, Superclasses
- Specialization and Generalization
- Constraints In Specialization/Generalization
- Modeling of UNION Types Using Categories
- Aggregation

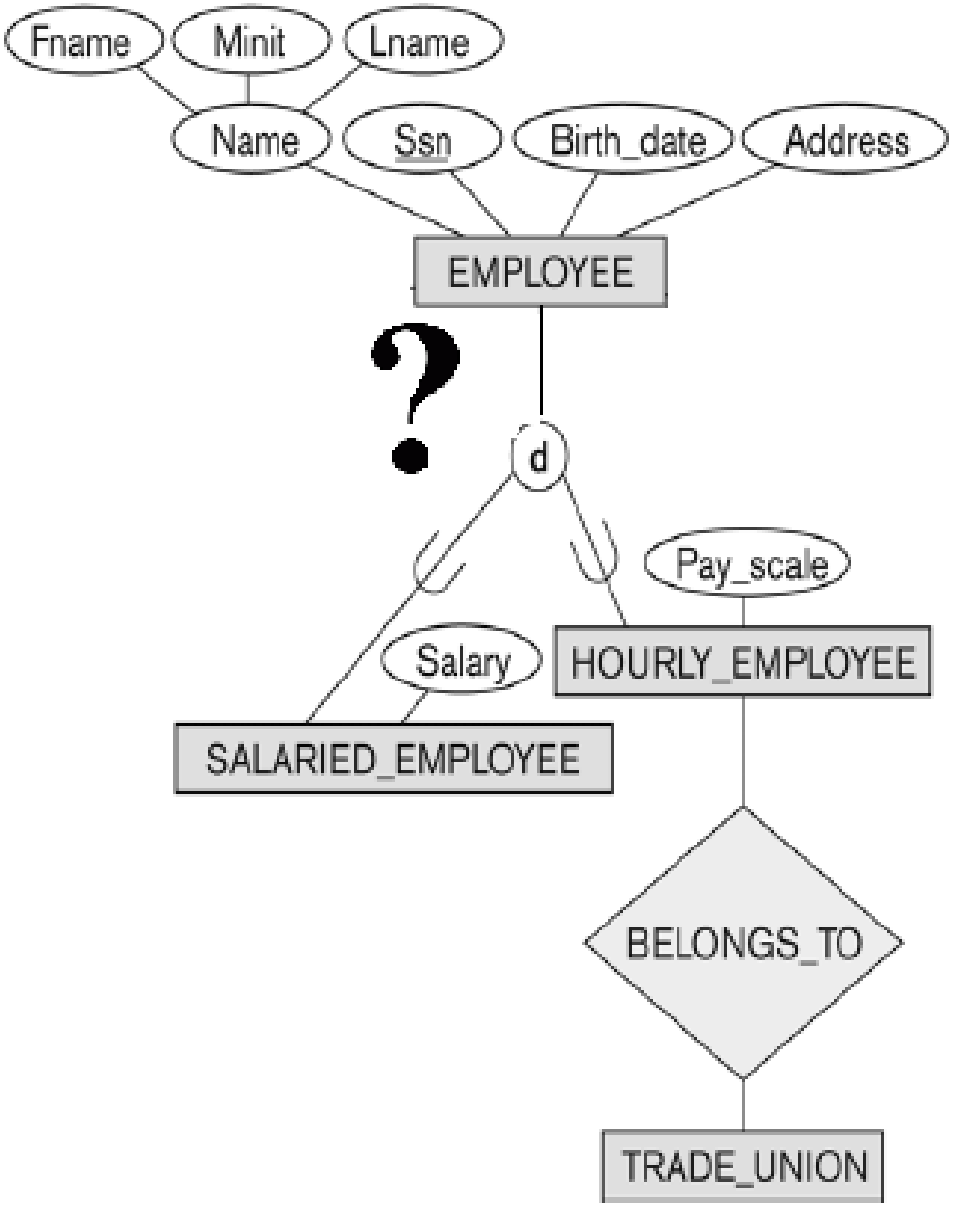
Restrictions
Total Vs Option

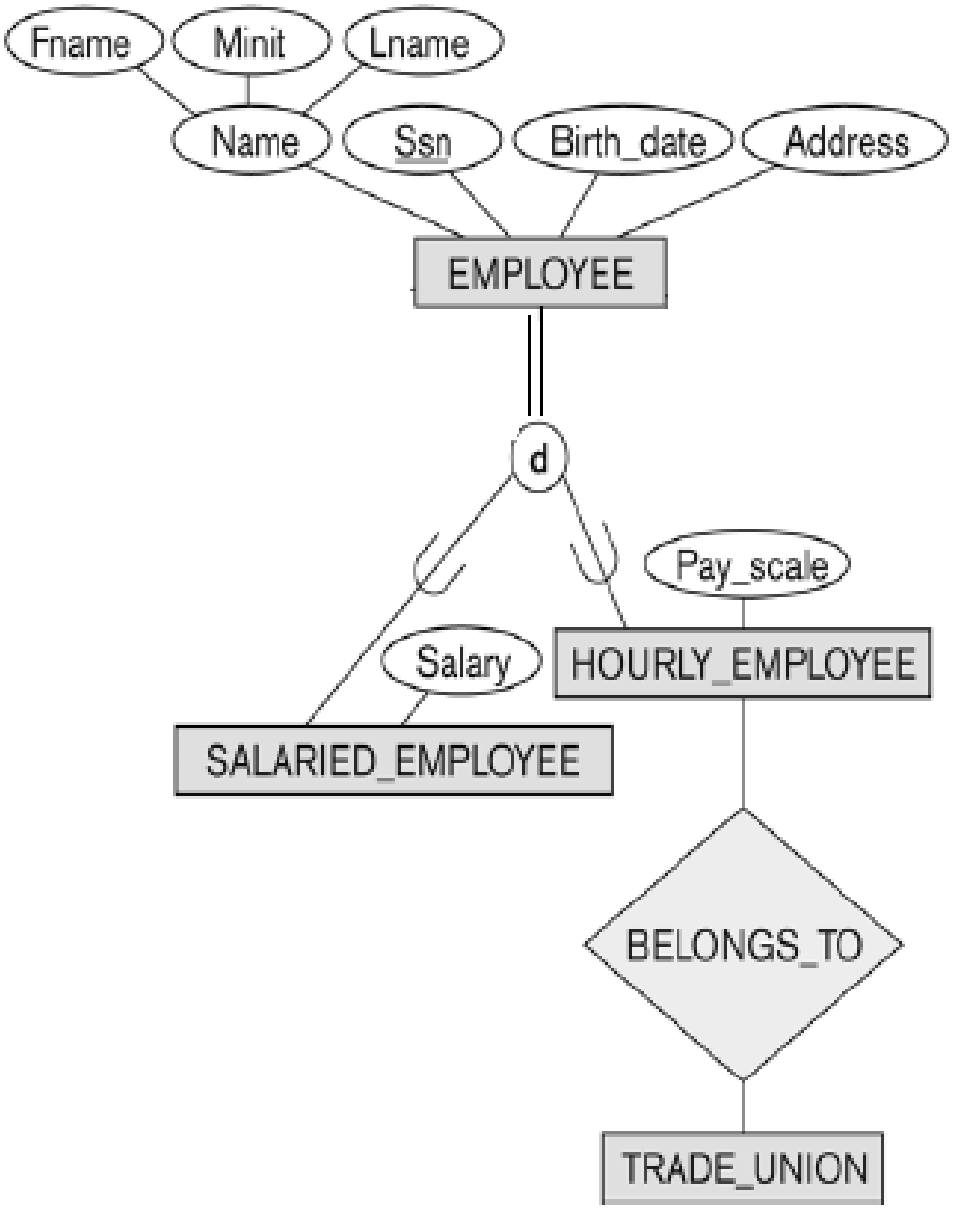
- **Total:**
  - *every* entity in the superclass must be a member of at least one subclass in the specialization
  - **total**: double line from superclass;
  - **partial**: optional participation in specialization

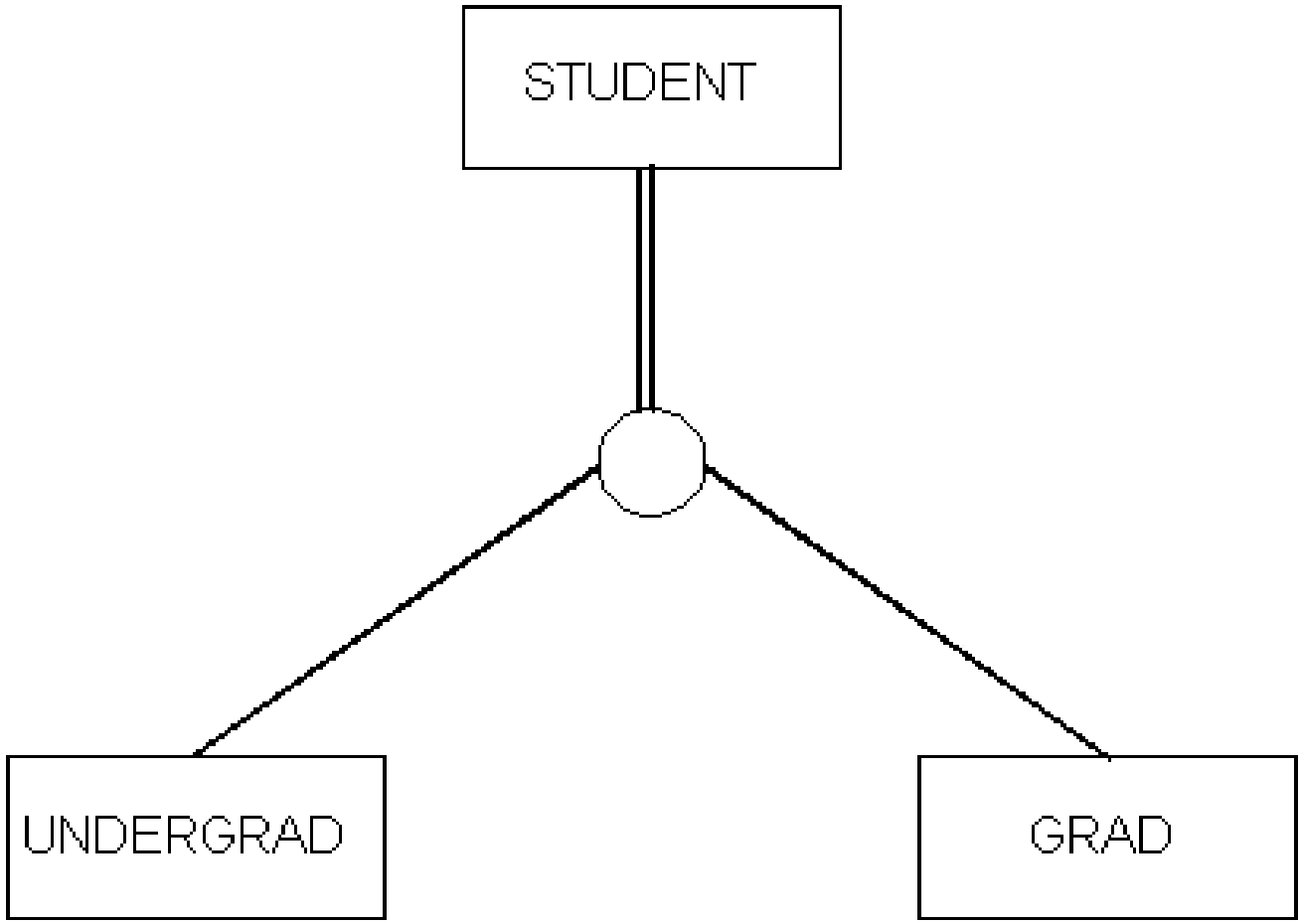
19

(b)


Vehicle_id
Price
License_plate_no
VEHICLE
d
No_of_passengers
Max_speed
CAR
No_of_axles
Tonnage
TRUCK


Fname
Minit
Lname
Name
Ssn
Birth_date
Address
EMPLOYEE
?
d
Pay_scale
Salary
HOURLY_EMPLOYEE
SALARIED_EMPLOYEE
BELONGS_TO
TRADE_UNION

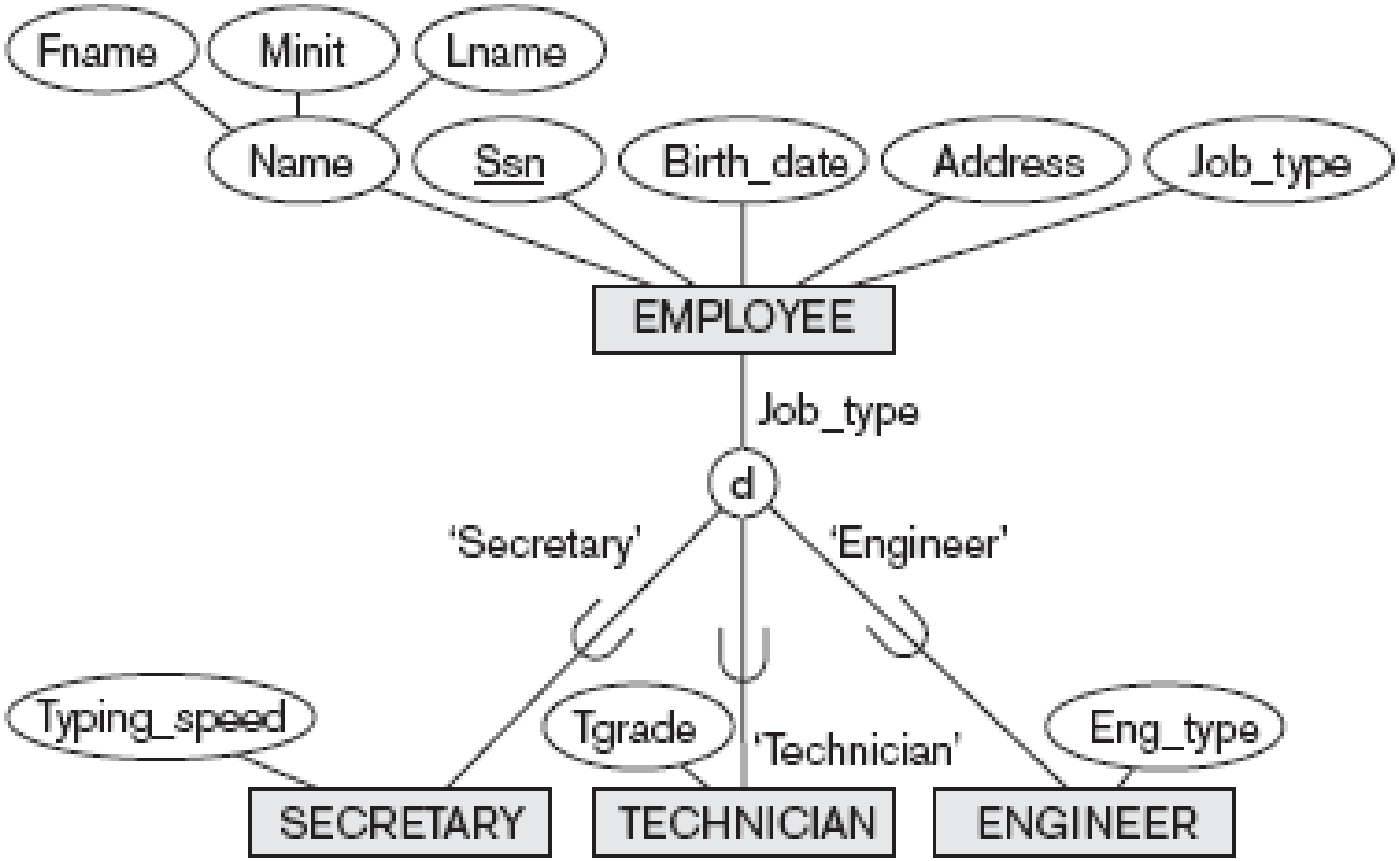

Fname
Minit
Lname
Name
Ssn
Birth_date
Address
EMPLOYEE
d
Pay_scale
Salary
HOURLY_EMPLOYEE
SALARIED_EMPLOYEE
BELONGS_TO
TRADE_UNION


STUDENT
UNDERGRAD
GRAD

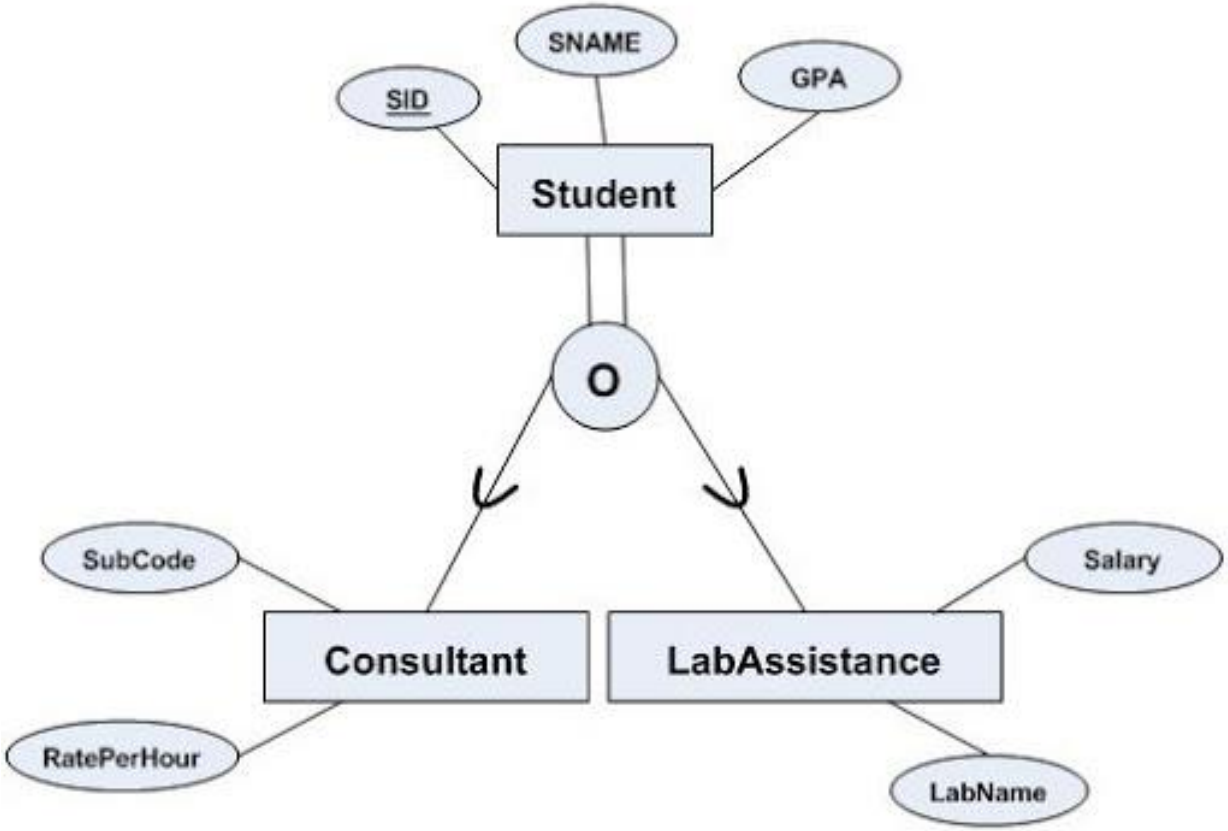
Restrictions
Disjoint Vs overlapping

- Disjoint:
  - entity can be a member of at most one of the subclasses of the specialization.
  - Letter **d**
- Overlapping:
  - Entity may be a member of more than one subclass of the specialization.
  - Letter **o**

Example 1: partial/disjoint



Example 2: total/overlapping

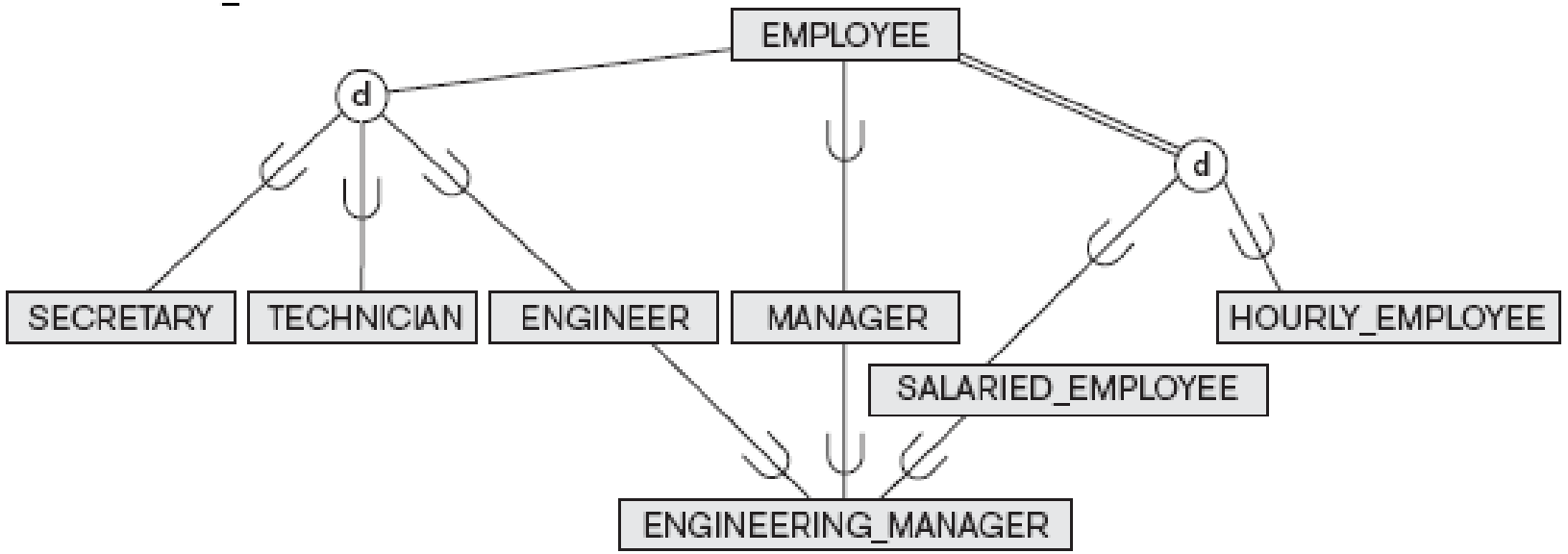


Hierarchy and Lattice

- Entity types can have complex sub/superclass relations
  - more specializations in a row
  - more superclasses
  - shared subclasses
- The sub/superclass-relation is *transitive*
  - So if *A* is-a *B* and *B* is-a *C*, then *A* is-a *C*
- Cycles are not allowed! (DAG)

Example 1: specialization lattice

A specialization lattice with shared subclass ENGINEERING_MANAGER.



Example 2: specialization lattice

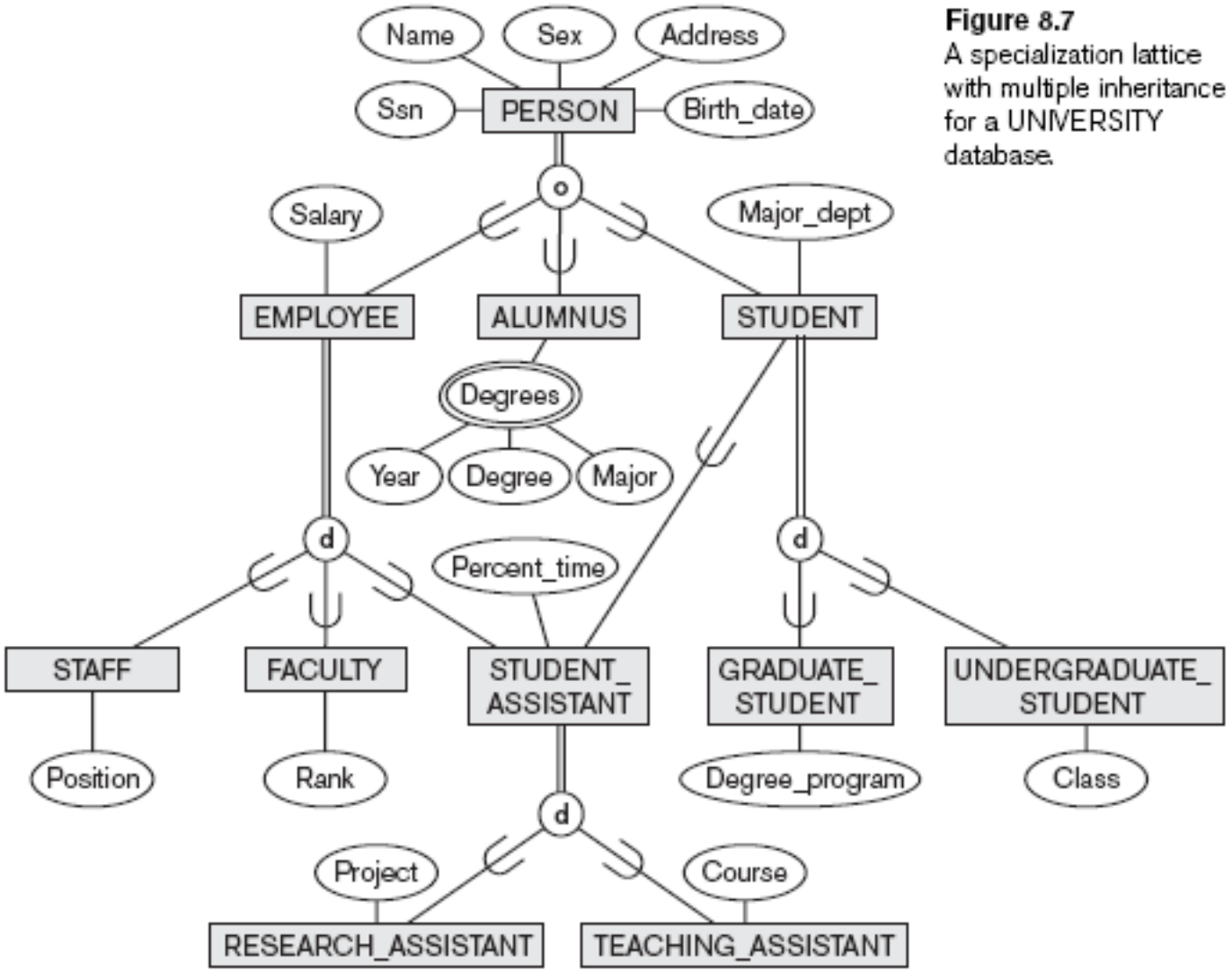


Figure 8.7
A specialization lattice with multiple inheritance for a UNIVERSITY database.

Multiple Inheritance

- **Multiple inheritance**
  - Subclass with more than one superclass
  - If attribute (or relationship) originating in the same superclass inherited more than once via different paths in lattice
    - Included only once in shared subclass
- **Single inheritance**
  - Some models and languages limited to single inheritance

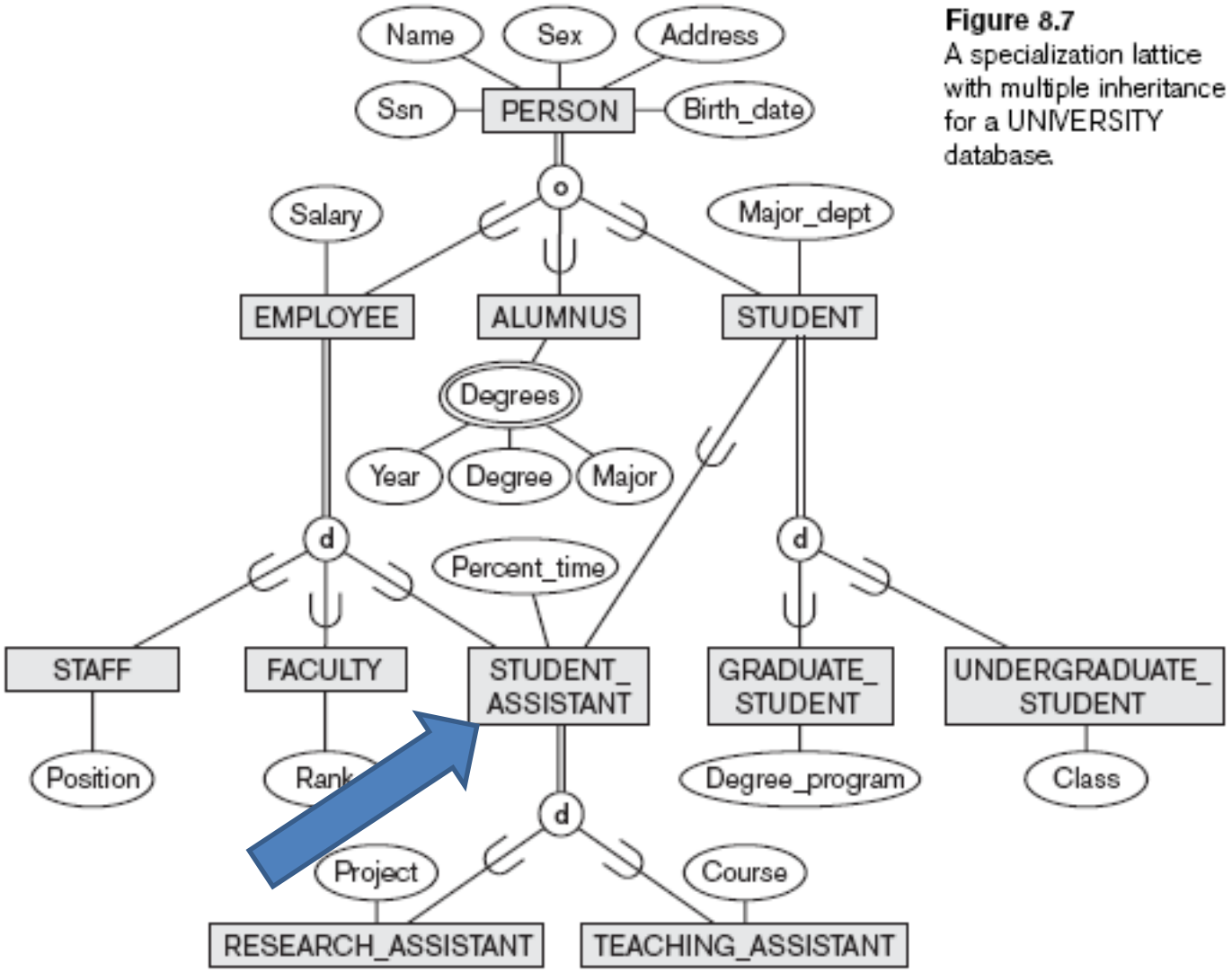
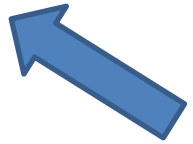


Figure 8.7
A specialization lattice with multiple inheritance for a UNIVERSITY database.

The Enhanced Entity-Relationship (EER) Model

- Subclasses, Superclasses
- Specialization and Generalization
- Constraints In Specialization/Generalization
- Modeling of UNION Types Using Categories
- Aggregation

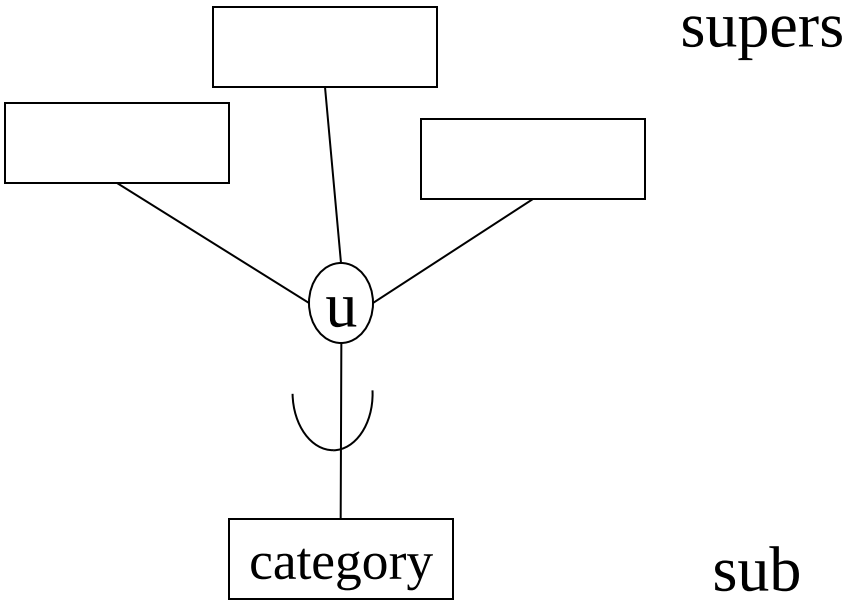


Categories

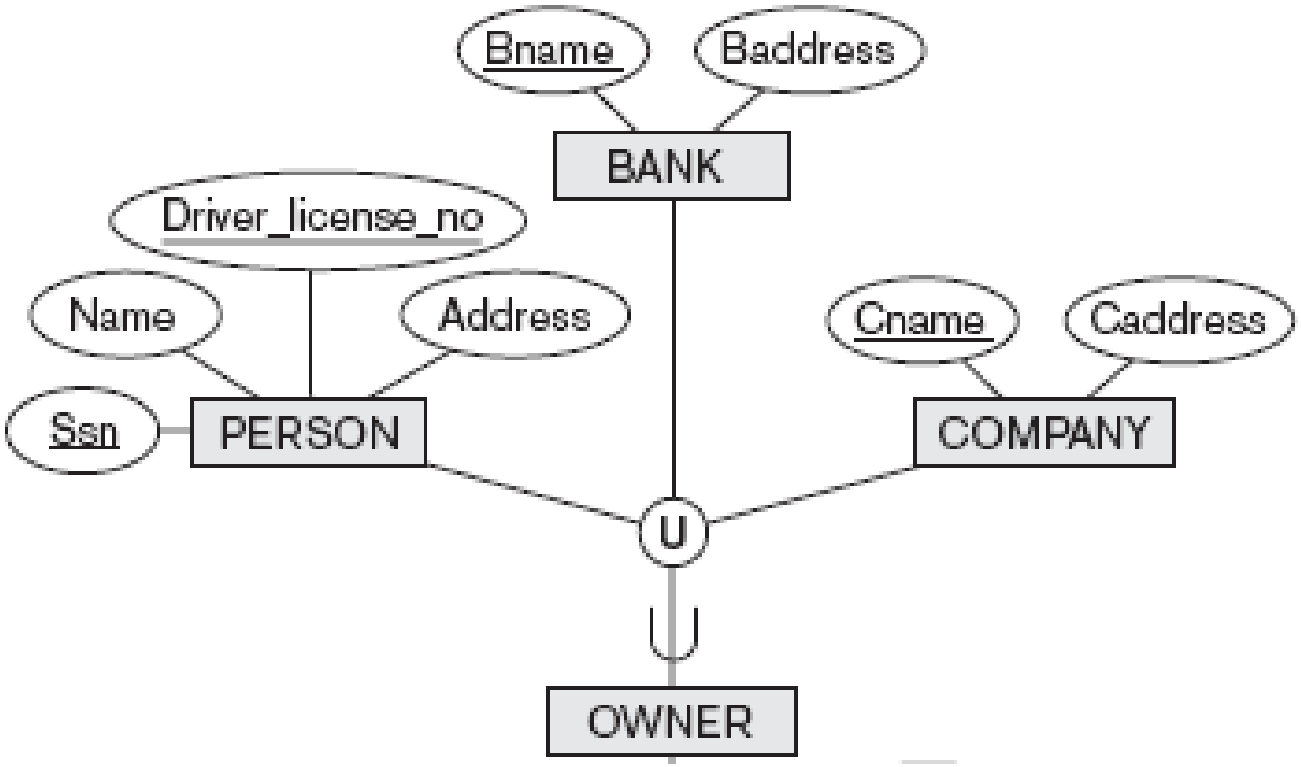
- Sometimes we want to create a subclass out of the *union* of some (different) entity types
  - Example: OWNER from PERSON and COMPANY
- This is called a *category* or *union-class*
- One subclass has more than one superclass, but these are not UNION-compatible!
- *union-compatible:* that is, the two relations must have the same set of attributes
- An entity from the category belongs to exactly one of the superclasses, not more

EER construct

- Letter **u** in the circle
- Category can participate totally or optionally
  - total: double line to category



Example: category



35

A Sample UNIVERSITY EER Schema

- The UNIVERSITY Database Example
  - UNIVERSITY database
    - Students and their majors
    - Transcripts, and registration
    - University's course offerings

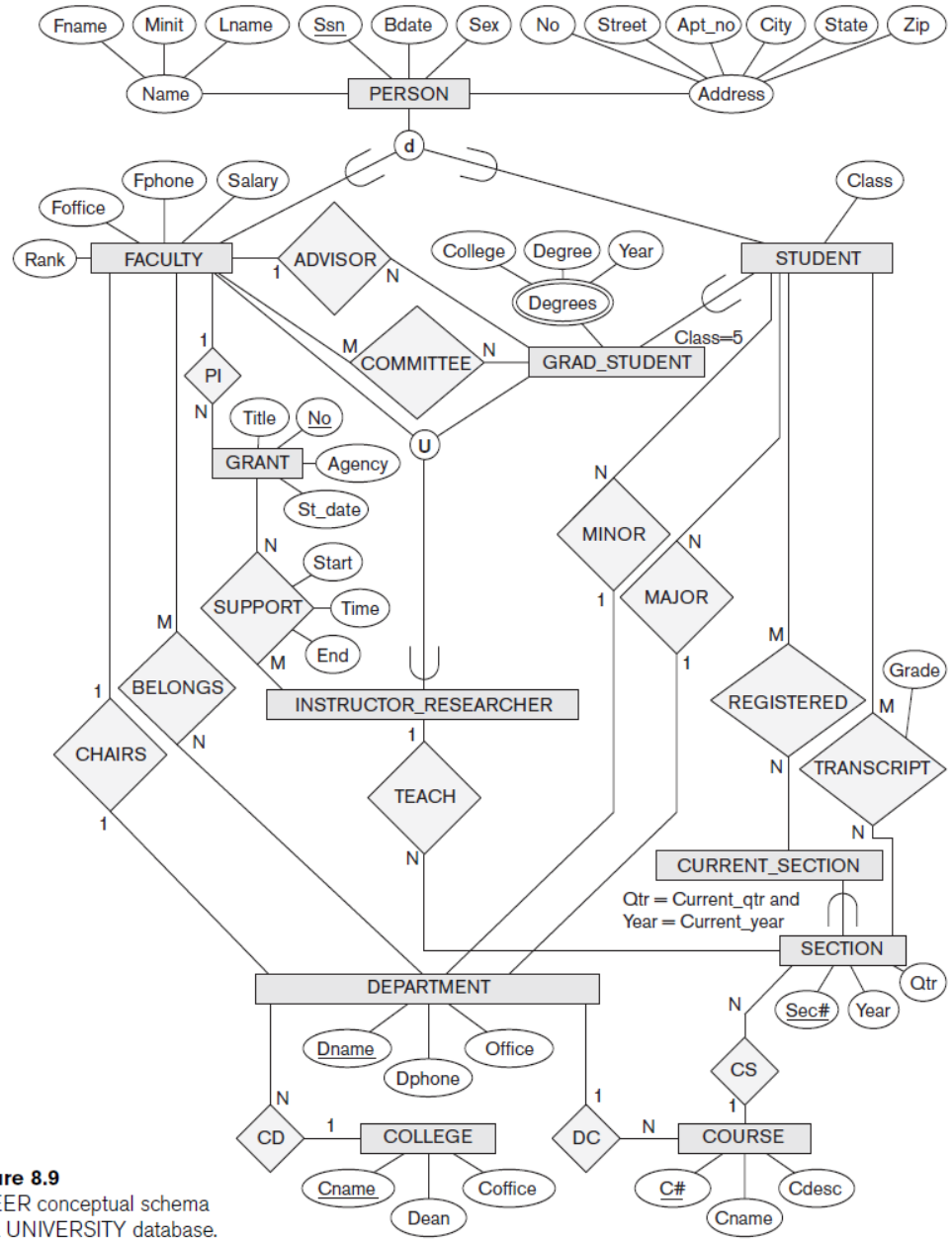


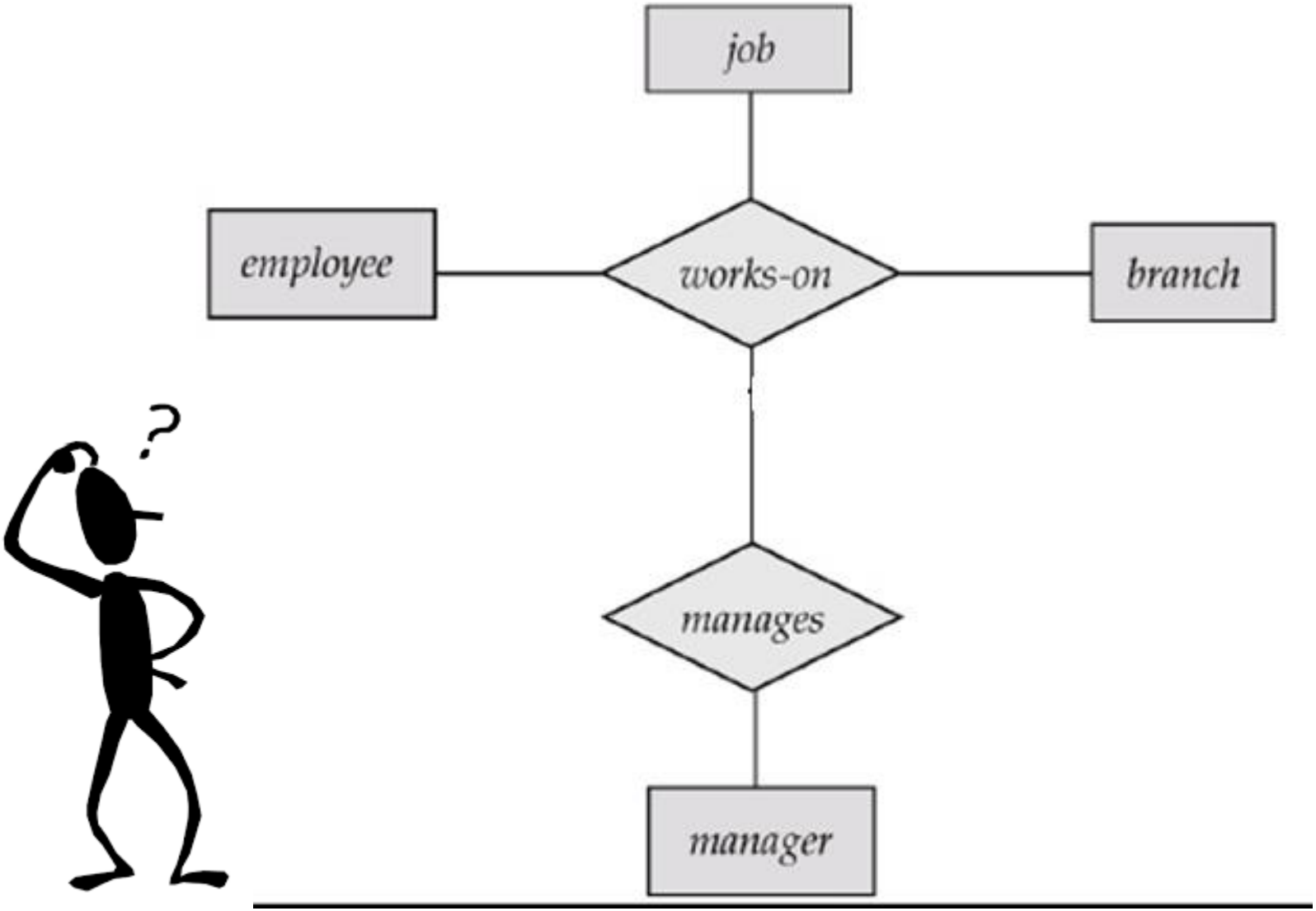
Figure 8.9
An EER conceptual schema for a UNIVERSITY database.

The Enhanced Entity-Relationship (EER) Model

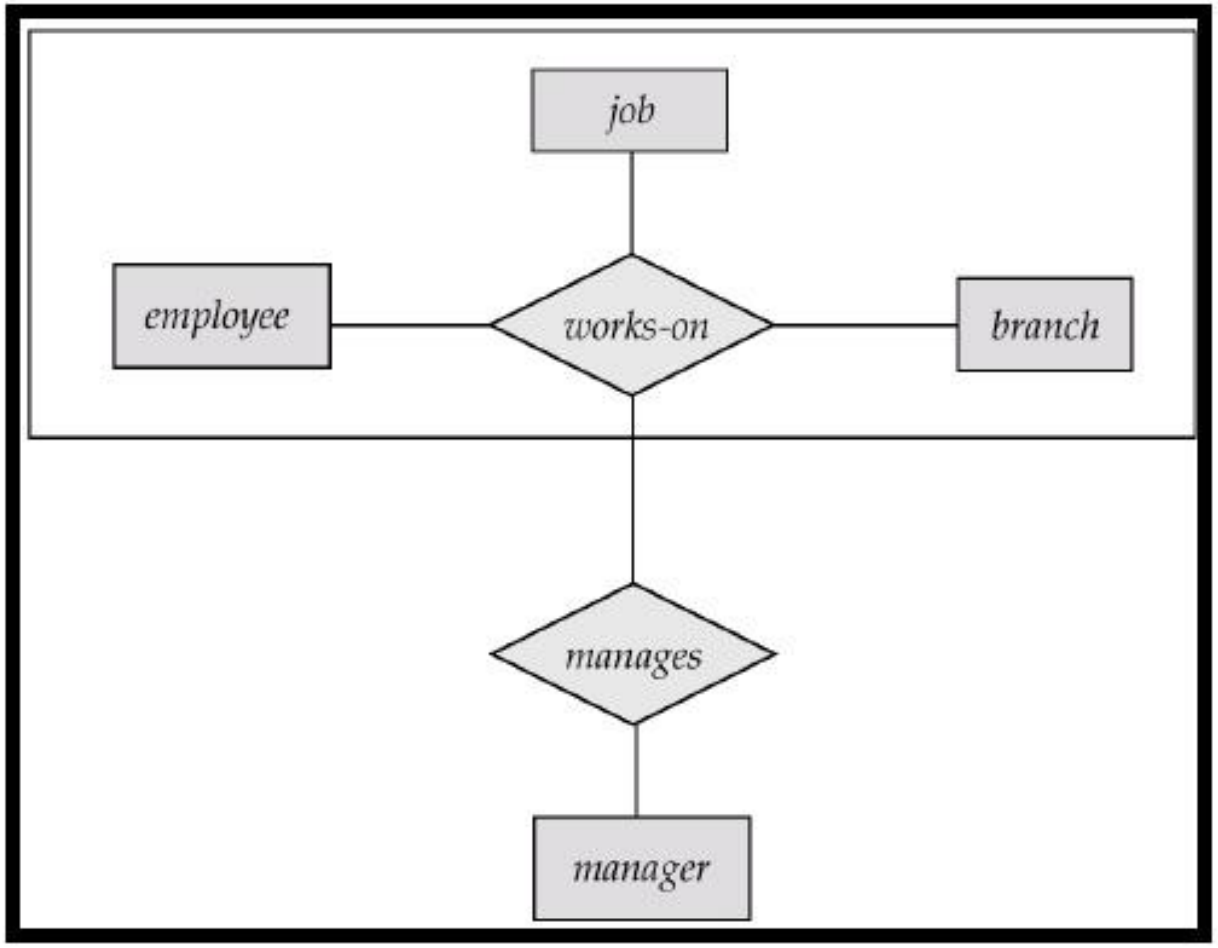
- Subclasses, Superclasses
- Specialization and Generalization
- Constraints In Specialization/Generalization
- Modeling of UNION Types Using Categories
- Aggregation 

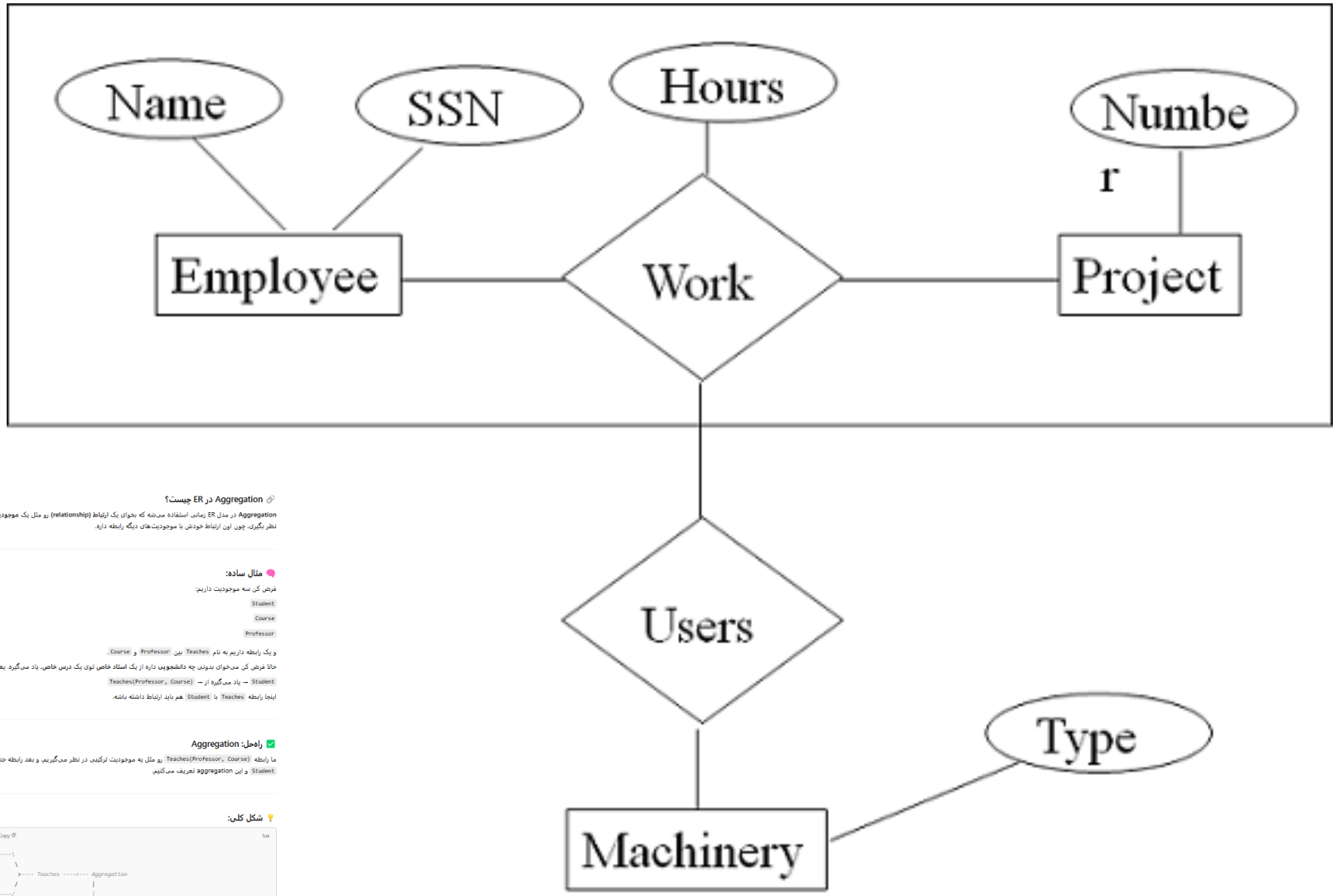
Aggregation and Association

- **Aggregation**
  - Abstraction concept for building composite objects from their component objects
- **Association**
  - Associate objects from several independent classes
- Main structural distinction
  - When an association instance is deleted
    - Participating objects may continue to exist


job
employee
works-on
branch
manages
manager

E-R Diagram With Aggregation





Aggregation در ER چیست؟

Aggregation در مدل ER زمانی استفاده می‌شه که بخوای یک ارتباط (relationship) رو مثل یک موجودیت (entity) در نظر بگیری. چون این ارتباط خودش با موجودیت‌های دیگه رابطه داره.

🧠 مثال ساده:

فرض کن سه موجودیت داریم:

`Student`

`Course`

`Professor`

و یک رابطه داریم به نام `Teaches` بین `Professor` و `Course`.

حالا فرض کن می‌خوای بدونی چه دانشجویی داره از یک استاد خاص توی یک درس خاص، یاد می‌گیره. یعنی:

`Student` ← یاد می‌گیره از ← `Teaches(Professor, Course)`

اینجا رابطه `Teaches` با `Student` هم باید ارتباط داشته باشه.

✅ راه‌حل: Aggregation

ما رابطه `Teaches(Professor, Course)` رو مثل یه موجودیت ترکیبی در نظر می‌گیریم، و بعد رابطه جدید `Learns` بین `Student` و این aggregation تعریف می‌کنیم.

💡 شکل کلی:

```
Professor ------\
                 \
                  >---- Teaches ------<---- Aggregation
                 /                          |
Course ---------/                           |
                                            |
                                          Learns
                                            |
                                         Student
```

Summary

- Enhanced ER or EER model
  - Extensions to ER model that improve its representational capabilities
  - Subclass and its superclass
  - Category or union type
  - Aggregation and Association

Quiz

- Consider the BANK ER schema in following figure

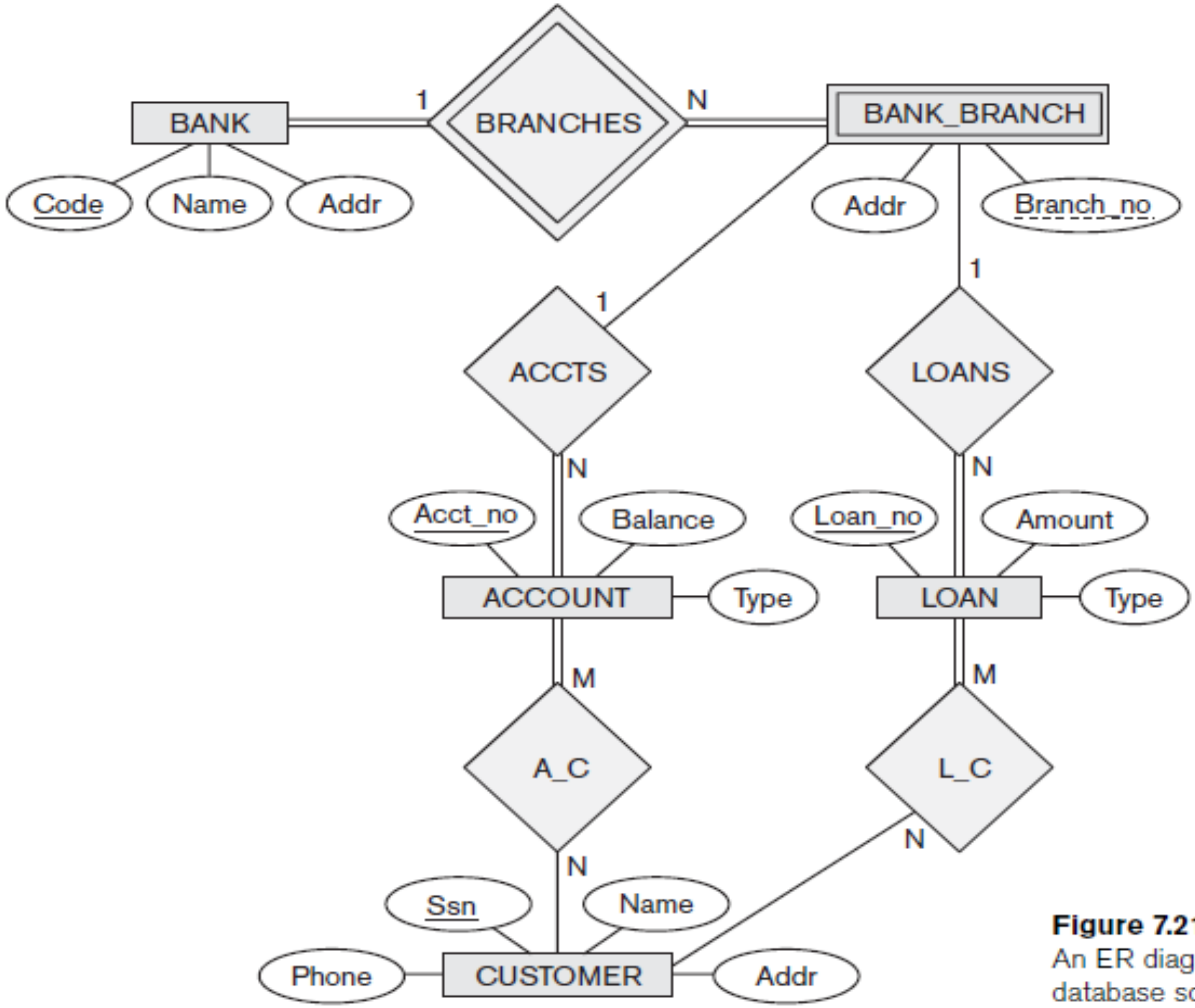


Figure 7.21
An ER diagram for a BANK database schema.

- Suppose that it is necessary to keep track of different types of ACCOUNTS (SAVINGS_ACCTS, CHECKING_ACCTS, ...) and LOANS (CAR_LOANS, HOME_LOANS, ...). Suppose that it is also desirable to keep track of each ACCOUNT's TRANSACTIONS (deposits, withdrawals, checks, ...) and each LOAN's PAYMENTS; both of these include the amount, date, and time.
- Modify the BANK schema, using ER and EER concepts of specialization and generalization. State any assumptions you make about the additional requirements.

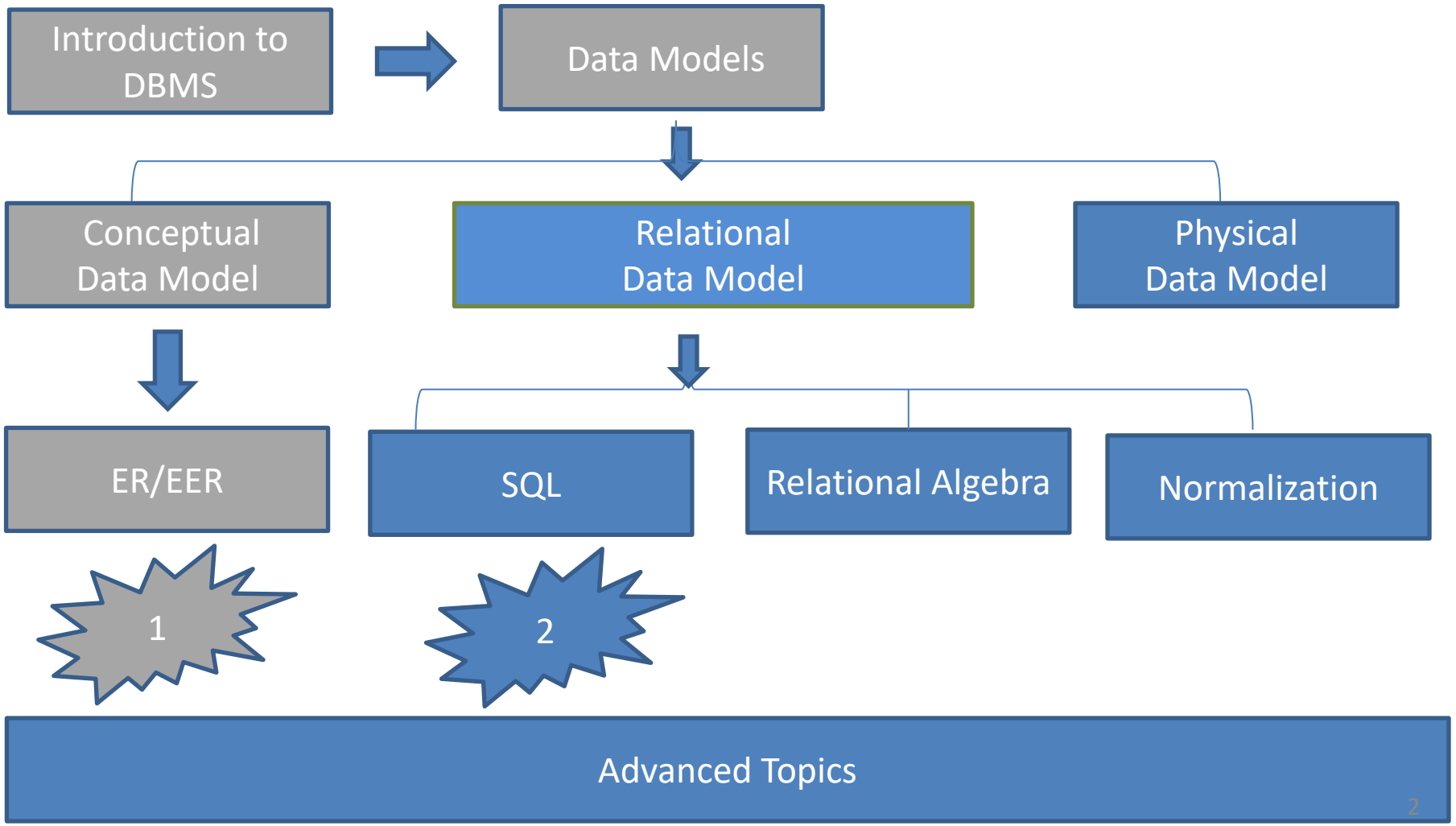
An Introduction to the Database Management Systems

By
Hossein Rahmani

Slides originally by Book(s) Resources

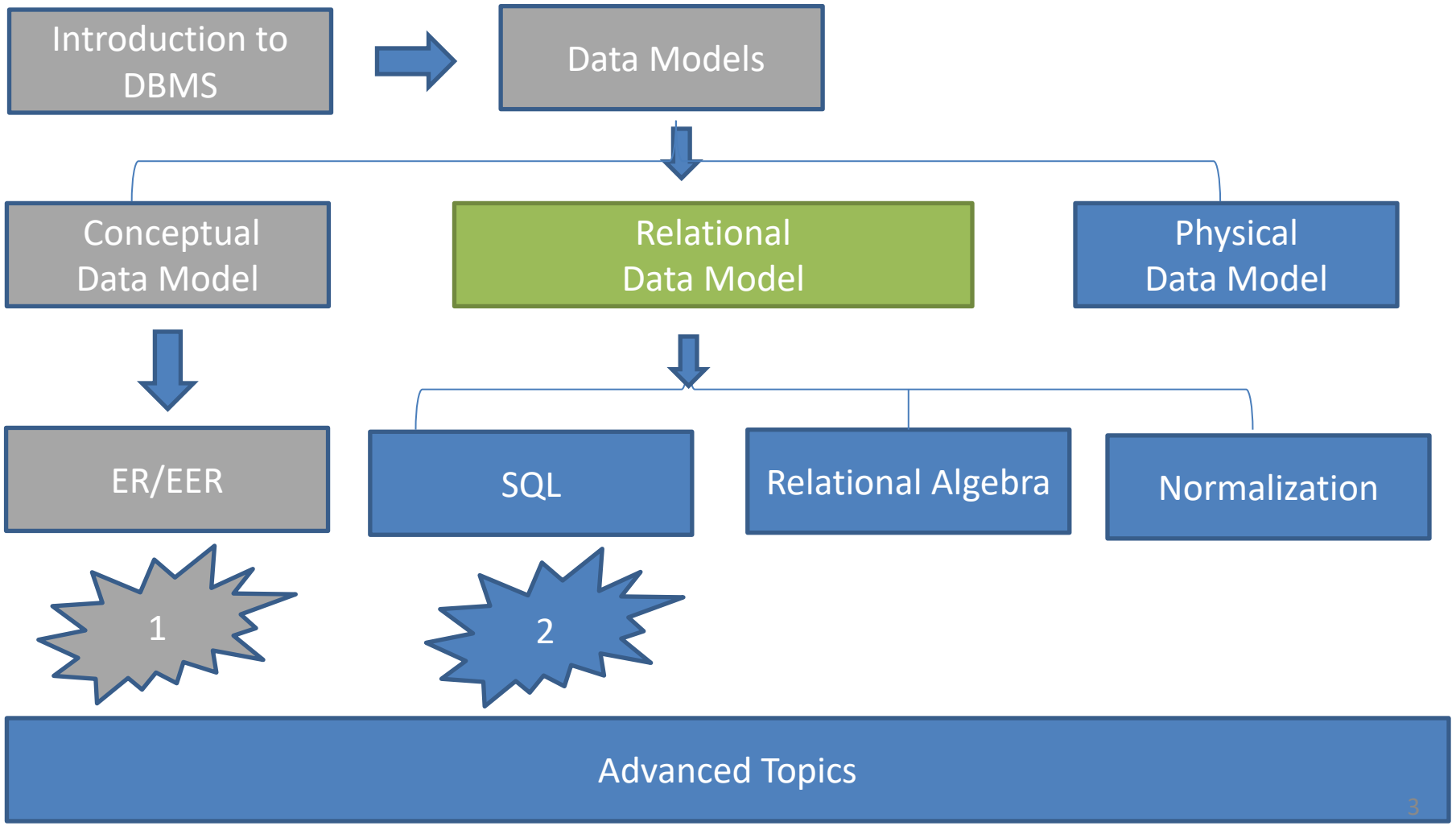
Road Map

(Might change!)



Road Map

(Might change!)



(Previously Discussed) Data models

- Conceptual (high-level)
  - for end users / analists

versus

- Physical (low-level)
  - for programmers

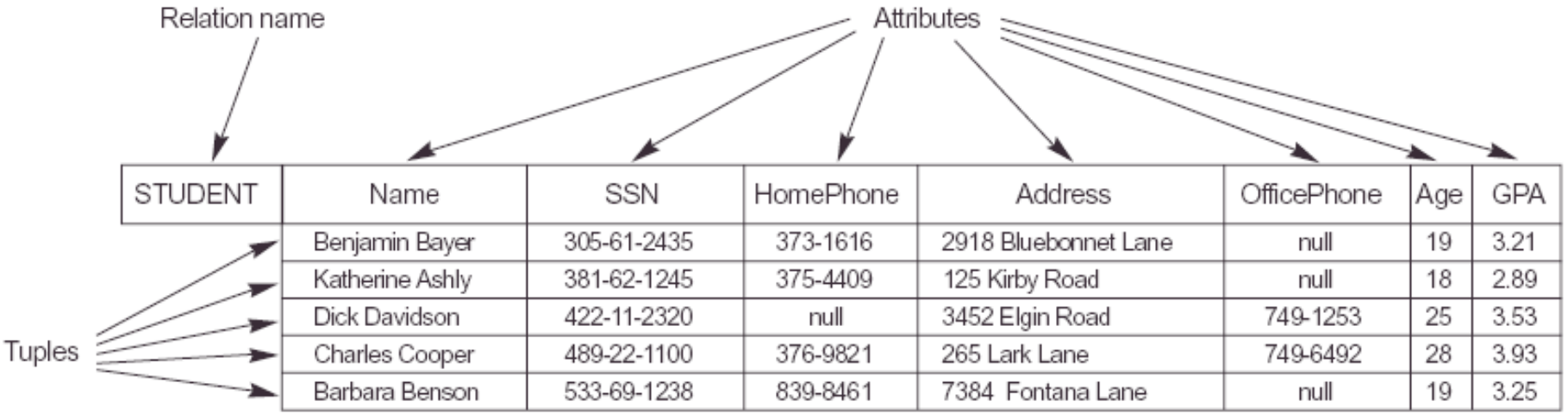
- In between: Representational (implementation) data model

Relational Model

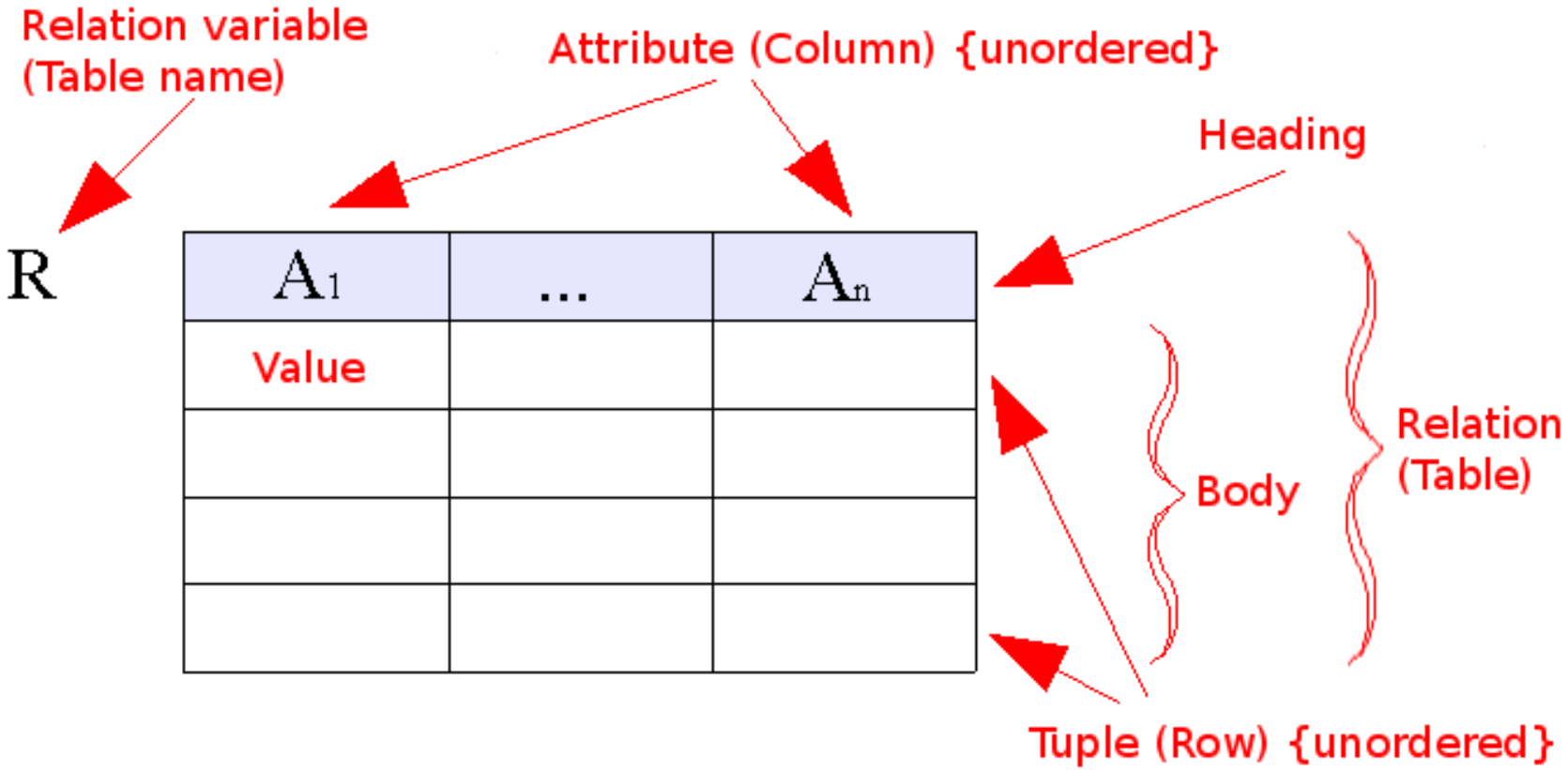
- We now move from the *conceptual* level to the *representational* (in-between) level: how do we represent databases (in a DBMS)?

- A database is represented by *relations*
- A relation is a table of which the columns are called *attributes* and the rows *tuples*
- Attributes each have a *domain*

Relations, attributes, tuples



STUDENT	Name	SSN	HomePhone	Address	OfficePhone	Age	GPA
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	null	19	3.21
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	null	18	2.89
	Dick Davidson	422-11-2320	null	3452 Elgin Road	749-1253	25	3.53
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	null	19	3.25


Relation variable
(Table name)
Attribute (Column) {unordered}
Heading
R
A1
...
An
Value
Body
Relation
(Table)
Tuple (Row) {unordered}

Domain

- A domain is a set of atomic values (so not composite and/or multi-valued):
  - Phone numbers in the Netherlands
  - Ages
  - Student-ids
- A domain is determined by:
  - Name
  - Data type
  - Format

Schema and diagram

- A *schema* in database theory is a formal (logical or mathematical) description of the *form* (e.g. a database or a relation)
- A *diagram* is the graphical representation of a schema

Relation schema

- A relation schema gives the form of a relation (intention)
- A relation schema $R(A_1,A_2,A_3,\ldots,A_n)$ of a relation of degree n consists of the name R of the relation and a list of n attributes $A_1,A_2,A_3,\ldots,A_n$
- An attribute A_i is the name of the role of a domain D_i in the relation
- D_i is the domain of A_i and is indicated by $dom(A_i)$

Relation schema

- Example:
  - Student(name, ssn, homephone, address, officephone, age, gpa)
  - Domains:
    - Dom(name) = names, e.g. string[40]
    - Dom(ssn) = social-security numbers, e.g. 10-digit integer
    - Dom(homephone) = local-phone-numbers, e.g. …
    - …

Relation (State)

- A relation r or r(R) (relation state or relation instance) of a relation schema $R(A_1,A_2,A_3,\ldots,A_n)$ is a *set of* n-*tuples* $r = \{t_1,t_2,\ldots,t_m\}$.
- Each n-tuple t in r (R) is an ordered *list* of values $t = <v_1,v_2,\ldots,v_n>$ in which each value v_i either is member of $dom(A_i)$ or the special value *null*

STUDENT	Name	SSN	HomePhone	Address	OfficePhone	Age	GPA

STUDENT	Name	SSN	HomePhone	Address	OfficePhone	Age	GPA
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	null	19	3.21
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	null	18	2.89
	Dick Davidson	422-11-2320	null	3452 Elgin Road	749-1253	25	3.53
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	null	19	3.25

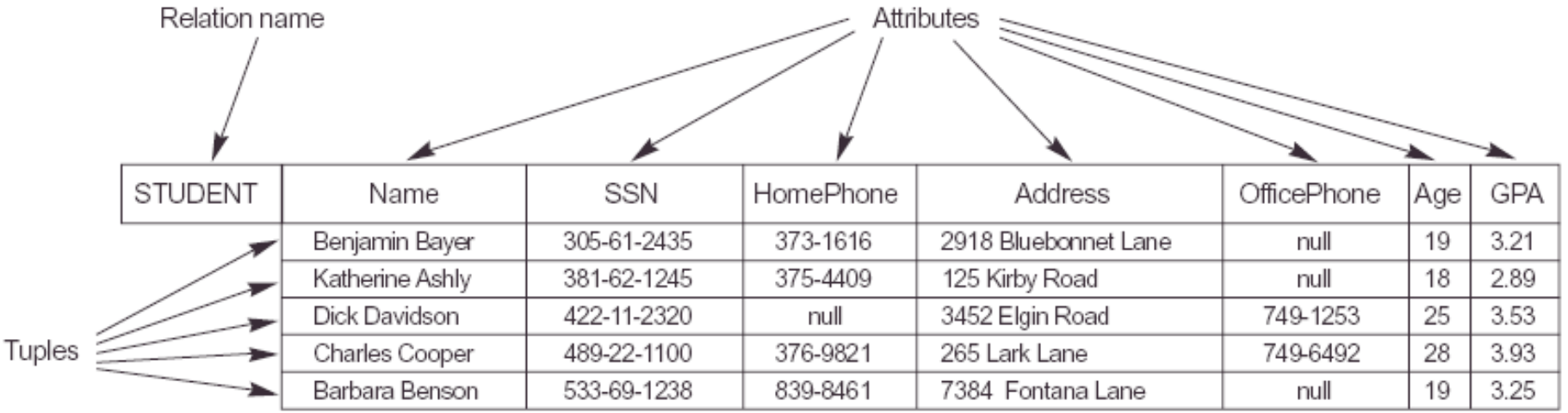
Relations

- A relation r(R) is a mathematical relation of degree *n* on the domains dom(A_1) .. dom(A_n), or:
- r (R) $\subseteq$ dom(A_1) $\times$ dom(A_2) .. $\times$ dom(A_n)
- The number of possible tuples in a relation r(R) is: |dom(A_1)|.|dom(A_2)|. $\cdots$.|dom(A_n)|

Properties of relations

- A relation is a **set** of tuples, so:
  - No ordering among the tuples
  - No two tuples can exist with the same values for all attributes
- Tuples are an ordered list: so, the ordering of the attributes is important!
- Attribute values are atomic

Relations, attributes, tuples



STUDENT	Name	SSN	HomePhone	Address	OfficePhone	Age	GPA
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	null	19	3.21
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	null	18	2.89
	Dick Davidson	422-11-2320	null	3452 Elgin Road	749-1253	25	3.53
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	null	19	3.25

Interpretations of relations

- Some relations contain facts on entities in the mini world, others on relationships between entities
- A relation schema can also be seen as a predicate with tuples as truth-values that satisfy the predicate

ER Model

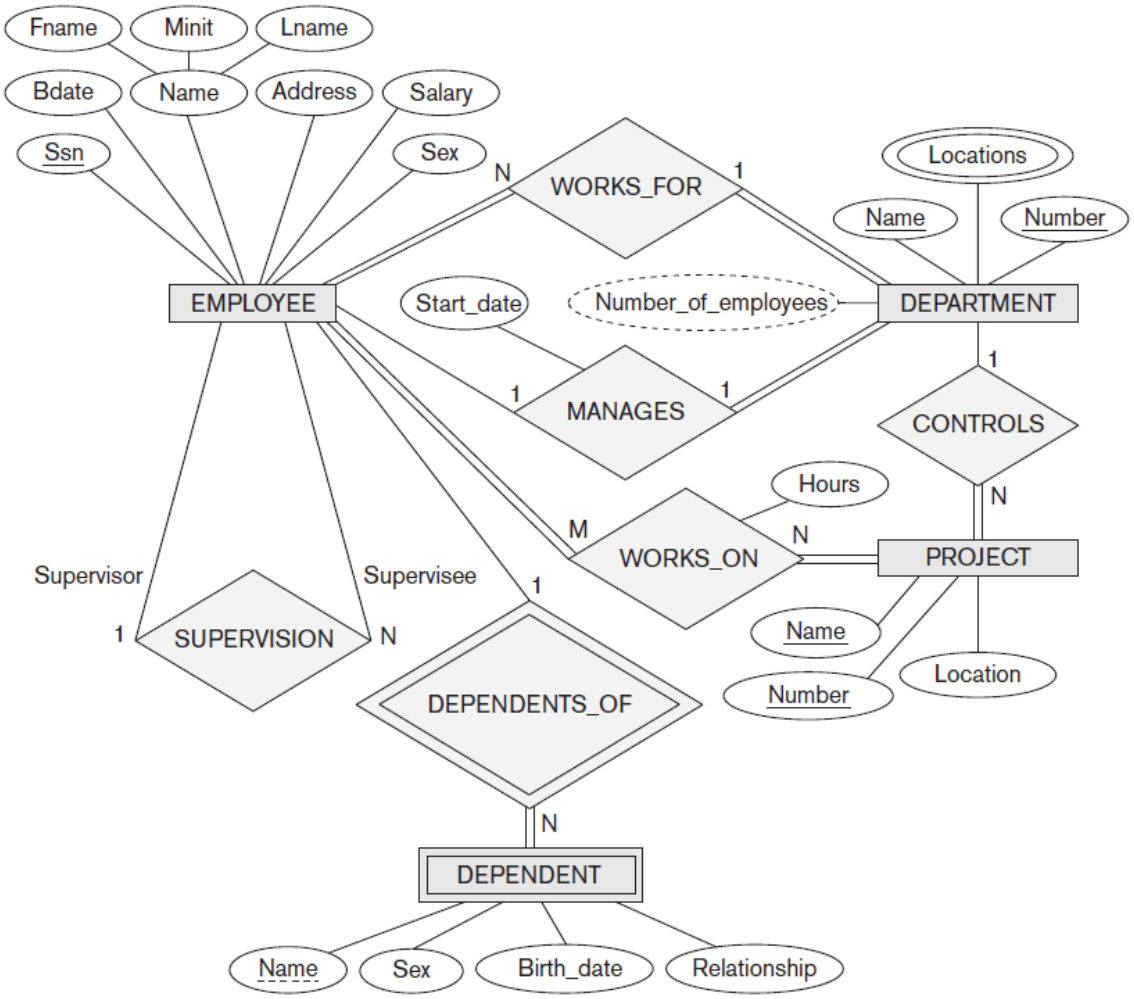


Figure 7.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter and is summarized in Figure 7.14.

Relational Model

EMPLOYEE

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO

DEPARTMENT

DNAME	DNUMBER	MGRSSN	MGRSTARTDATE

DEPT_LOCATIONS

DNUMBER	DLOCATION

PROJECT

PNAME	PNUMBER	PLOCATION	DNUM

WORKS_ON

ESSN	PNO	HOURS

Relational database schemas

- A relational database **schema** S is a set of relation schemas $S = \{R_1, R_2, \ldots R_k\}$ and a set of integrity constraints IC
- A relational database **state** DB is a set of relation states $\{r_1(R_1), r_2(R_2), \ldots, r_k(R_k)\}$ such that each r_i fulfills the integrity constraints in IC

Example: Relational Database Schema

EMPLOYEE

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO

DEPARTMENT

DNAME	DNUMBER	MGRSSN	MGRSTARTDATE

DEPT_LOCATIONS

DNUMBER	DLOCATION

PROJECT

PNAME	PNUMBER	PLOCATION	DNUM

WORKS_ON

ESSN	PNO	HOURS

Example: Relational Database State

EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John		Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin		Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia		Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer		Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh		Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce		English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad		Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James		Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS	DNUMBER	DLOCATION
		Houston
		Stafford
		Bellaire
		Sugarland

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1981-06-19

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1986-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

Constraints

- Three types of constraints
  - 1) inherent (model-based) constraints or implicit constraints
  - 2) constraints in the database model (specified in the DDL): schema-based or explicit constraints
  - 3) other constraints: application-based or semantic constraints (business rules)

Examples of constraints (1st category)

- Attribute names within a relation schema must be different
- All tuples within a relation state must be different
- …
- They are inherent in the data model

Integrity constraints (2nd category)

- Domain constraints
  - Given by dom(A_i)
- Key constraints
- Null constraints
  - (which attribute can/cannot be null)
- Entity-integrity constraints
- Referential integrity constraints

Domain Constraints

- Typically include:
  - Numeric data types for integers and real numbers
  - Characters
  - Booleans
  - Fixed-length strings
  - Variable-length strings
  - Date, time, timestamp
  - Money
  - Other special data types

SID	Name	Class (semester)	Age
8001	Ankit	1st	19
8002	Srishti	1st	18
8003	Somvir	4th	22
8004	Sourabh	6th	A

Not Allowed. Because Age is an Integer Attribute.

Integrity constraints (2nd category)

- Domain constraints
  - Given by dom(A_i)
- Key constraints
- Null constraints
  - (which attribute can/cannot be null)
- Entity-integrity constraints
- Referential integrity constraints

Key constraints

- **Superkey**: Subset SK of attributes of a relation schema for which holds that no two different tuples can have equal values:

  $t_1[SK] = t_2[SK]$ iff $t_1 = t_2$

- A **key** K is a *minimal* superkey: you cannot remove an attribute from K without loosing the superkey property

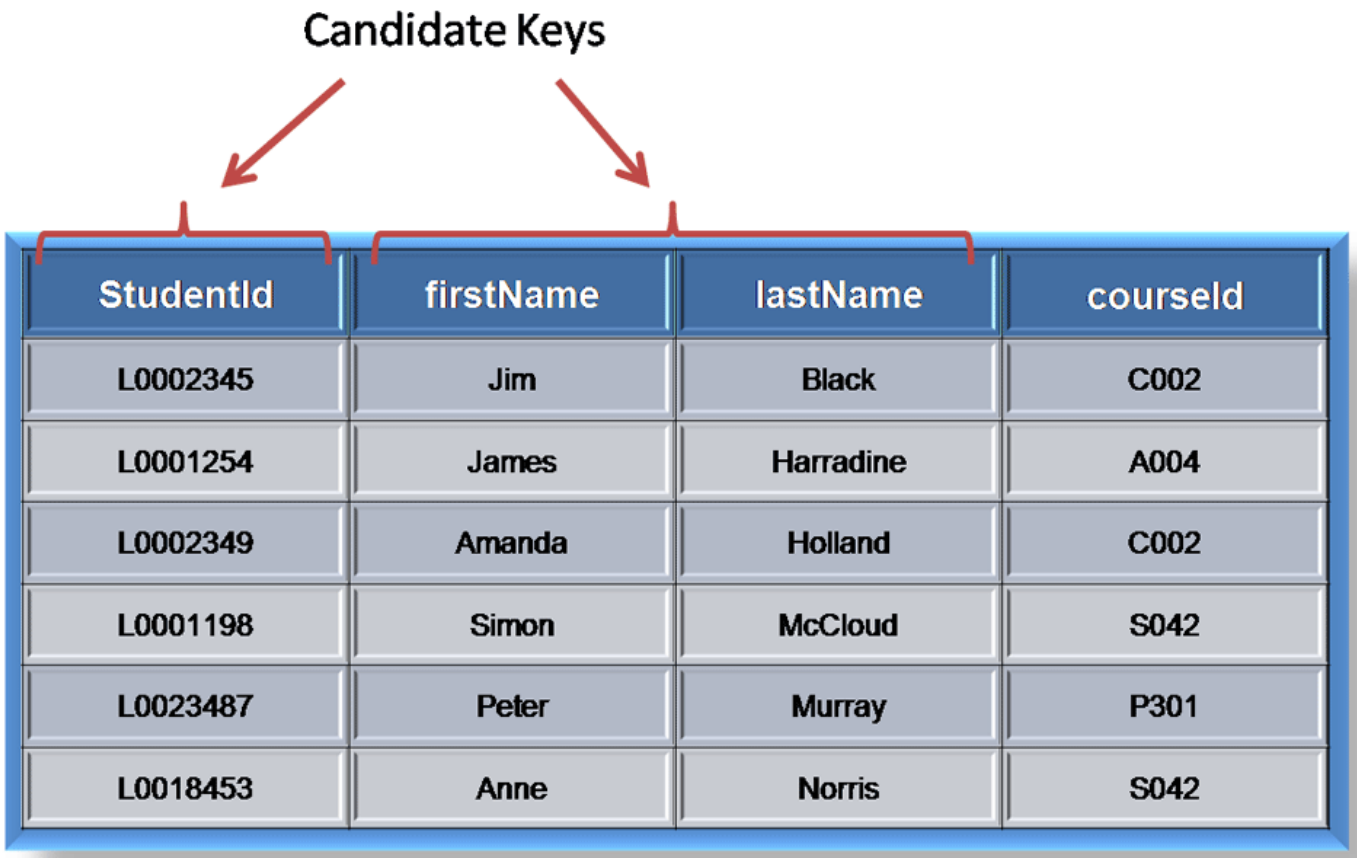
Key constraints

- A relation schema can have more than one key. Each key is a **candidate key**
- One of the candidate keys is denoted as **primary key**
- The attributes in the primary key identify the tuples
- In the relation schema we underline the attributes of the primary key

CAR

License_number	Engine_serial_number	Make	Model	Year
Texas ABC-739	A69352	Ford	Mustang	02
Florida TVP-347	B43696	Oldsmobile	Cutlass	05
New York MPO-22	X83554	Oldsmobile	Delta	01
California 432-TFY	C43742	Mercedes	190-D	99
California RSK-629	Y82935	Toyota	Camry	04
Texas RSK-629	U028365	Jaguar	XJS	04

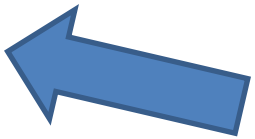
Figure 3.4
The CAR relation, with two candidate keys: License_number and Engine_serial_number.


Candidate Keys
StudentId
firstName
lastName
courseId
L0002345
Jim
Black
C002
L0001254
James
Harradine
A004
L0002349
Amanda
Holland
C002
L0001198
Simon
McCloud
S042
L0023487
Peter
Murray
P301
L0018453
Anne
Norris
S042

Integrity constraints (2nd category)

- Domain constraints
  - Given by dom(A_i)
- Key constraints
- Null constraints
  - (which attribute can/cannot be null)
- Entity-integrity constraints
- Referential integrity constraints

Integrity constraints (2nd category)

- Domain constraints
  - Given by dom(A_i)
- Key constraints
- Null constraints
  - (which attribute can/cannot be null)
- Entity-integrity constraints 
- Referential integrity constraints

Entity-integrity constraint

- The primary key may never be null in a tuple
- The primary key identifies the tuple
- Be aware: a tuple does not always represent an entity!

Entity Constraints

EMPLOYEE

ENO	Name	Age	DNO
1	Ankit	19	10
2	Srishti	18	11
3	Somvir	22	10
4	Sourabh	19	10

DEPARTMENT

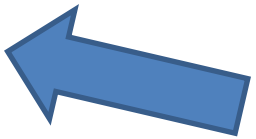
DNO	D.Location
10	Rohtak
11	Bhiwani
12	Hansi

NULL	Kavya	21	10

Insertion into EMPLOYEE table is not allowed, Because this insertion violates the Entity Integrity constraints or Integrity Rule 1 as the primary key(ENO) cannot contain a null value.

Integrity constraints (2nd category)

- Domain constraints
  - Given by dom(A_i)
- Key constraints
- Null constraints
  - (which attribute can/cannot be null)
- Entity-integrity constraints
- Referential integrity constraints

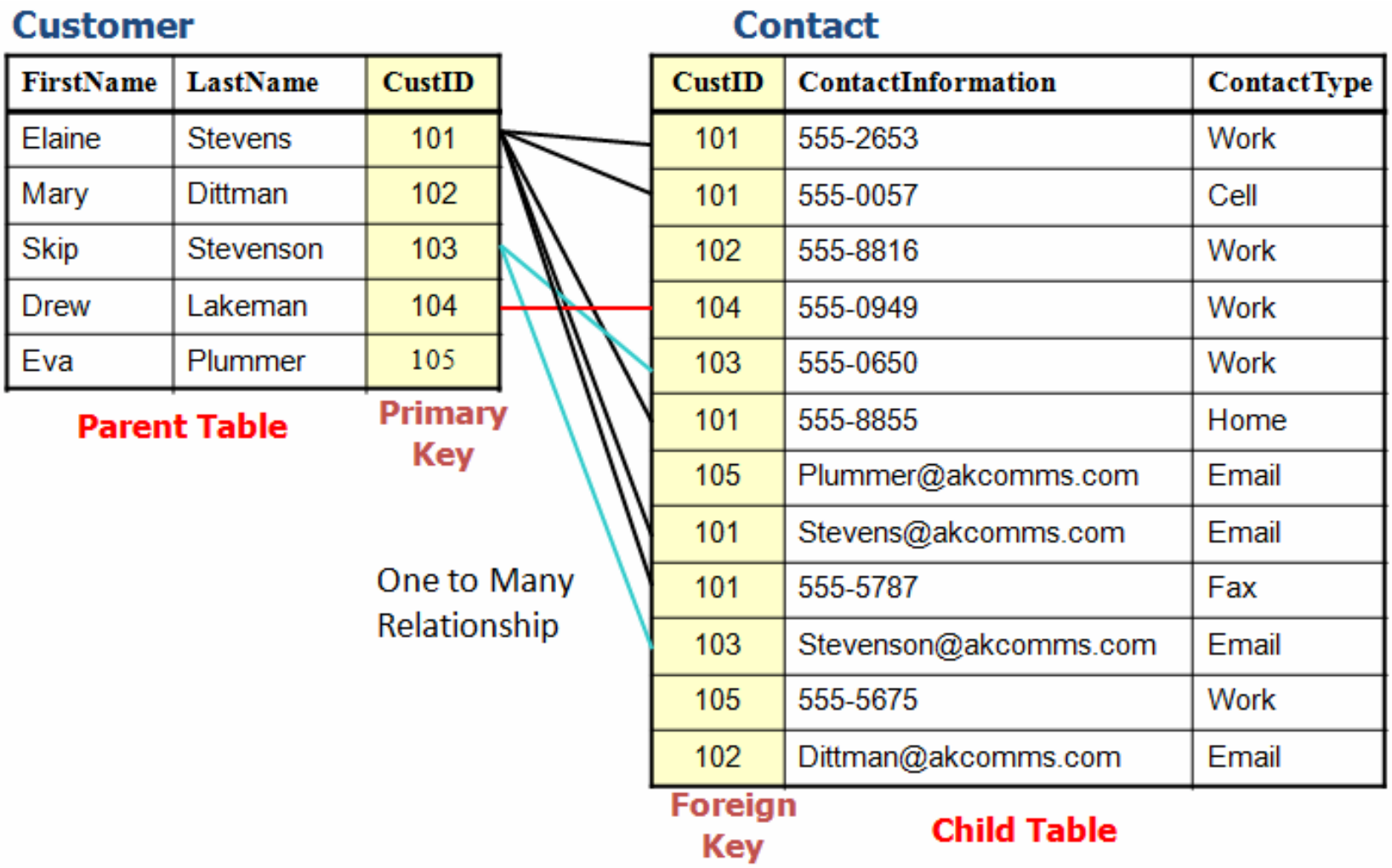


Referential integrity constraint

- Constraints that hold for multiple relation schemas at once, meant to guard the consistency among relations
- *Informal*: a tuple that refers to a tuple in another relation must always refer to an existing tuple
- *Formally*: foreign key

Foreign key

- A set attributes FK of a relation schema R_1 is a *foreign key* of R_1 *referring to* R_2 if
  - a 1-to-1 relation exists between attributes a_i in FK and attributes a_j in the primary key PK of R_2 such that $dom(a_i) = dom(a_j)$;
  - the value of FK in all $t_1 \in r(R_1)$ either is *null* or exists as value of PK in $t_2 \in r(R_2)$:
    $\forall\, t_1 \in r(R_1)$: { $t_1[FK] = null \vee$
    $\exists\, t_2 \in r(R_2)$: $t_1[FK] = t_2[PK]$ }.


Customer
Contact
FirstName
LastName
CustID
Elaine
Stevens
101
Mary
Dittman
102
Skip
Stevenson
103
Drew
Lakeman
104
Eva
Plummer
105
Parent Table
Primary Key
One to Many Relationship
CustID
ContactInformation
ContactType
101
555-2653
Work
101
555-0057
Cell
102
555-8816
Work
104
555-0949
Work
103
555-0650
Work
101
555-8855
Home
105
Plummer@akcomms.com
Email
101
Stevens@akcomms.com
Email
101
555-5787
Fax
103
Stevenson@akcomms.com
Email
105
555-5675
Work
102
Dittman@akcomms.com
Email
Foreign Key
Child Table

Parent Table

Emp

EMPNO	ENAME	SAL	COMM	DEPTNO	IDNO
101	Sami	4500		10	
102	Scott	4300	300	10	A1232
103	Smith	4400	200	20	B3423

PK

Foreign Key Refers to
Empno of EMP table.

Attendance

Child Table

EMPNO	MONTH	DAYS
101	JAN	23
101	FEB	24
102	JAN	21

Integrity constraints

- These five restrictions belong to a relational database schema
- Some of them are indicated graphically: *primary key* and *foreign key*
- The DDL of a DBMS contains special constructions for integrity constraints

Other constraints (3rd category)

- The five constraints mentioned do not include *semantic* constraints
    - the salary of an employee should not exceed that of his supervisor
- To this end, special languages exist: *constraint specification language*, or mechanisms like *triggers* or *assertions* are used

Constraint violations

- During changes of a database, the integrity constraints should not be violated
- There are three kinds of changes
  - inserting tuples to a relation
  - deleting tuples from a relation
  - updating tuples in a relation

Inserting tuples

- A new tuple t must have
  - domain values (or null) for all attributes $t[A] \in dom(A) \cup \{null\}$
  - no null for indicated attributes (when null is not allowed) and for all attributes in the primary key
  - unique values for the primary key and for all other candidate keys
  - valid foreign keys

Figure 3.6
One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Figure 3.6
One possible database state for the COMPANY relational database schema.

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

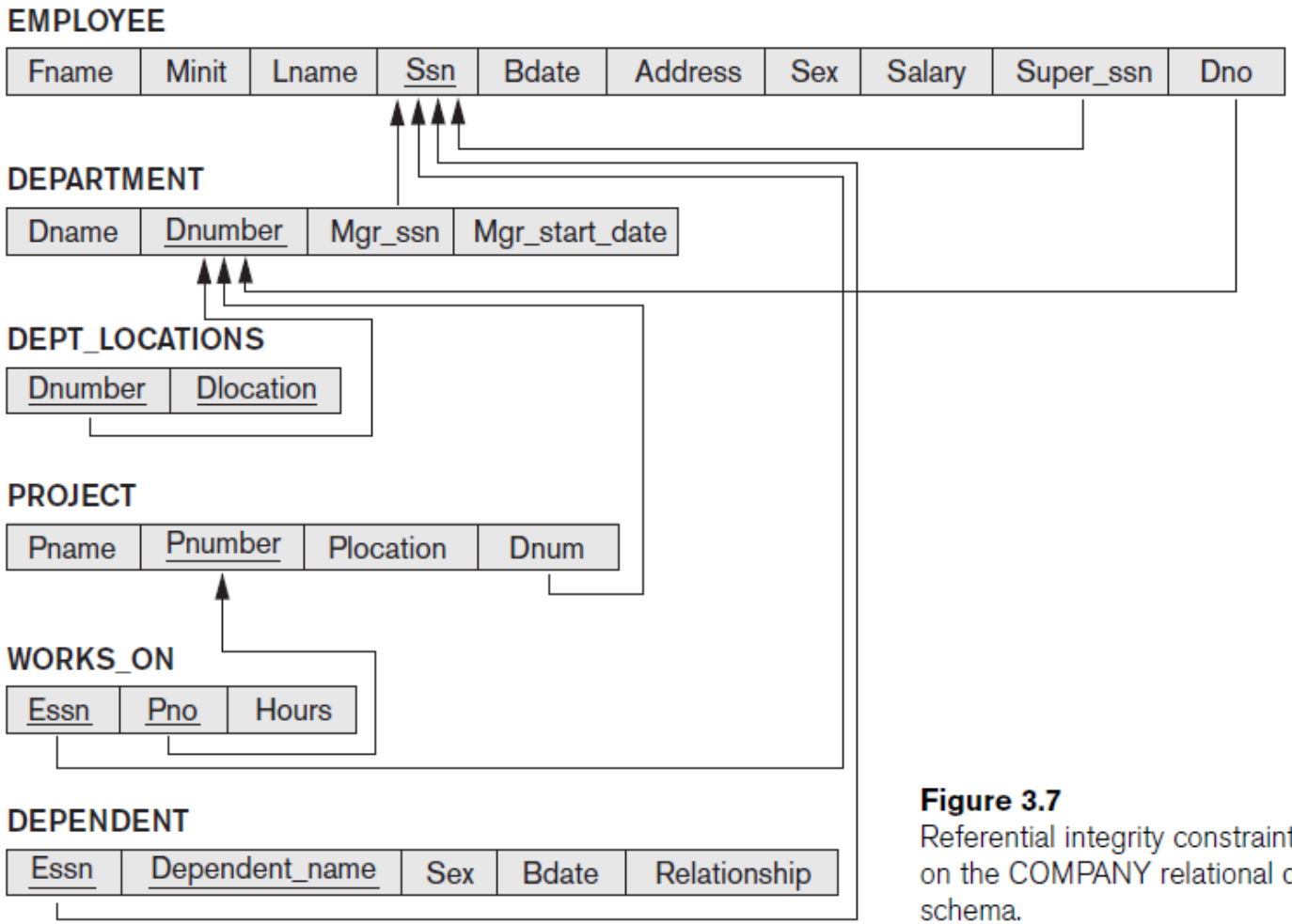


Figure 3.7
Referential integrity constraints displayed on the COMPANY relational database schema.

- *Operation*:
  Insert <'Alicia', 'J', 'Zelaya', '999887777', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, '987654321', 4> into EMPLOYEE.
  Result: This insertion violates the key constraint because another tuple with the same Ssn value already exists in the EMPLOYEE relation, and so it is rejected.
- *Operation*:
  Insert <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windswept, Katy, TX', F, 28000, '987654321', 7> into EMPLOYEE.
  Result: This insertion violates the referential integrity constraint specified on Dno in EMPLOYEE because no corresponding referenced tuple exists in DEPARTMENT with Dnumber = 7.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

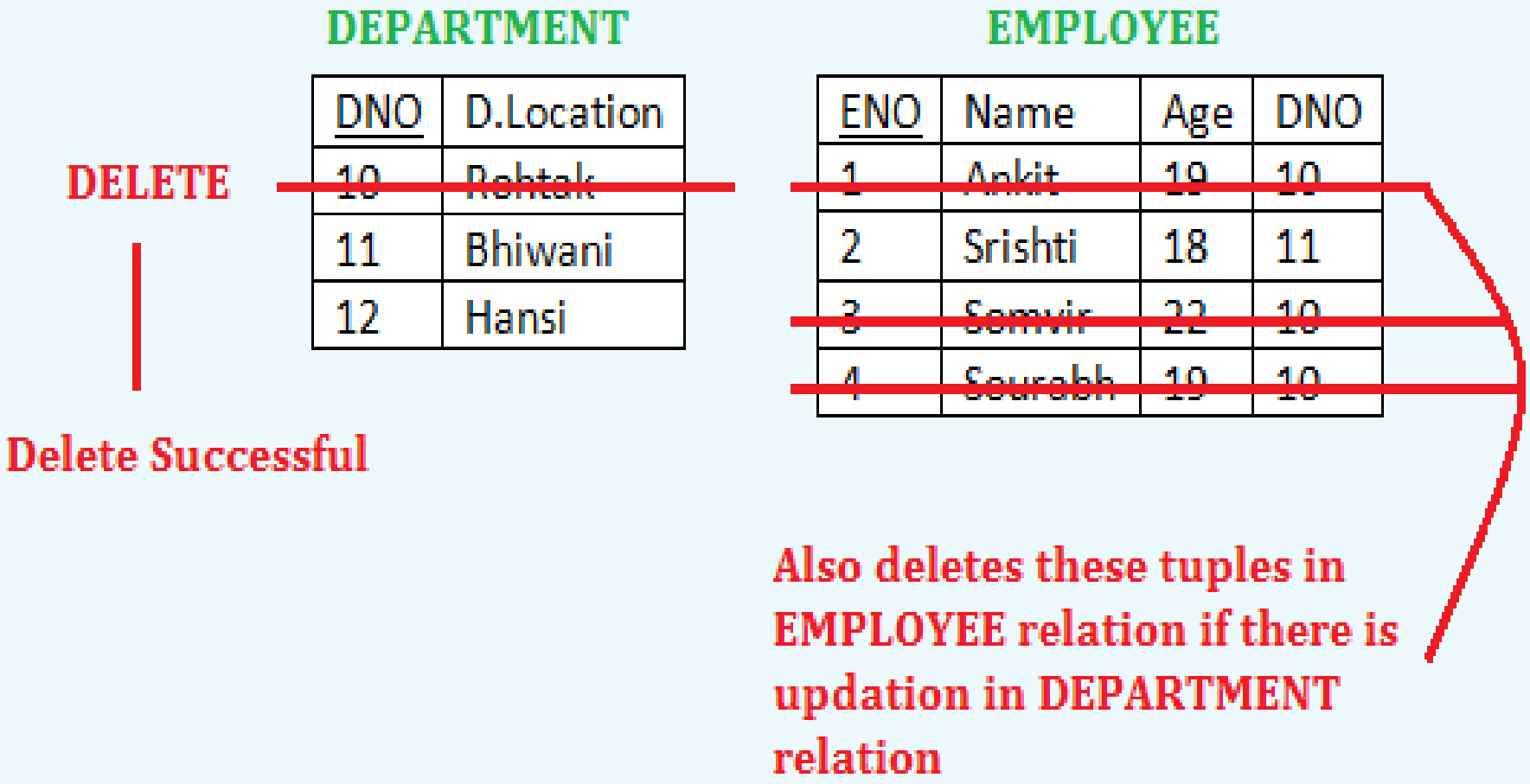
Deleting tuples

- No reference to this tuple (t_1) must exist:
  $\neg \exists\, t_2 \in r(R_2)$: $t_2[FK] = t_1[PK]$
- Three reactions when t_1 is referred to:
  - refuse deletion;
  - propagate (cascade): also delete t_2 etc.
  - change t_2, e.g. make $t_2[FK]$ = null.
- This reaction is defined with the foreign key

- *Operation*:
  Delete the EMPLOYEE tuple with Ssn = '333445555'.
  Result: This deletion will result in even worse referential integrity violations, because the tuple involved is referenced by tuples from the EMPLOYEE, DEPARTMENT, WORKS_ON, and DEPENDENT relations.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1


DEPARTMENT
EMPLOYEE
DELETE
DNO
D.Location
10
Rohtak
11
Bhiwani
12
Hansi
ENO
Name
Age
DNO
1
Ankit
19
10
2
Srishti
18
11
3
Somvir
22
10
4
Saurabh
19
10
Delete Successful
Also deletes these tuples in EMPLOYEE relation if there is updation in DEPARTMENT relation

ON DELETE SETS NULL

DEPARTMENT

DNO	D.Location
~~10~~	~~Rohtak~~
11	Bhiwani
12	Hansi

DELETE

Delete Successful

EMPLOYEE

ENO	Name	Age	DNO
~~1~~	~~Ankit~~	~~19~~	~~10~~
2	Srishti	18	11
~~3~~	~~Somvir~~	~~22~~	~~10~~
~~4~~	~~Sourabh~~	~~19~~	~~10~~

ENO	Name	Age	DNO
NULL	NULL	NULL	NULL
2	Srishti	18	11
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL

Puts NULL in referencing relation if there is deletion in referenced relation

Update tuples

- After update the tuple must have
  - no extra-domain values or forbidden null values
  - valid foreign keys
  - unique key values
- When the primary key is changed, either:
  - refuse change,
  - change referring keys to null
  - change referring keys to the new value (propagate)

- *Operation*:
  Update the Dno of the EMPLOYEE tuple with Ssn = '999887777' to 7.
  Result: Unacceptable, because it violates referential integrity.
- *Operation*:
  Update the Ssn of the EMPLOYEE tuple with Ssn = '999887777' to '987654321'.
  Result: Unacceptable, because it violates primary key constraint by repeating a value that already exists as a primary key in another tuple; it violates referential integrity constraints because there are other relations that refer to the existing value of Ssn.

The Transaction Concept

- **Transaction**
  - Executing program
  - Includes some database operations
  - Must leave the database in a valid or consistent state
- **Online transaction processing (OLTP) systems**
  - Execute transactions at rates that reach several hundred per second

Quiz 1

- Consider the following relations for a database that keeps track of business trips of salespersons in a sales office:

SALESPERSON(Ssn, Name, Start_year, Dept_no)
TRIP(Ssn, From_city, To_city, Departure_date, Return_date, Trip_id)
EXPENSE(Trip_id, Account#, Amount)

- A trip can be charged to one or more accounts. Specify the foreign keys for this schema, stating any assumptions you make.

Quiz 2

- Consider the following relations for a database that keeps track of student enrollment in courses and the books adopted for each course:

STUDENT(Ssn, Name, Major, Bdate)
COURSE(Course#, Cname, Dept)
ENROLL(Ssn, Course#, Quarter, Grade)
BOOK_ADOPTION(Course#, Quarter, Book_isbn)
TEXT(Book_isbn, Book_title, Publisher, Author)

- Specify the foreign keys for this schema, stating any assumptions you make.

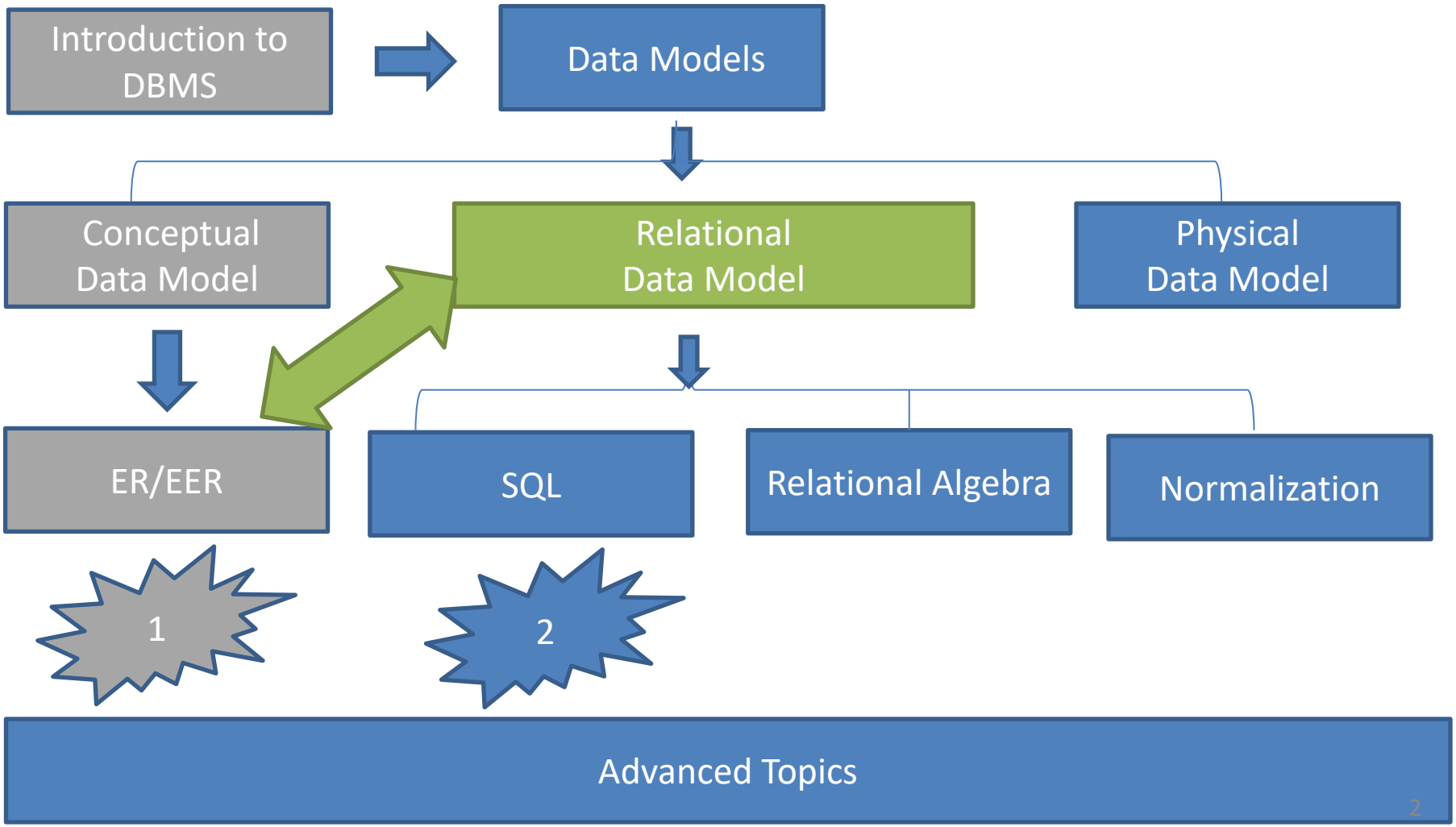
An Introduction to the Database Management Systems

By
Hossein Rahmani

Slides originally by Book(s) Resources

Road Map

(Might change!)



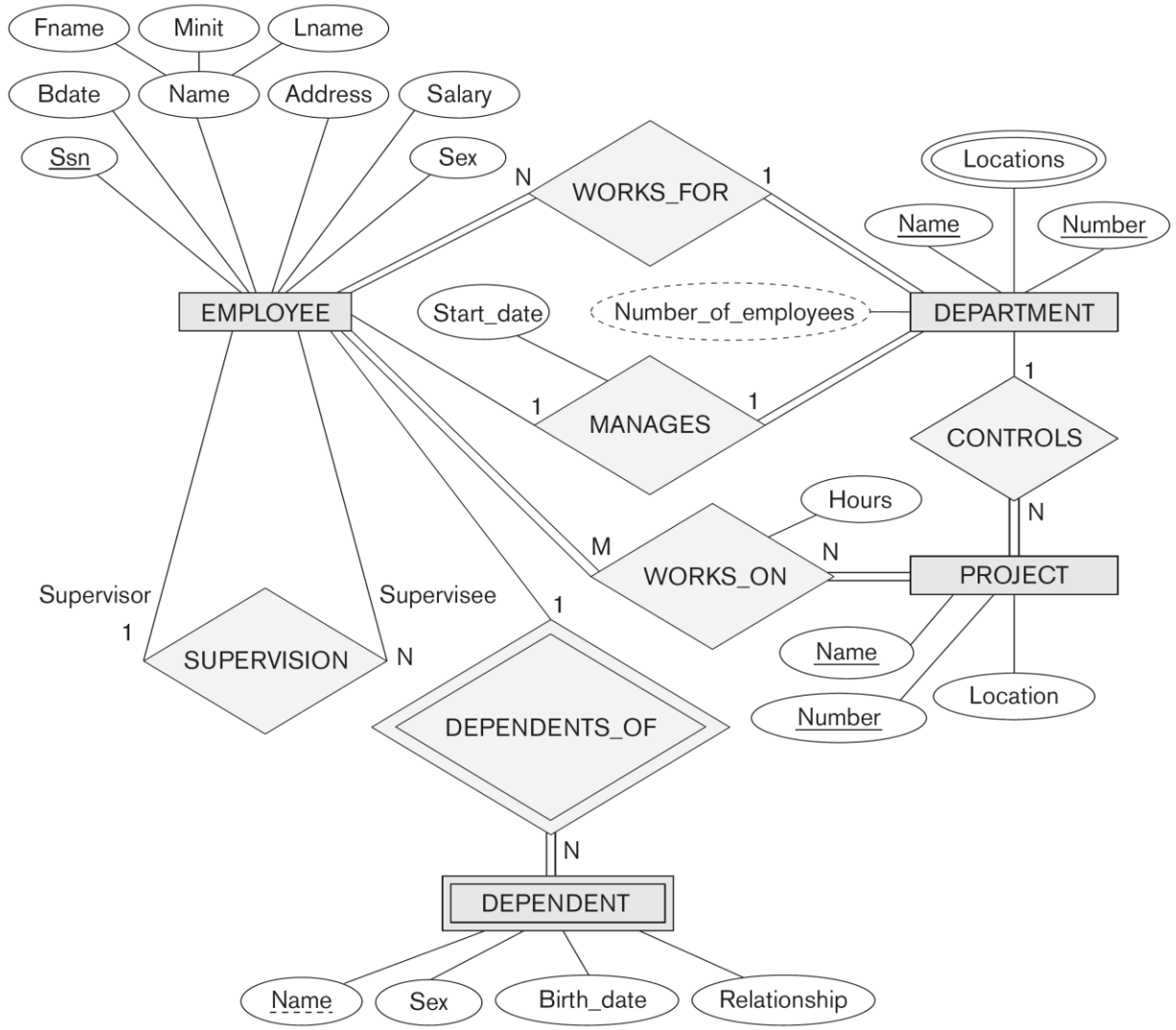
Logical design / “Data mapping”

- We have to translate all constructs from the ER model into constructs of the relational model
- It starts with a cookbook recipe
- Then creativity and insight is required

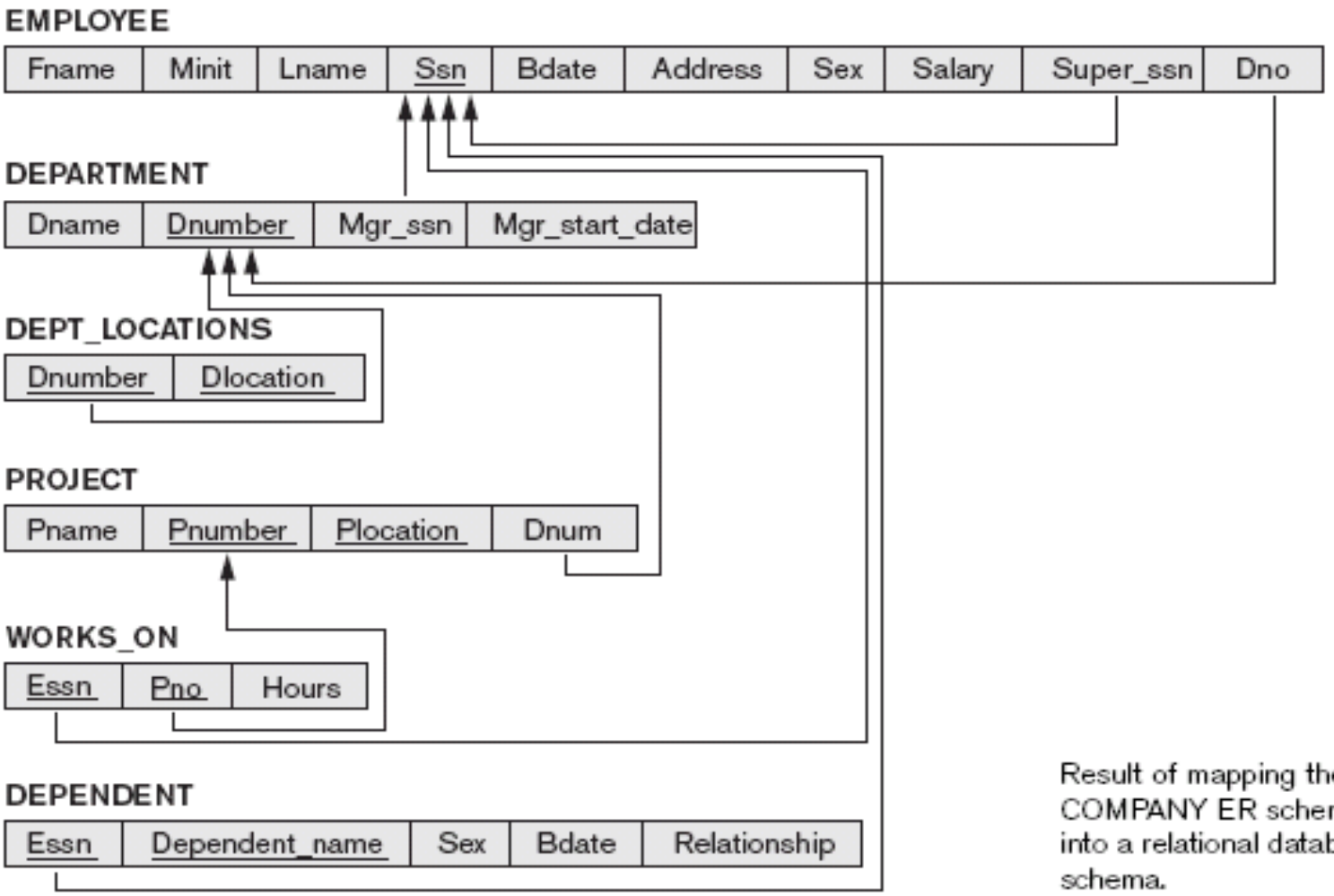
3

Start from conceptual ER diagram

Figure 9.1
The ER conceptual schema diagram for the COMPANY database.



Finish at relational database schema



Result of mapping the COMPANY ER schema into a relational database schema.

Step 1: Strong entity types

- Create a relation R for each strong entity type E
- Include all single-valued attributes and all single-valued parts of composite attributes
- Appoint one of the keys in E as primary key in R

Step 1: the example

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary

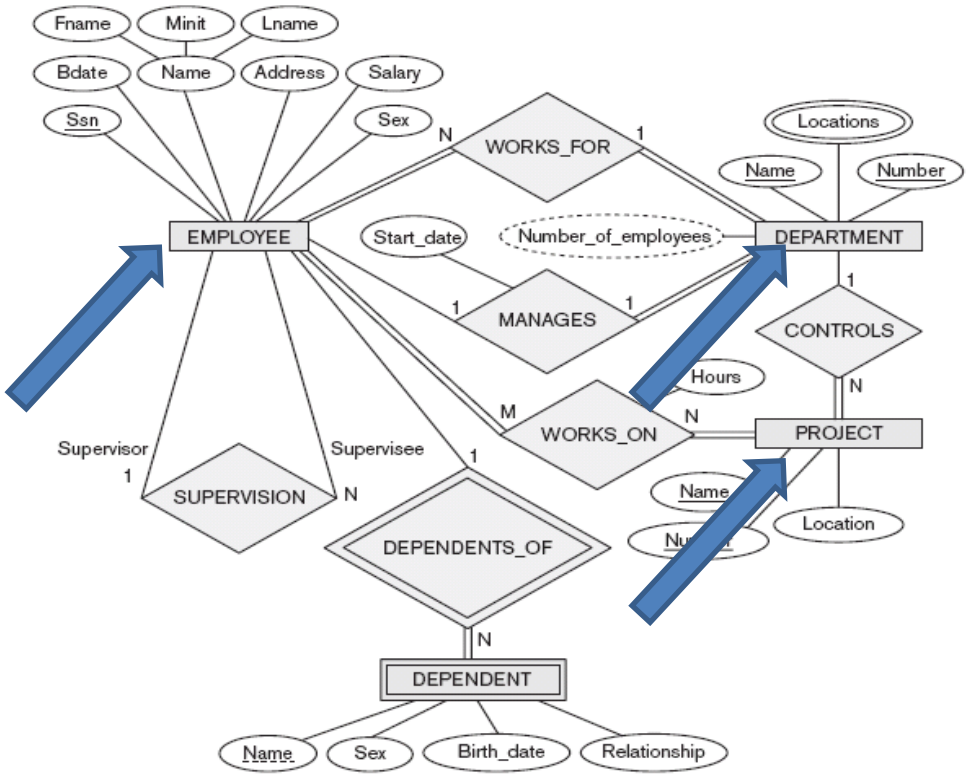
DEPARTMENT

Dname	Dnumber

PROJECT

Pname	Pnumber	Plocation

The ER conceptual schema diagram for the COMPANY database.



Step 2: Weak Entity types

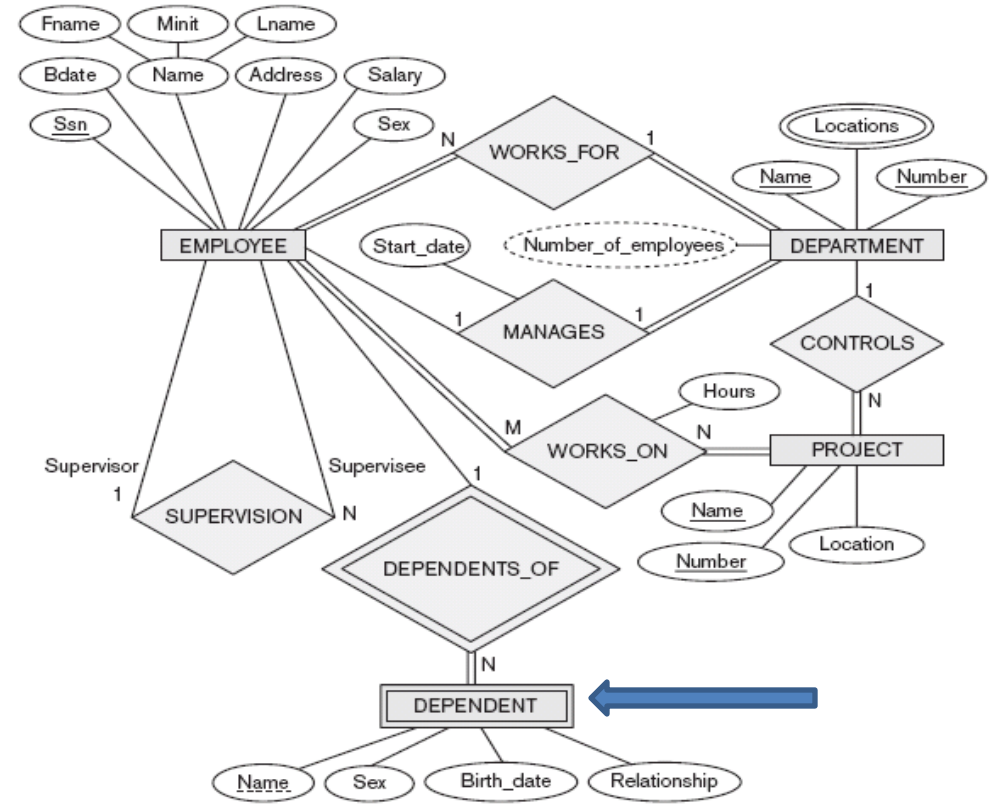
- Make a relation R for each weak entity type W, including the identifying relation
- Add all single-valued attributes and single-valued parts of composite attributes
- Add a foreign key FK to R pointing to the primary key of the identifying entity
- The primary key of R is the foreign key FK together with a partial key of W

Step 2: the example

DEPENDENT

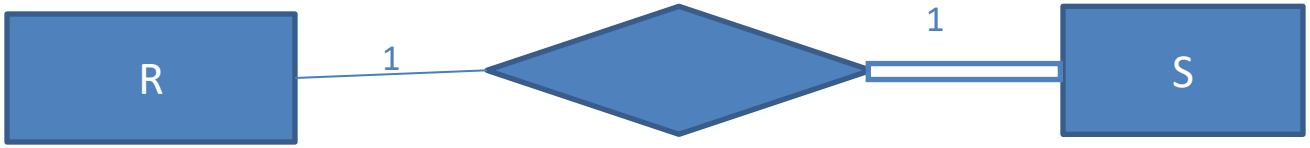
Essn	Dependent_name	Sex	Bdate	Relationship

The ER conceptual schema diagram for the COMPANY database.

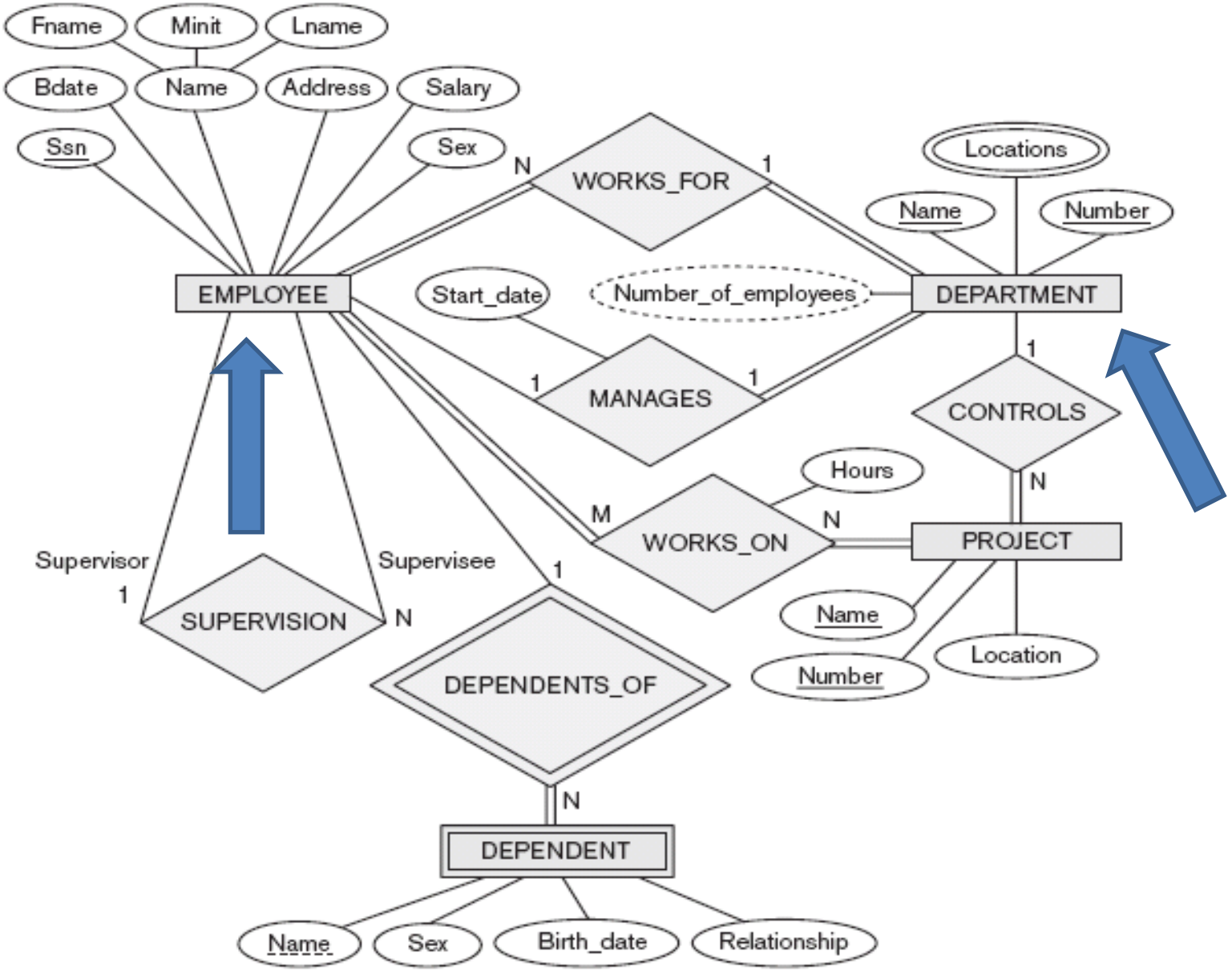


Step 3: 1:1 relations

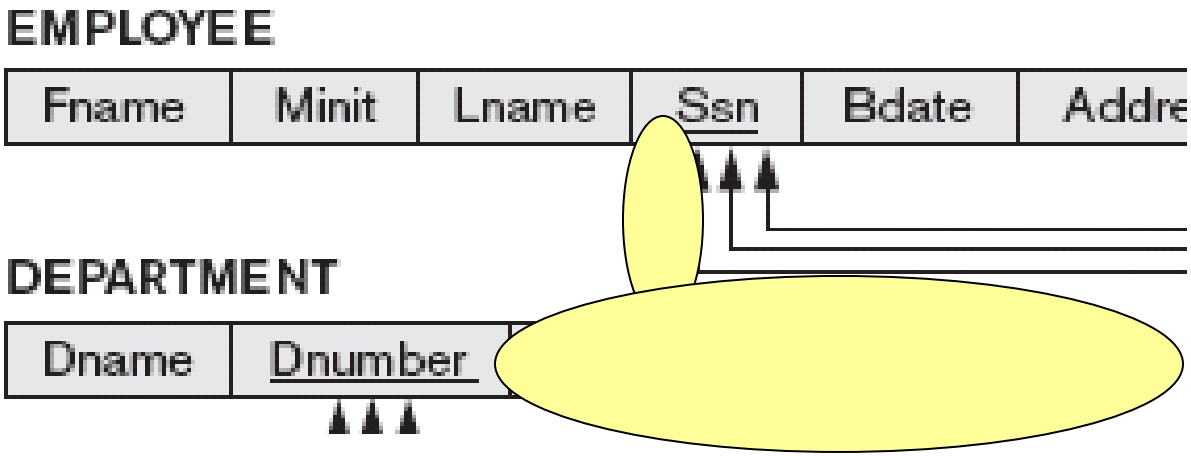
- Identify relations S and R that represent the connected entity types
- Add to one of them (say S, preferably totally participating):
  - A foreign key to R
  - all (single-valued) attributes of the 1-1 relation



The ER conceptual schema diagram for the COMPANY database.

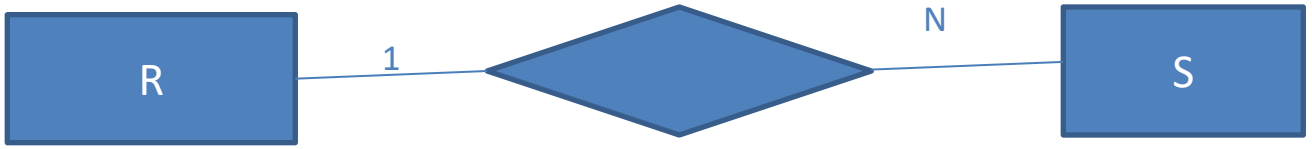


Step 3: the example

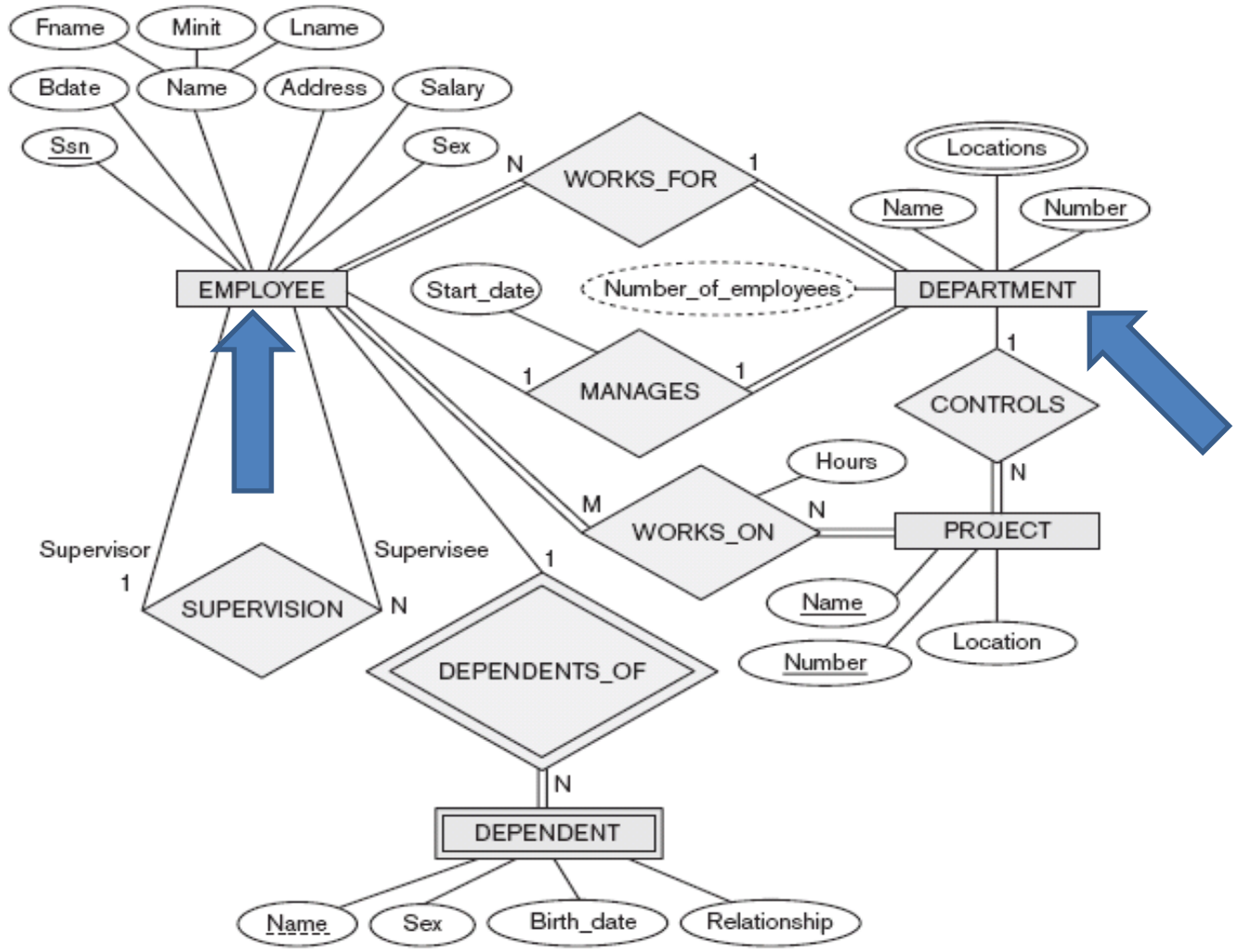


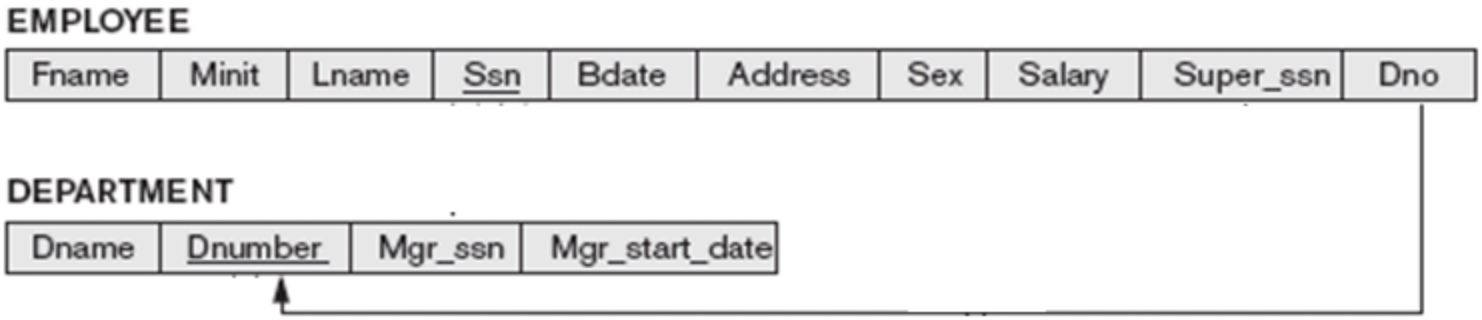
Step 4: 1:N Relations

- Identify S and R that represent the connected entity types, having S at the N-side
- Add a foreign key in S, pointing to R
- Add all (single-valued) attributes of the relation to S

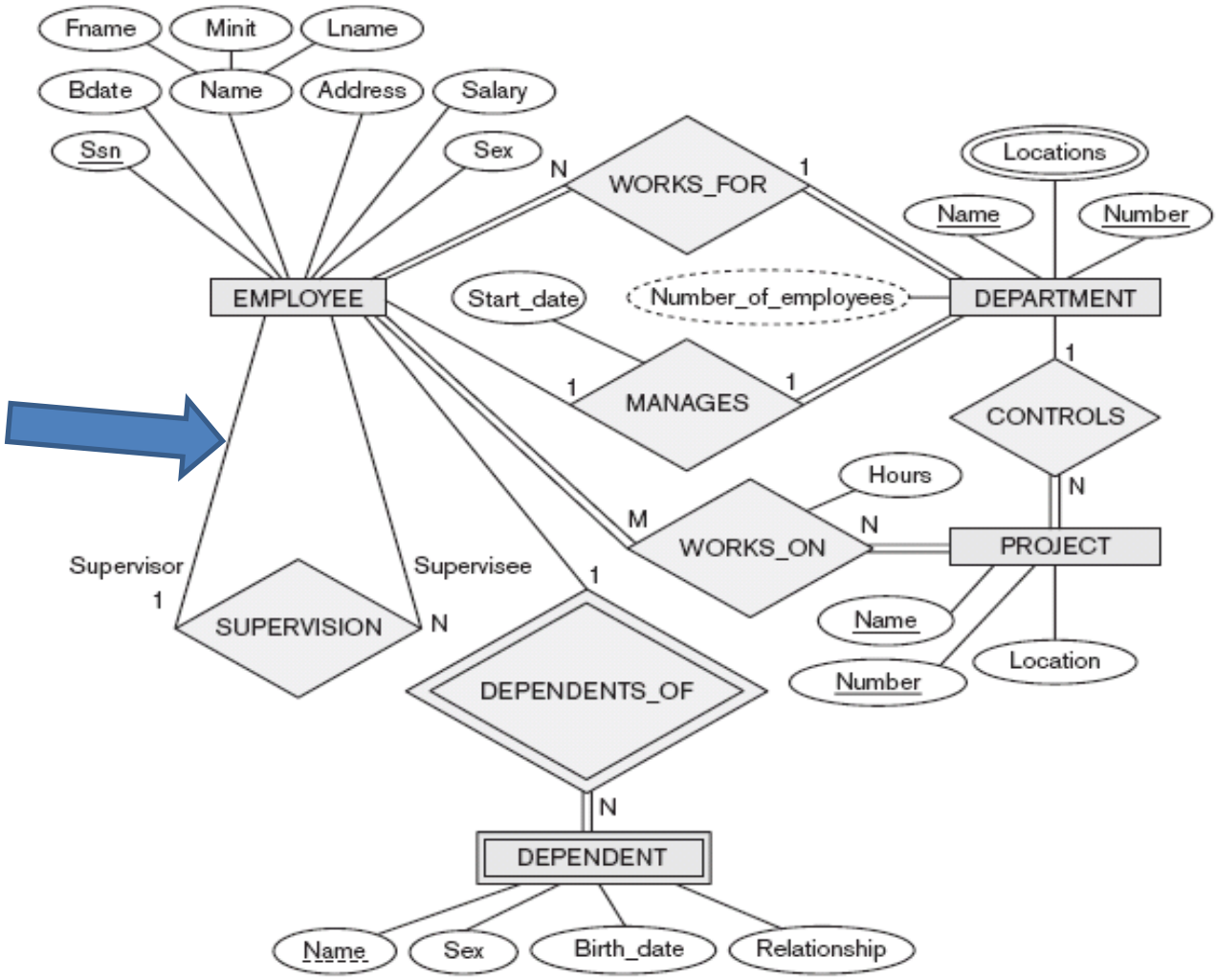


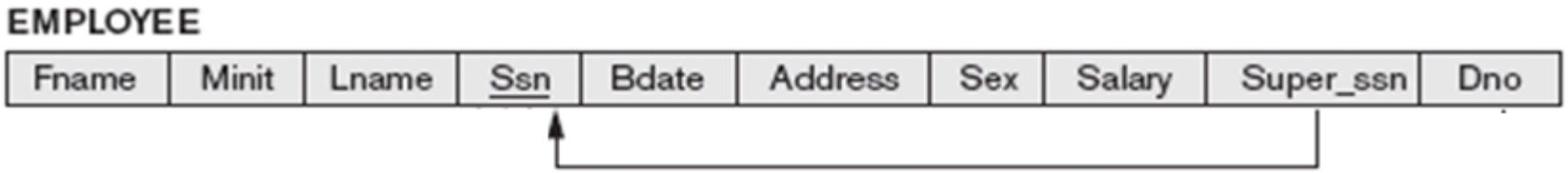
The ER conceptual schema diagram for the COMPANY database.



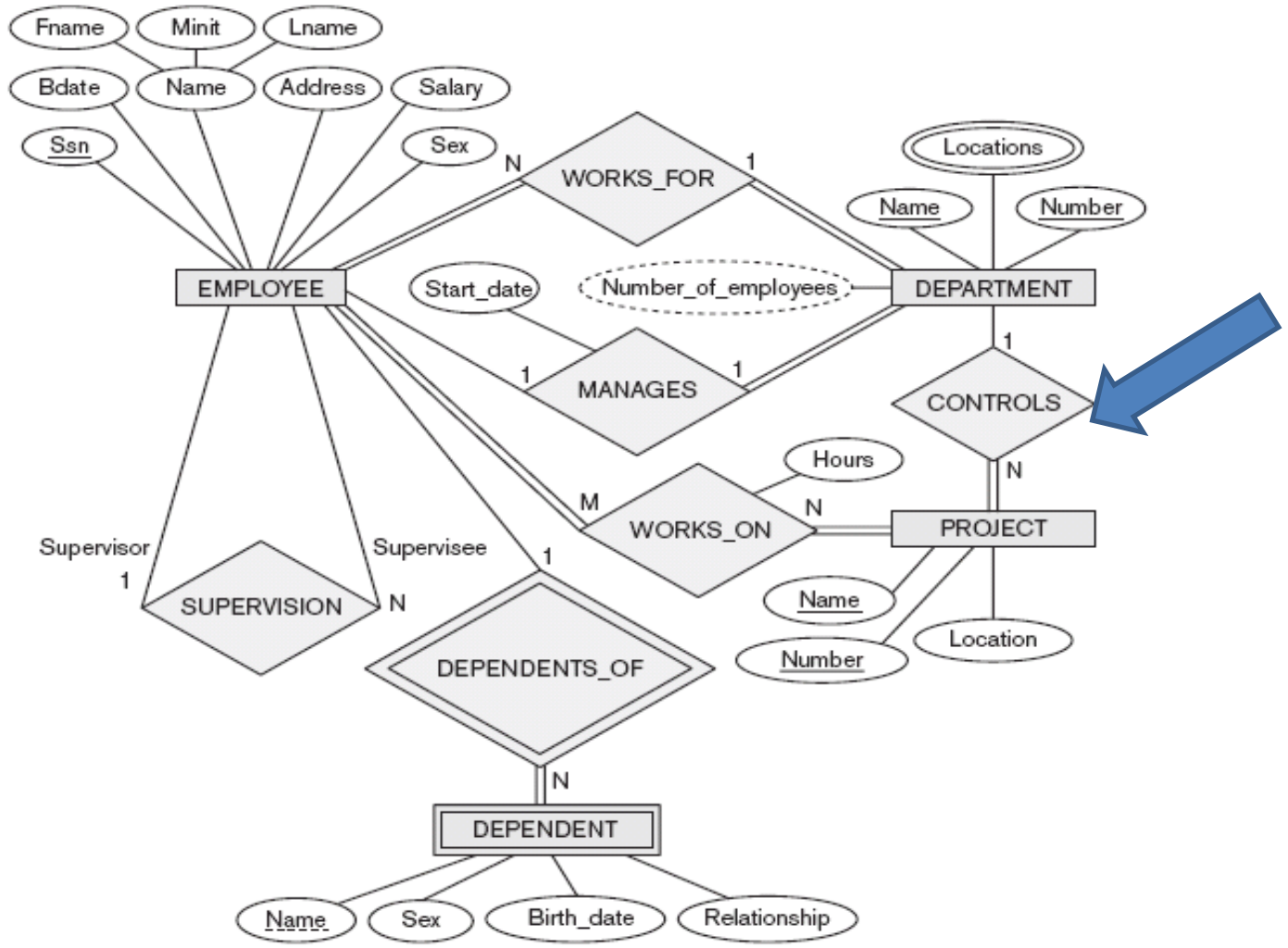

EMPLOYEE
Fname
Minit
Lname
Ssn
Bdate
Address
Sex
Salary
Super_ssn
Dno
DEPARTMENT
Dname
Dnumber
Mgr_ssn
Mgr_start_date

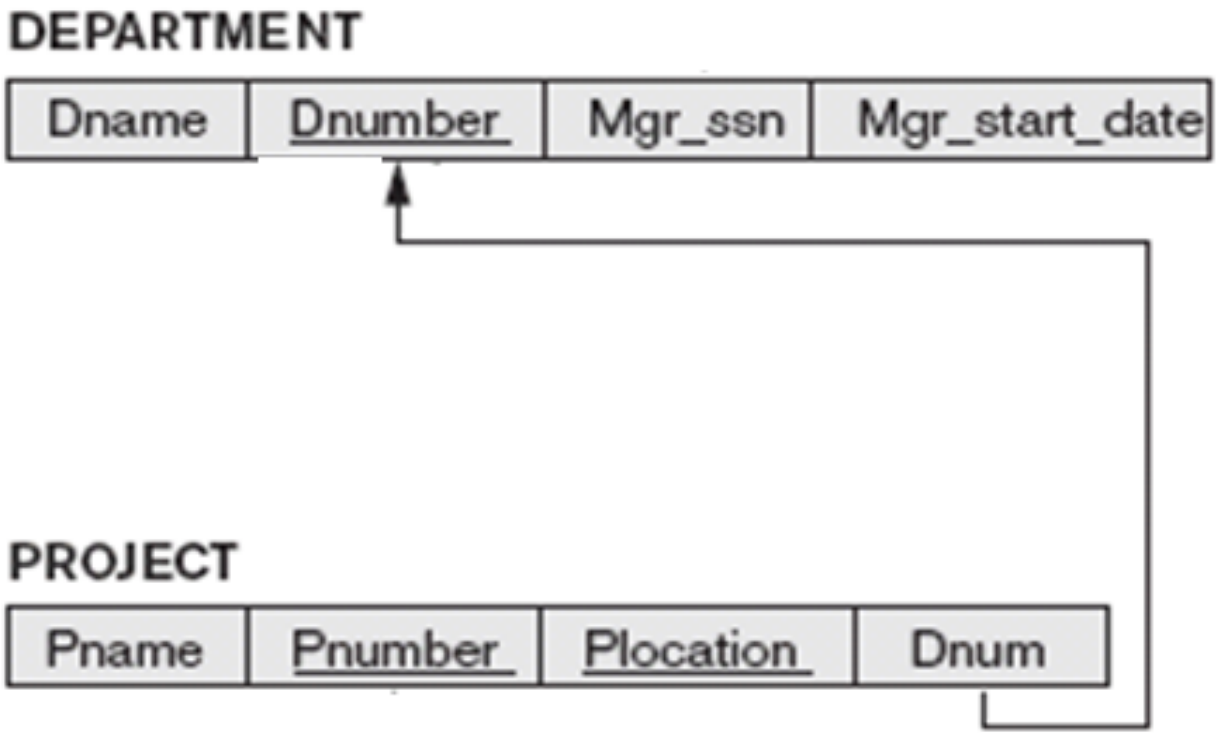
The ER conceptual schema diagram for the COMPANY database.




EMPLOYEE
Fname
Minit
Lname
Ssn
Bdate
Address
Sex
Salary
Super_ssn
Dno

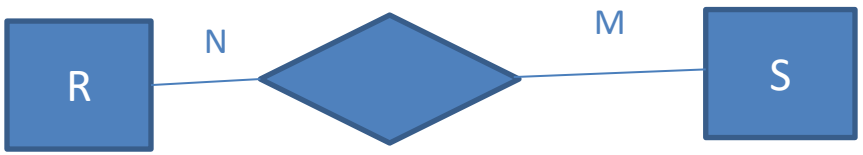
The ER conceptual schema diagram for the COMPANY database.



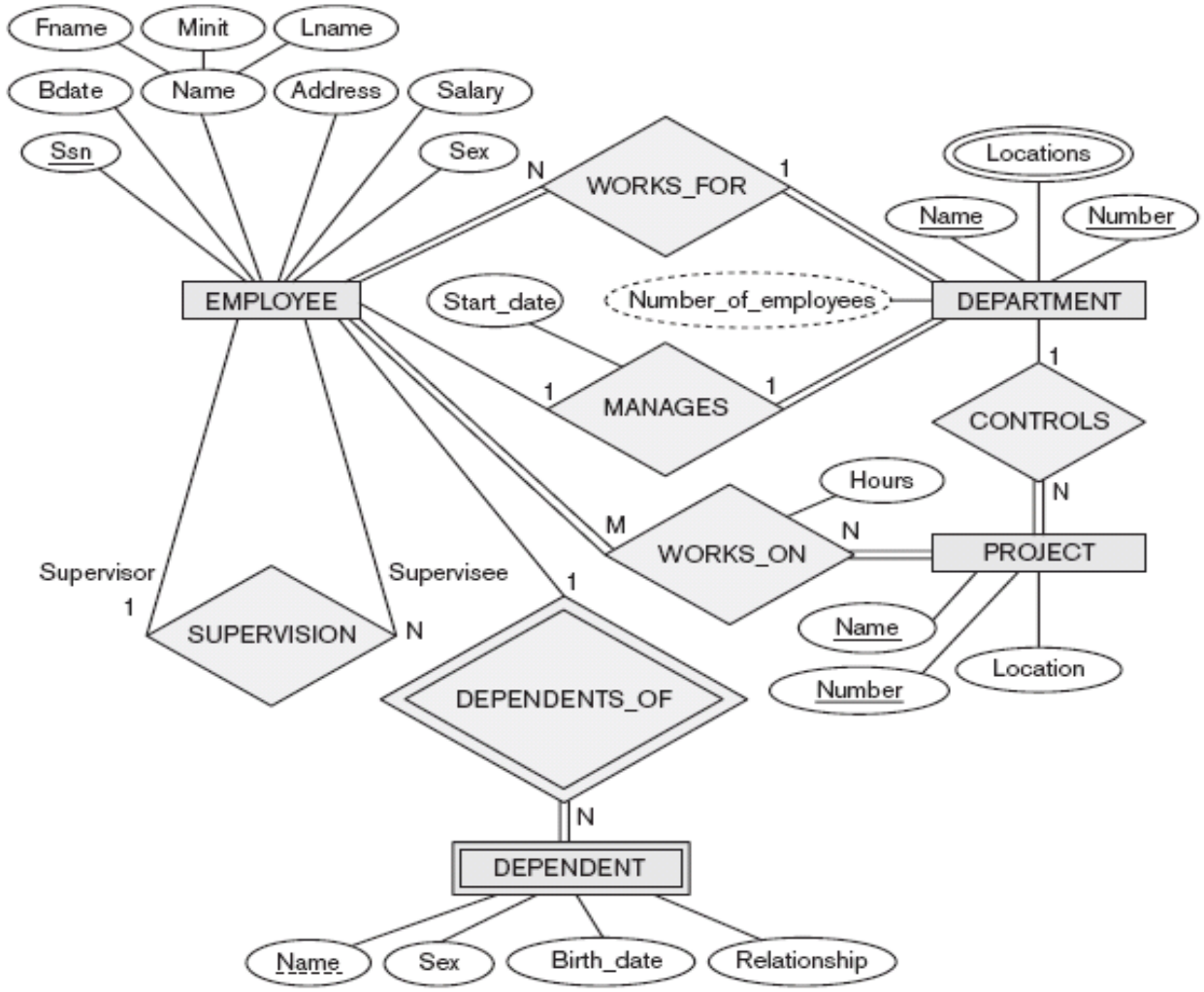

DEPARTMENT
Dname
Dnumber
Mgr_ssn
Mgr_start_date
PROJECT
Pname
Pnumber
Plocation
Dnum

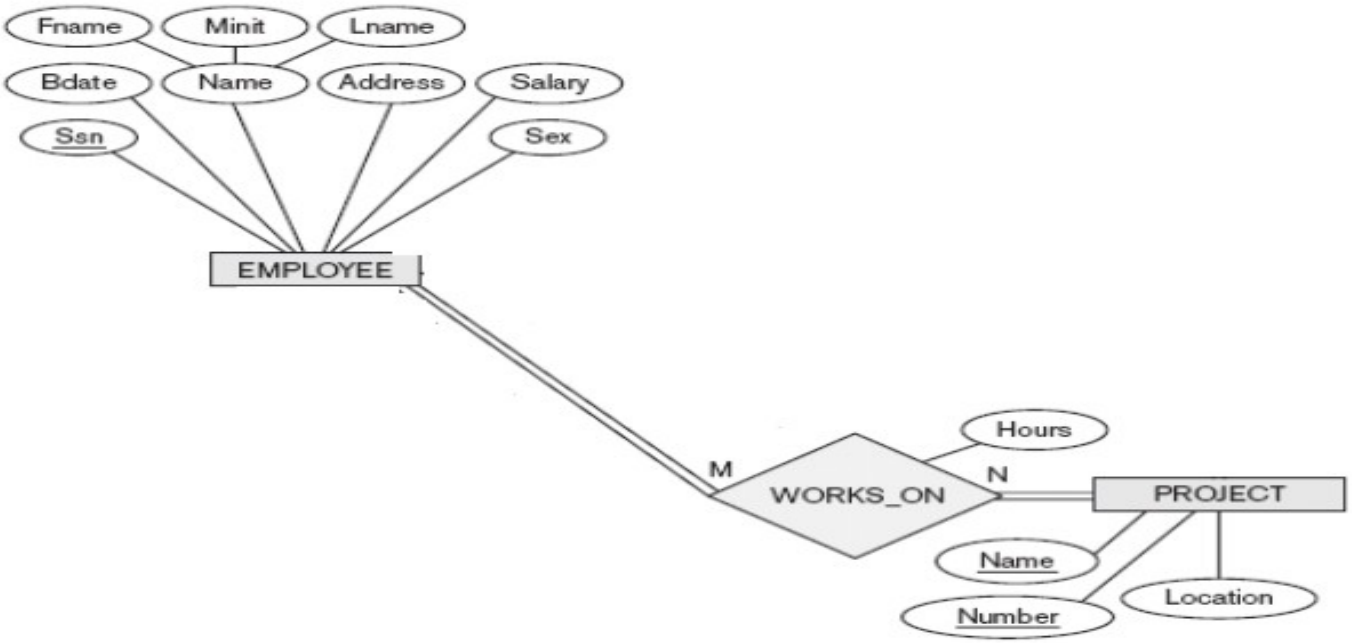
Step 5: M:N Relations

- Identify S and T that represent the connected entity types
- Make a new relation R
- Include:
  - Foreign keys to both entities S and T
  - all (single-valued) attributes
  - as primary key: The combination of the two foreign keys



The ER conceptual schema diagram for the COMPANY database.





EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

PROJECT

Pname	Pnumber	Plocation	Dnum

WORKS_ON

Essn	Pno	Hours

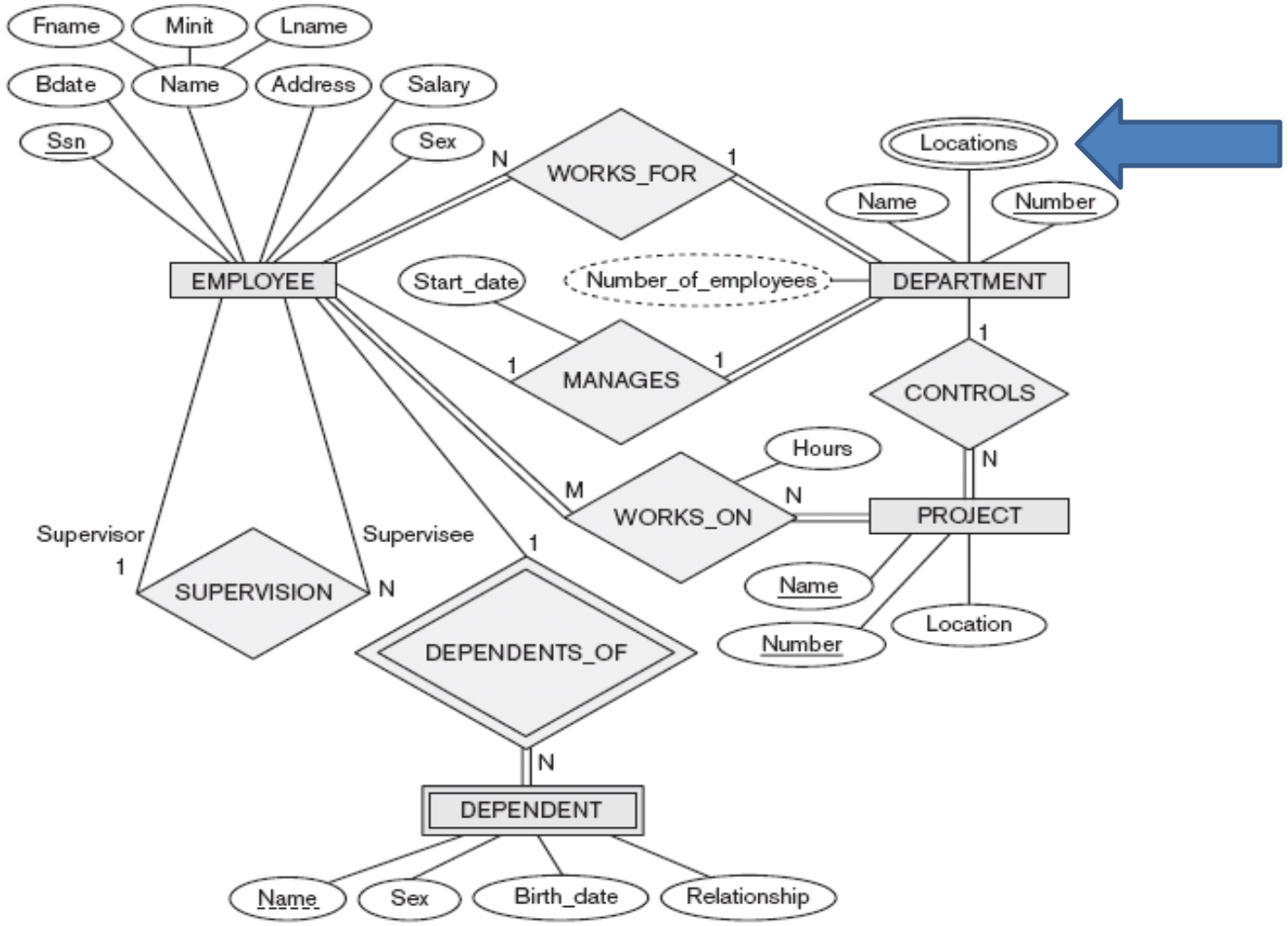
Note

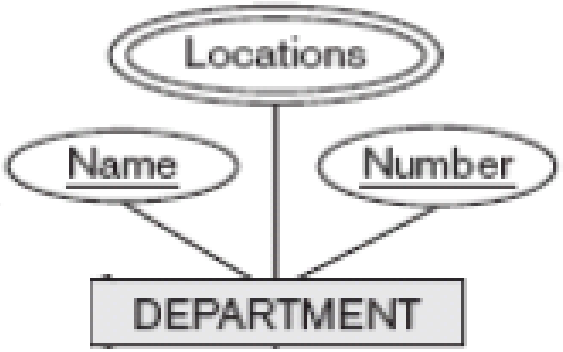
- Note that step 5 **can** be used to model 1:1 and 1:N relations also.
- In that case also define a new relation R for the 1:1 or 1:N relation from S to T
- This is, however, never needed and only useful in rare cases (e.g., when only very few instances of T are related to S, to avoid many NULL values)

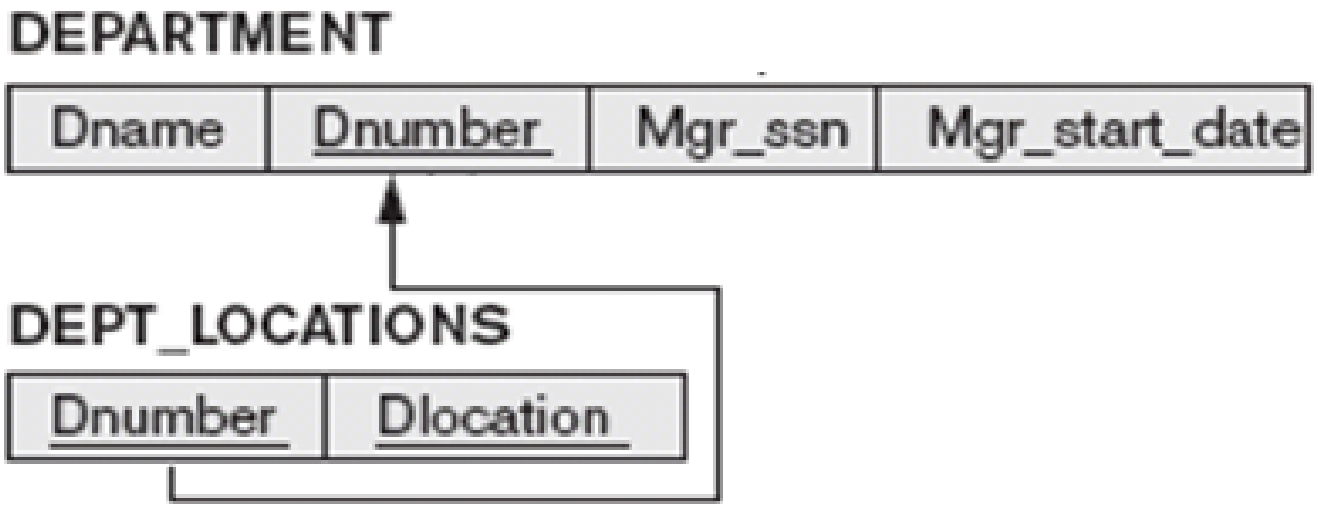
Step 6: multivalued attributes

- Make a relation R for each multivalued attribute A of entity E or relation S.
- Imagine K as primary key of S
- Add to R a foreign key to K
- The primary key of R is the combination of all attributes in R

The ER conceptual schema diagram for the COMPANY database.



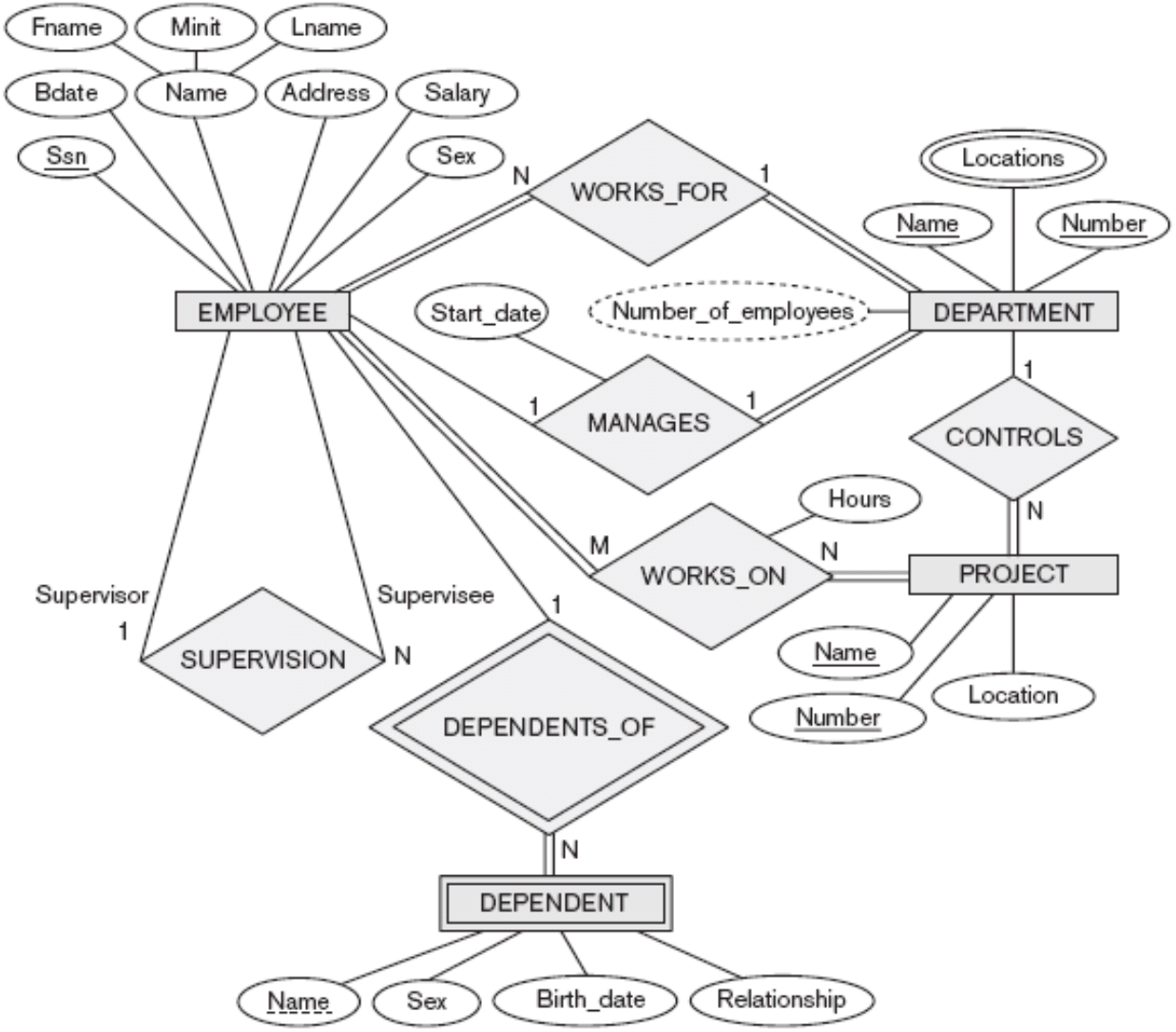

Locations
Name
Number
DEPARTMENT


DEPARTMENT
Dname
Dnumber
Mgr_ssn
Mgr_start_date
DEPT_LOCATIONS
Dnumber
Dlocation

Step 7: Relation types of degree > 2

- Create a new relation R
- Add foreign keys for all participations
- Add single-valued attributes
- Primary key is the combination of foreign keys

The ER conceptual schema diagram for the COMPANY database.



Final relational database schema

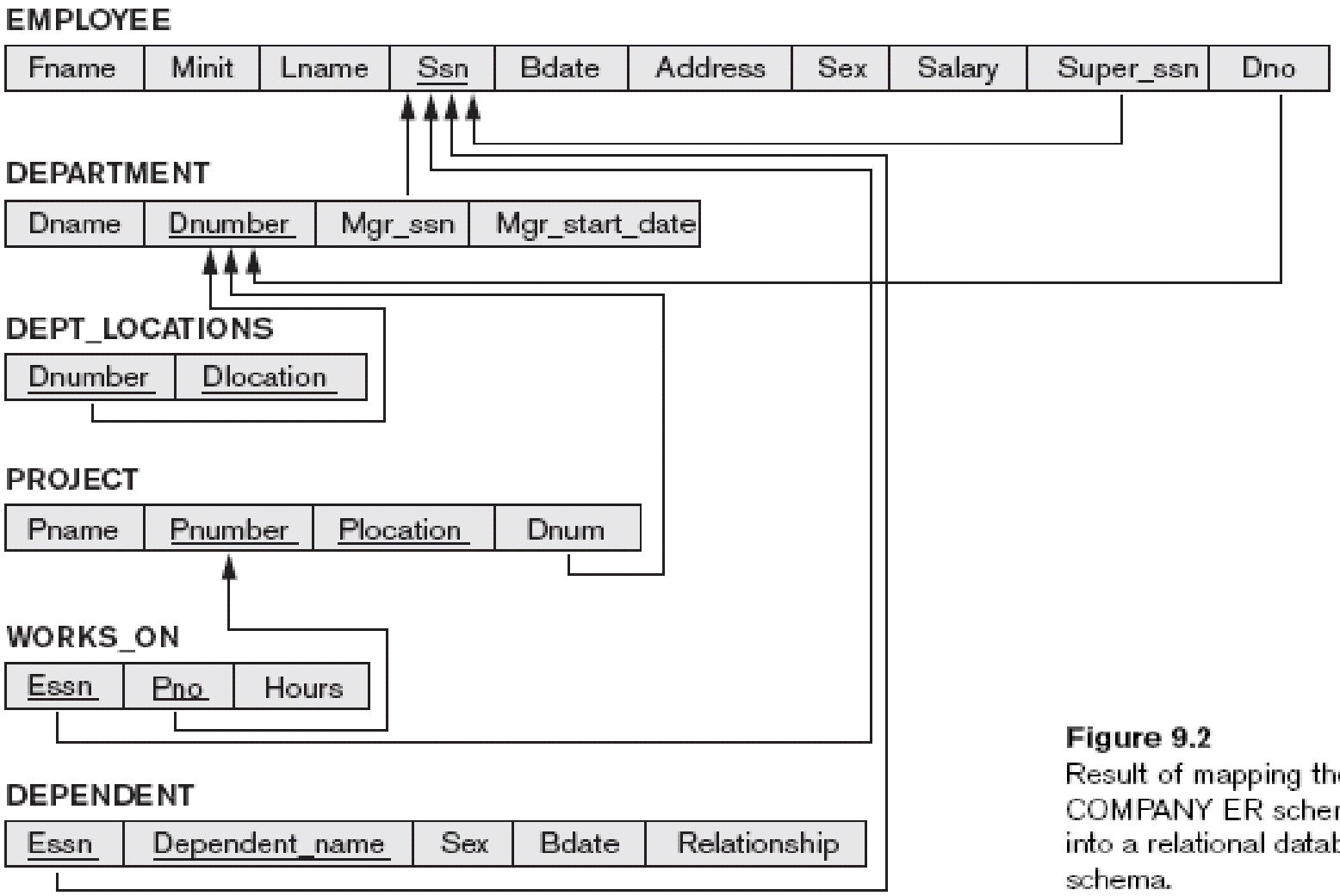


Figure 9.2
Result of mapping the COMPANY ER schema into a relational database schema.

Entity types in ER diagrams

- Until now, each entity was assigned to one entity type
- In many cases an entity type has numerous subgroupings or subtypes of its entities that are meaningful and need to be represented explicitly
- Members of the EMPLOYEE entity type may be distinguished further into SECRETARY, ENGINEER, MANAGER, etc

Subclasses and Superclasses

- Each of these subgroupings is called a **subclass** or **subtype** of the EMPLOYEE entity type, and the EMPLOYEE entity type is called the **superclass** or **supertype**
- Each subclass member is the same as the entity in the superclass, but in a distinct *specific role*

EER to Relational Model

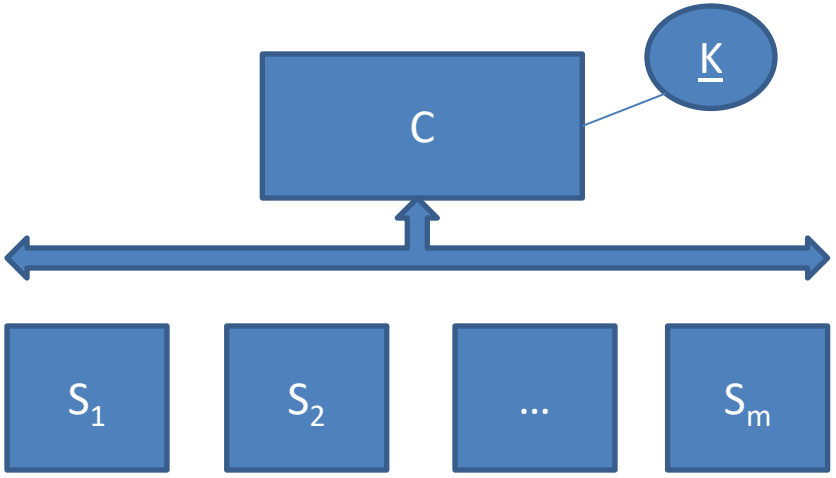
- There are several ways in which sub/superclasses can be translated into relations (see next slides)
- Inheritance of attributes and relations is not automatic and has to be programmed explicitly in a relational DBMS with SQL
- We need a new type of DBMS!

Mapping EER to relation schema

- We extend our 7-step algorithm for transforming an ER diagram into a relation schema with 2 additional steps
- Step 8: mapping of generalizations / specializations (4 options: 8A-8D)
- Step 9: mapping of categories

Step 8: generalization / specialization

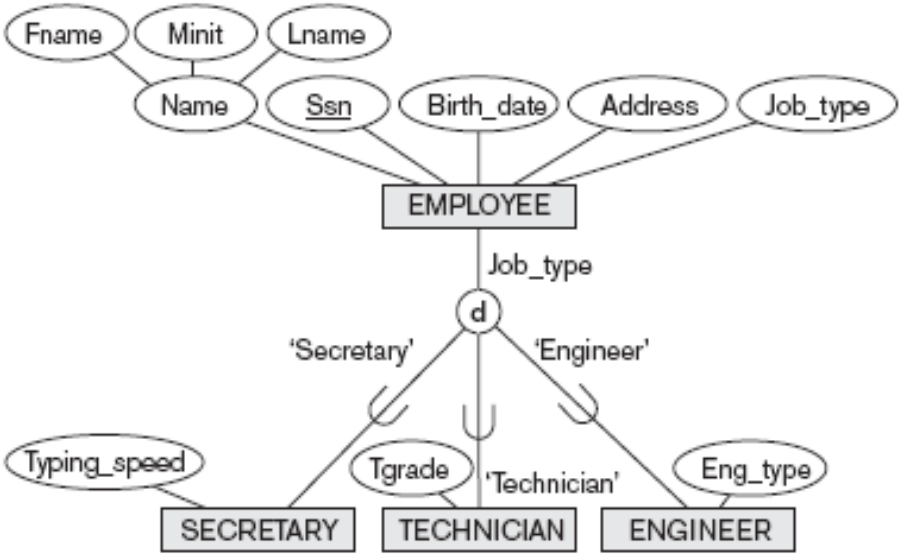
- Convert each specialization with *m* subclasses $\{S_1, \ldots, S_m\}$ and superclass *C*, with attributes $\{k, a_1, \ldots, a_n\}$, *k* being the primary key, into a relation scheme, using 1 of 4 options

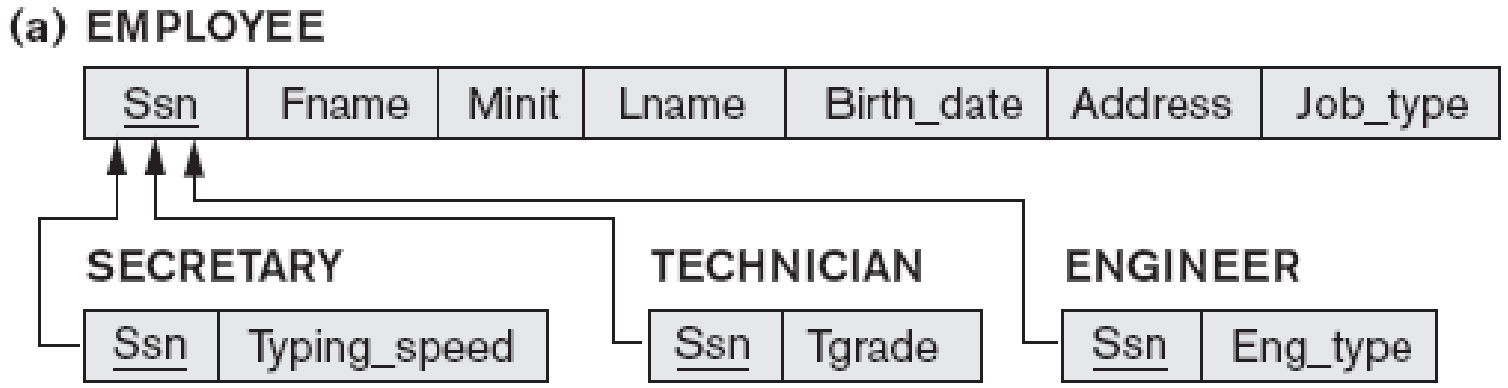


Step 8A: multiple relations – superclass and subclasses

- Create a relation L for C with attributes $\{k, a_1, \ldots, a_n\}$, and PK(L) = k; create a relation L_i for every subclass S_i, with attributes $\{k\}$ U {attributes of S_i}, and PK(L_i) = k
- Works for **any** specialization (total, partial, overlapping, disjoint)
- Example: next slide

Step 8A: example



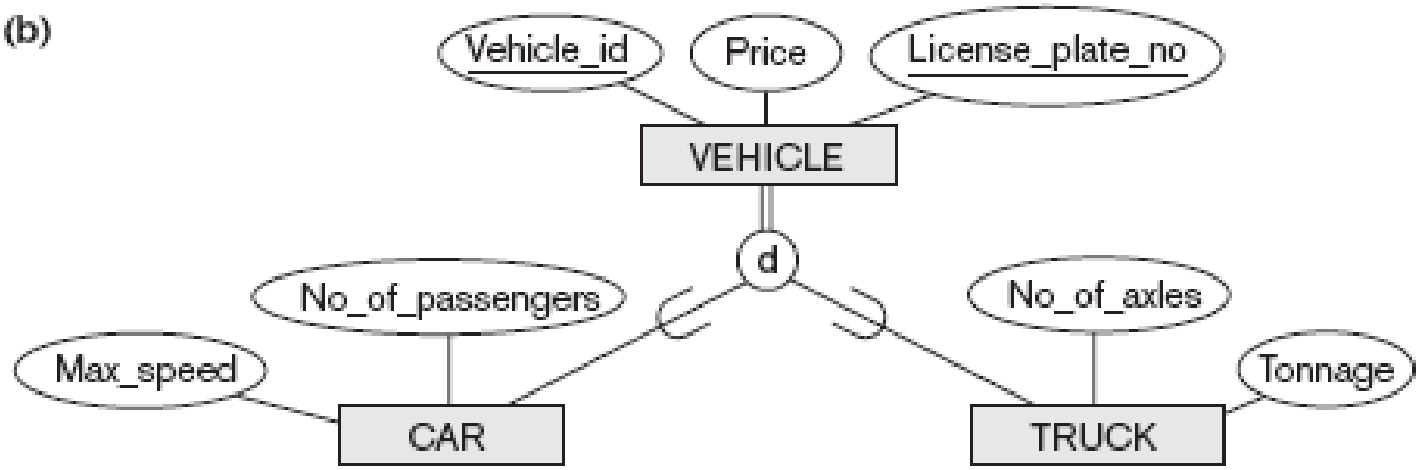


Step 8B: multiple relations – subclasses only

- Like 8A, but don't create a relation for *C*, only for the subclasses S_i, with attributes $\{k, a_1, \ldots, a_n\}$ U {attributes of S_i} and $PK(L_i) = k$
- Only for **total** specialization (otherwise, entries may "get lost"); recommended to be ***disjoint*** (otherwise duplications)
- Example: next slide

Step 8B: example

(b)



(b) CAR

Vehicle_id	License_plate_no	Price	Max_speed	No_of_passengers

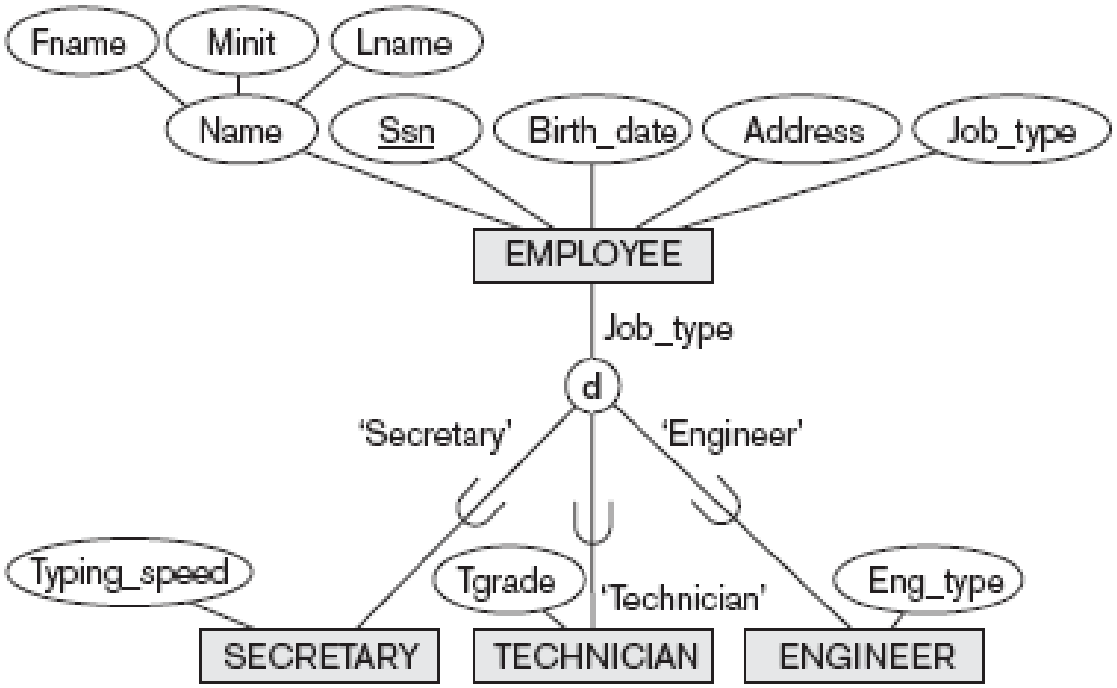
TRUCK

Vehicle_id	License_plate_no	Price	No_of_axles	Tonnage

Step 8C: single relation with 1 type attribute

- Single relation L with attributes $\{k, a_1, \ldots, a_n\}$ U {attributes of S_1} U … U {attributes of S_m} U $\{t\}$, t being a *type* (or *discriminating*) attribute, and PK(L) = k
- The type attribute is only needed if it is not already contained in the superclass
- Only for ***disjoint*** specialization; possibly many NULL values
- Example: next slide

Step 8C: example



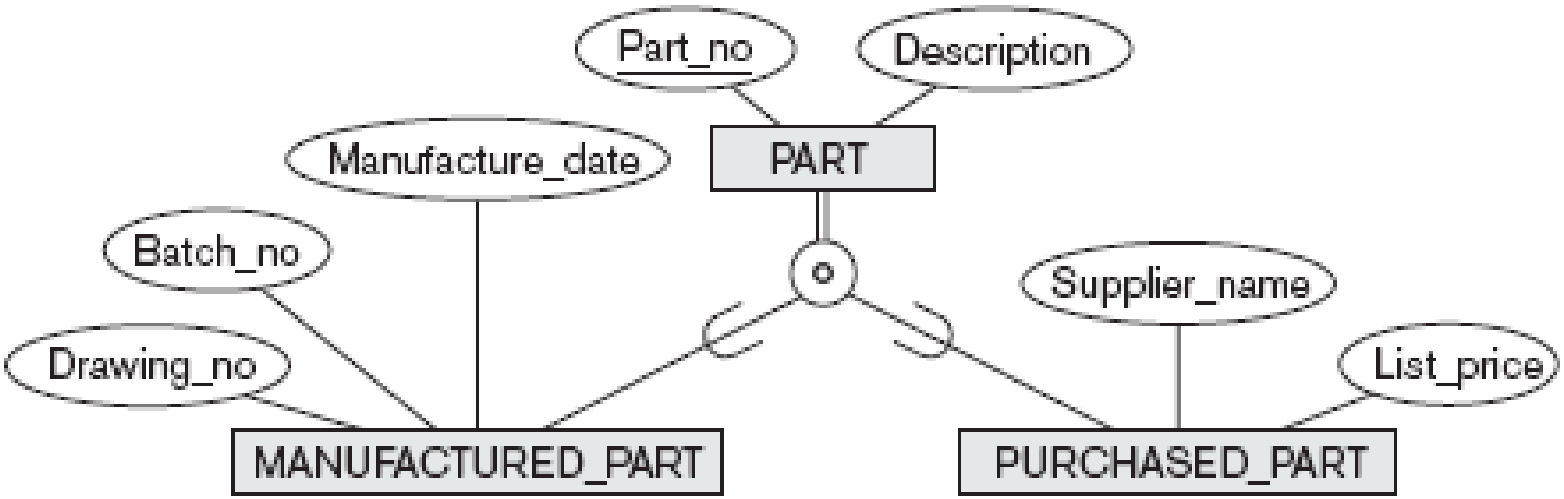
(c) EMPLOYEE

Ssn	Fname	Minit	Lname	Birth_date	Address	Job_type	Typing_speed	Tgrade	Eng_type

Step 8D: single relation with multiple type attributes

- Single relation L with attributes $\{k, a_1, \ldots, a_n\}$ U {attributes of S_1} U … U {attributes of S_m} U $\{t_1, \ldots, t_m\}$, t_i being Boolean *type* (or *discriminating*) attributes, and PK(L) = k
- Useful for **overlapping** specializations (but works also for ***disjoint***);
- Example: next slide

Step 8D: example



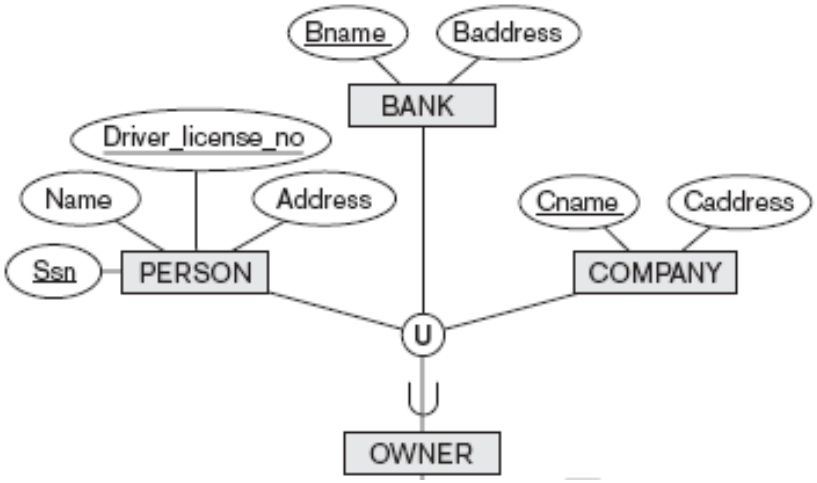
(d) PART

Part_no	Description	Mflag	Drawing_no	Manufacture_date	Batch_no	Pflag	Supplier_name	List_price

Step 9: mapping of union types (categories)

- Generate a special relation with only 1 attribute, the so-called *surrogate* key, for the category, and separate relations for all superclasses.
- Example: next slide

Step 9: example



PERSON

Ssn	Driver_license_no	Name	Address	Owner_id

BANK

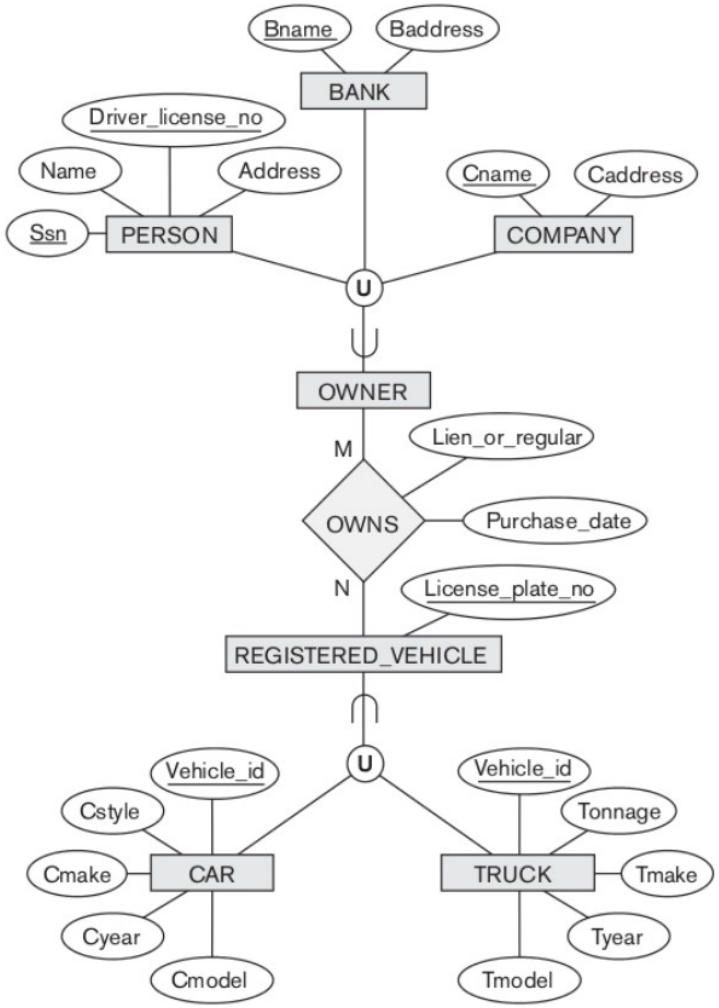
Bname	Baddress	Owner_id

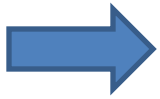
COMPANY

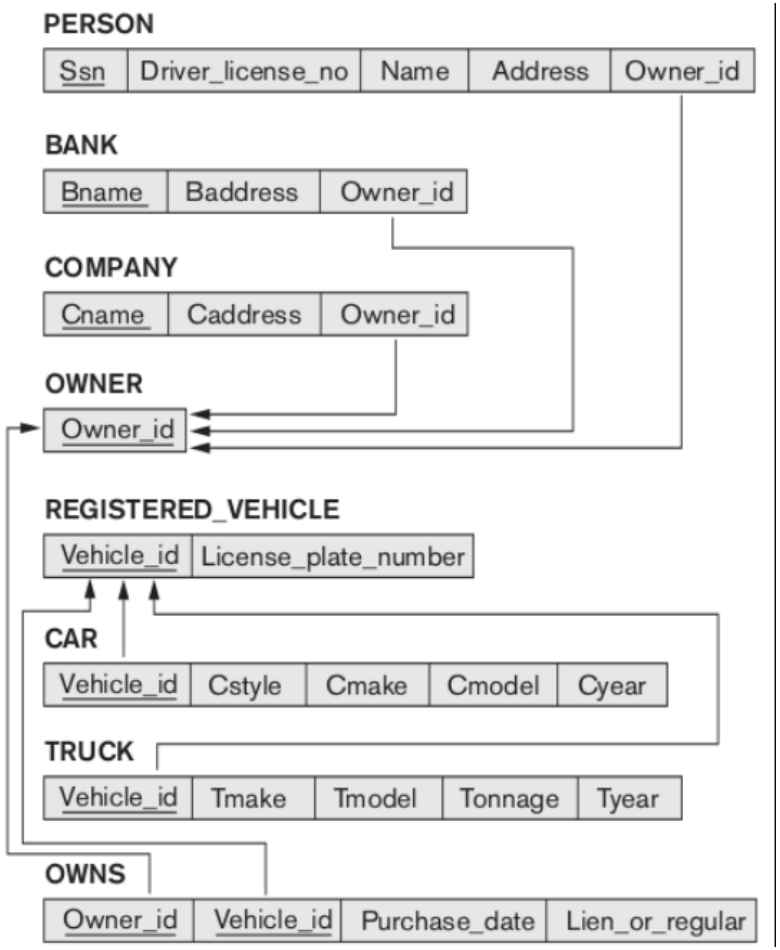
Cname	Caddress	Owner_id

OWNER

Owner_id


Bname
Baddress
BANK
Driver_license_no
Name
Address
Ssn
PERSON
Cname
Caddress
COMPANY
U
OWNER
M
Lien_or_regular
OWNS
Purchase_date
N
License_plate_no
REGISTERED_VEHICLE
U
Vehicle_id
Cstyle
Cmake
CAR
Cyear
Cmodel
Vehicle_id
Tonnage
TRUCK
Tmake
Tyear
Tmodel




PERSON
Ssn
Driver_license_no
Name
Address
Owner_id
BANK
Bname
Baddress
Owner_id
COMPANY
Cname
Caddress
Owner_id
OWNER
Owner_id
REGISTERED_VEHICLE
Vehicle_id
License_plate_number
CAR
Vehicle_id
Cstyle
Cmake
Cmodel
Cyear
TRUCK
Vehicle_id
Tmake
Tmodel
Tonnage
Tyear
OWNS
Owner_id
Vehicle_id
Purchase_date
Lien_or_regular

Notes

- Steps 8A-8D/9 may be intermixed where appropriate
- For a more elaborated way of handling generalization / specialization, and especially (multiple) inheritance, we might need a more suited DBMS system, like an object-oriented DBMS (OODBMS).

Ready? No!

- The schema has to be refined:
  - Entities with 1-to-1 relations might be joined into one table
  - High-degree relation-types might be included in a table
  - If there are no keys for a strong entity type, a identification code has to be invented
  - Domain restrictions have to be recorded
  - Candidate keys have to be appointed (using UNIQUE).

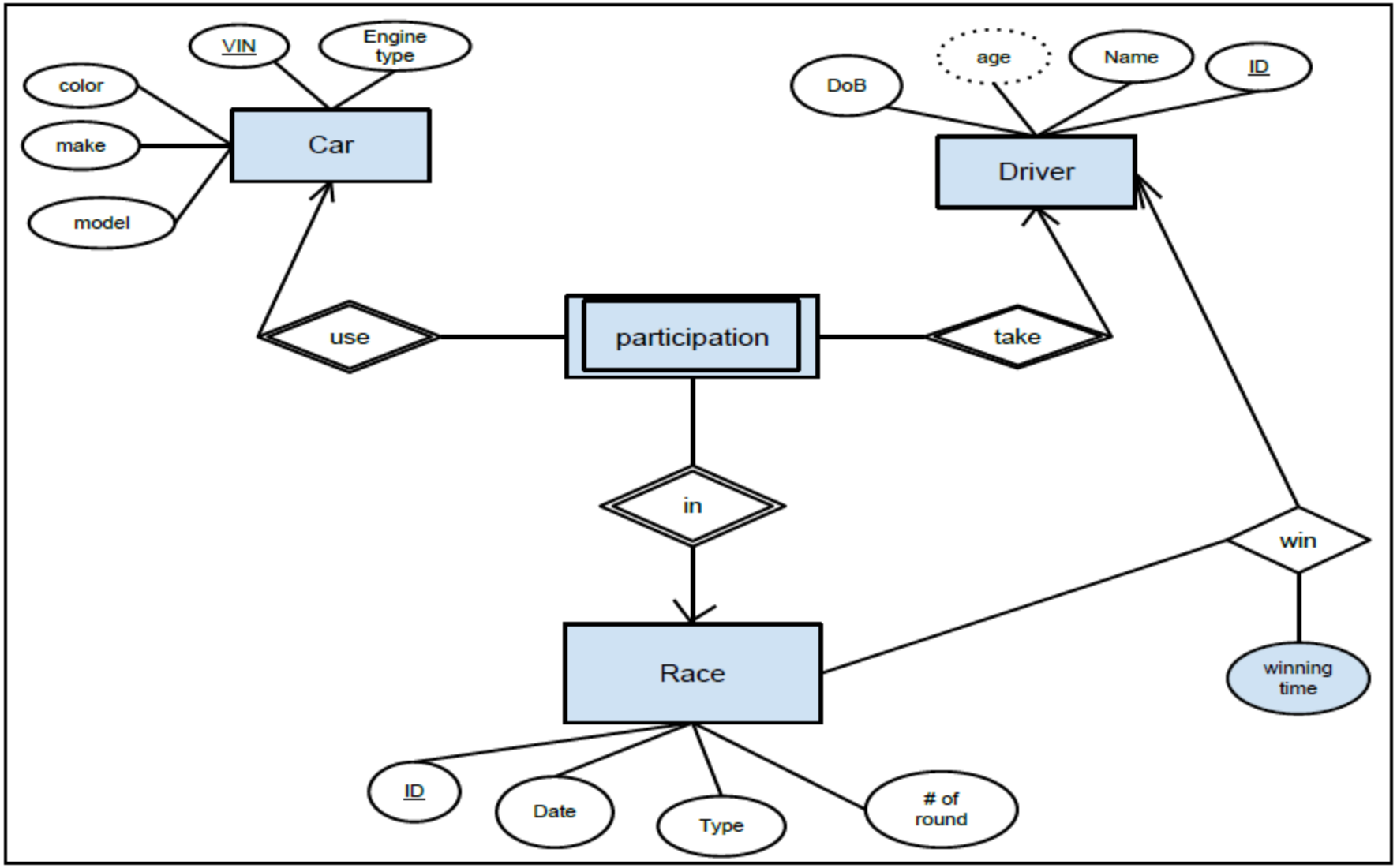
47

And...

- Set cascade on delete and update for foreign keys of:
  - weak entity types
  - M:N and higher-degree relations
  - Multivalued attributes
- Take over extra restrictions

Quiz
(Car Race Database)

- Convert the following ER model to Relational Model.


VIN
Engine type
color
make
model
Car
age
Name
ID
DoB
Driver
use
participation
take
in
win
Race
winning time
ID
Date
Type
# of round